ELECTRICAL & EMBEDDED SYSTEMS ENGINEER

Xiyuan (Amy) Liu

I design, bring up and verify hardware: mixed-signal PCBs, avionics display electronics, FPGA logic and the embedded firmware that ties it together.

Harvey Mudd College · B.S. Engineering (ECE), expected May 2027 · San Jose, CA

xiyuanliu-amy-blog.netlify.app | amyliu01@g.hmc.edu | LinkedIn | GitHub



6
avionics display panels taken to production release

450+
hardware requirements authored in DOORS

35
formal proofs written for a RISC-V quantum controller bridge

1st
place at FAR Unlimited 2026 as rocketry avionics lead

SKILLS

Hardware
PCB schematic & layout, mixed-signal & RF, SI / EMI debugging, bring-up, DO-160G / MIL-STD qualification

Digital & embedded
SystemVerilog, FPGA, embedded C, ARM Cortex-M, RISC-V, UVM & formal verification, JTAG, Python

Tools
Altium, Xpedition, KiCad, LTspice, Questa, Verilator, SymbiYosys, MATLAB / Simulink, ANSYS, DOORS

Lab
Oscilloscope, VNA, logic analyzer, DMM

RECOGNITION

- RTX Display Department Outstanding Employee, Certificate of Excellence (2026)
- Passed FE Electrical & Computer Engineering (EIT), 2026
- 1st, 3rd and 5th at FAR Unlimited (2026, 2025, 2024)
- 3rd place, HMC ICPC qualifier (2022, 2023)

IN THIS PORTFOLIO

- RTX Collins Aerospace: avionics display hardware co-op
- HRL Laboratories: RISC-V quantum controller verification
- Rocketry avionics lead, plus clinic and research work
- Full course project write-ups: FPGA/MCU systems, RF, processor design, filters, control

FULL PORTFOLIO ONLINE

The full write-ups follow in this PDF. Every title also links to its live page, which has the videos and interactive demos.

xiyuanliu-amy-blog.netlify.app

Contents

Each entry is the full page from my portfolio site. Click a title to open it online, where videos and demos play.

EXPERIENCE

RTX Collins Aerospace 3

HRL Laboratories 7

Rocketry avionics (MARC) 16

Physics of Soft Matter Lab 24

USDA × Auburn clinic 26

MICROPROCESSOR SYSTEMS (E155)

E155 final: Interactive 2D Physics Engine 28

E155 Lab 1: FPGA Bring-up & Seven-Segment Decoder 30

E155 Lab 2: Multiplexed Seven-Segment Display 33

E155 Lab 3: Keypad Scanner 37

E155 Lab 4: Digital Audio Player 42

E155 Lab 5: Motor Speed with Interrupts 45

E155 Lab 6: IoT Temperature Sensor & Web Server 47

RF CIRCUIT DESIGN (E157)

E157 Lab 2: Reflections on Transmission Lines 51

E157 Lab 3: VNA Calibration & L-Match 55

CIRCUIT DESIGN

E85 Lab 10: Multicycle Controller 60

E85 Lab 11: Datapath & Full Processor 64

E101: Analog & Digital Filters 69

SIGNAL PROCESSING & CONTROL

E102: Digital Controller for an RC Circuit 75

INDUSTRY

RTX Collins Aerospace

Electrical Engineer Co-Op

June 2025 – August 2026 · Avionics SBU, display products

Hardware design, bring-up, verification and qualification for avionics display control panels. Helped take six panels through final production release to the customer.

Altium | DO-160G | MIL-STD | EMI | DOORS | JTAG / boundary scan

HIGHLIGHTS

- Designed a level-shifted JTAG adapter in Altium for Ethernet-card programming and boundary-scan debug. It released first pass with no re-spin, and I led its bring-up, documentation and qualification.
- Ran DO-160G and MIL-STD environmental qualification with the test technicians. Diagnosed a radiated-emissions failure and fixed it with series damping on a high-speed clock, recovering three weeks of schedule.
- Authored 450+ hardware requirements and 300+ verification test cases in DOORS, keeping system-to-hardware traceability through five formal PREP reviews.
- Modified a circuit card layout and rerouted 10+ signal lines so two display units could undergo proton-radiation testing of a replacement flash memory.
- Wrote a PCB quick-turn guide defining design constraints for faster prototyping with less downstream redesign.
- Received the Display Department Outstanding Employee "Certificate of Excellence" (2026).

The role

I joined the avionics display group as a co-op and worked across the hardware lifecycle of cockpit display control panels: design changes, first-article bring-up, verification against requirements, and environmental qualification. Over the co-op, six panels moved through final production release to the customer.

Replacing a flash memory, and proving it survives radiation

The panels store their display firmware and configuration in a parallel NOR flash that had gone obsolete, so it needed a replacement. At cruise altitude, high-energy neutrons can flip a bit in memory (a single-event upset) or push a chip into a high-current latch-up that forces a restart. A flipped bit in RAM is usually caught by error correction, but a flipped bit in flash stays there through power cycles. On a cockpit display that could mean corrupted characters or a mislabeled altitude reading that only shows up weeks later.

Proton beam testing exposes exactly that kind of failure, so the customer required it. I modified a circuit card layout and rerouted more than ten signal lines so two display units could be instrumented for the test. The goal was to confirm that the new part doesn’t produce persistent upsets, and that the firmware recovers correctly when one happens.

Chasing an EMI failure

During DO-160G qualification, a unit kept failing radiated emissions at 150 MHz, which lined up with the second harmonic of a 74 MHz clock. I stepped up the series resistor at the clock driver to slow its edges, checked the before and after waveforms on an oscilloscope, and the unit passed. Then came the harder question of which earlier test results still held after the change. Slower edges are worst in the cold, so we reran the low-temperature operating test and kept the rest. That recovered about three weeks of schedule.

Rebuilding a JTAG adapter

The AFD-3010E Ethernet video card needed a new adapter for programming and boundary-scan testing. Before choosing any termination, I traced the real direction of every signal to find where each line actually starts and ends. That decided where series damping and far-end termination belonged, and ruled out terminating both ends at once, which can

overdrive the chip. I designed the level-shifted board in Altium. It worked on the first spin, and I led its bring-up, documentation and qualification.

Untangling an old emissions test

One audio-frequency emissions test had a mystery box in its setup. It appeared in old test reports, its contents had changed over the years with no clear record, and the procedure drawing didn't quite agree with the requirements. Going back to the DO-160 standard, I found two separate test methods, one for a single wire and one for a bundle, that the procedure had blended together. We had a bundle, so we used that method, redlined the procedure with the technicians and engineers, and swapped the box for a clear one so future reports show exactly which parts were used.

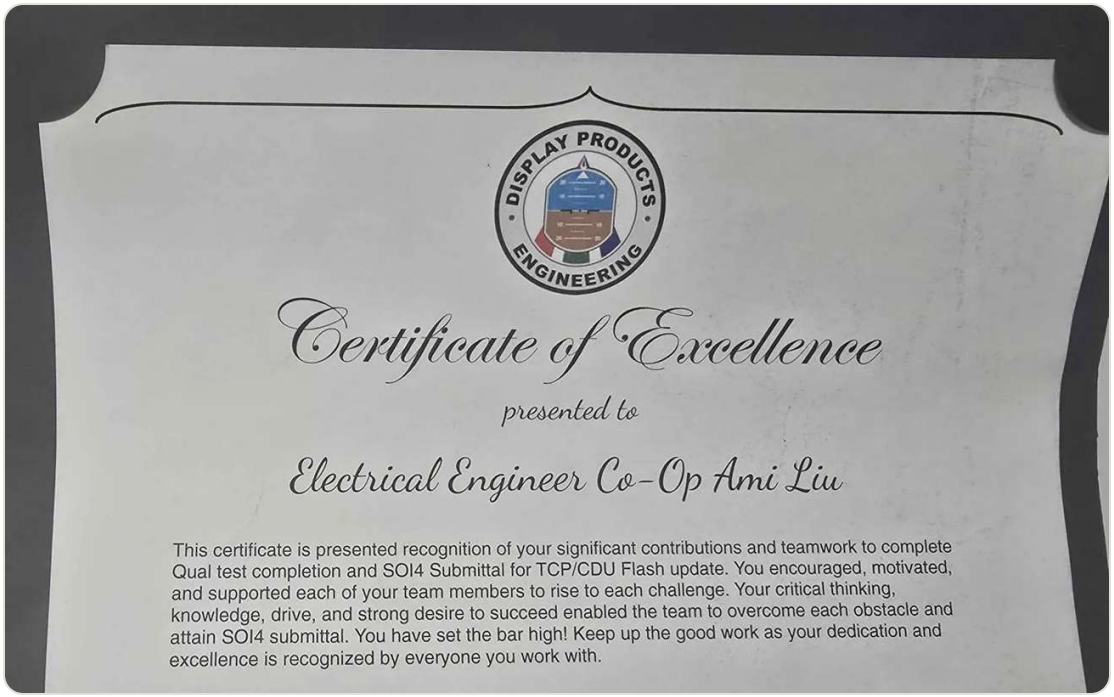
Smaller pieces

- **Requirements:** 450+ hardware requirements and 300+ verification test cases in DOORS, traced through five formal PREP reviews, plus Service Bulletin drafts for the technical writers.
- **Gasket model:** an analytical model of gasket deformation used to set geometric constraints.
- **Quick-turn PCB guide:** a guide for prototype boards, written with input from layout, component and manufacturing engineers so that what works as a prototype carries over to production without a redesign.

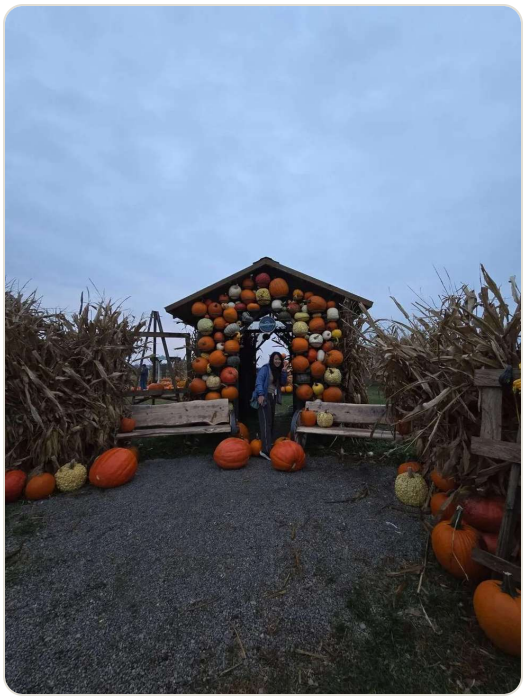
Recognition

In March 2026 the Display Products Engineering group gave me its Certificate of Excellence for helping the team finish qualification testing and submit the flash-update program for certification review.

> "Amy's willingness to challenge opinions when something doesn't make sense showcases her critical thinking and commitment to delivering high-quality work."
>
> From my manager's 2025 year-end review



Certificate of Excellence from Display Products Engineering, Collins Aerospace



At a company Halloween event

CLINIC

HRL Laboratories

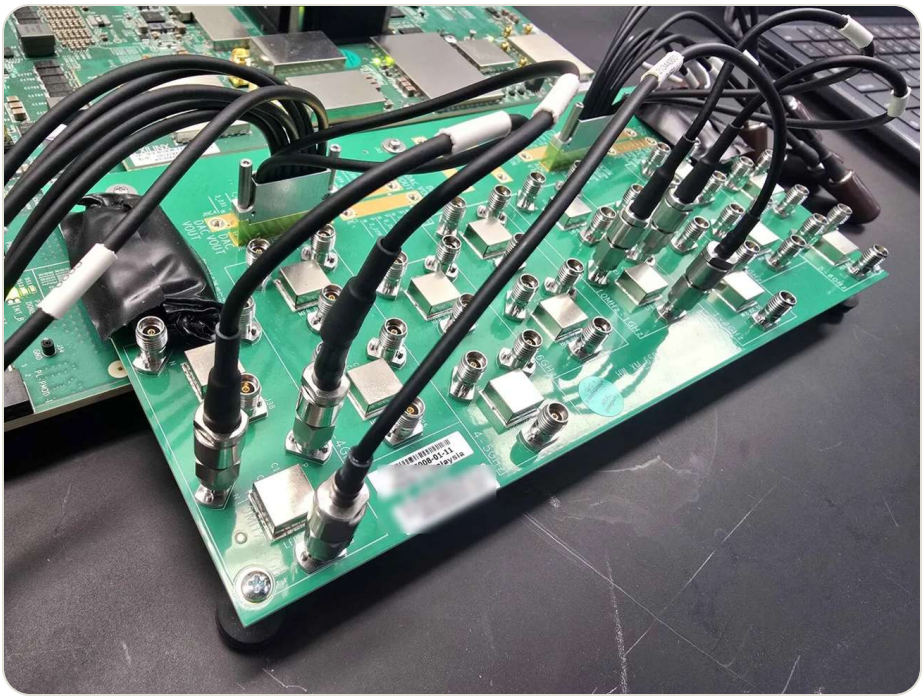
Electrical & Embedded Systems Engineer, Verification

September 2026 – present · Harvey Mudd Engineering Clinic

Replacing QICK's proprietary tProcV2 timed processor with a RISC-V controller generated by the RISC-Q framework, so quantum control firmware can be written with standard RISC-V tools. I designed the team's verification plan and built its verification flow.

SymbiYosys | Formal verification | SystemVerilog | Verilator | RISC-V | Xilinx RFSoC

Team repository ↗

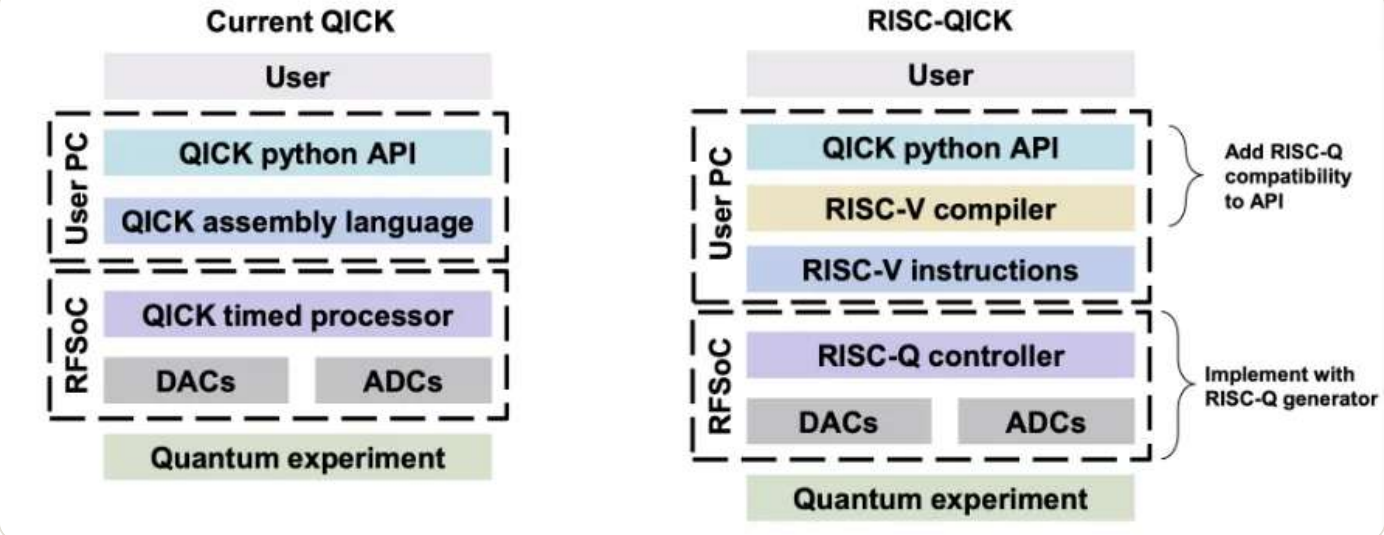


HIGHLIGHTS

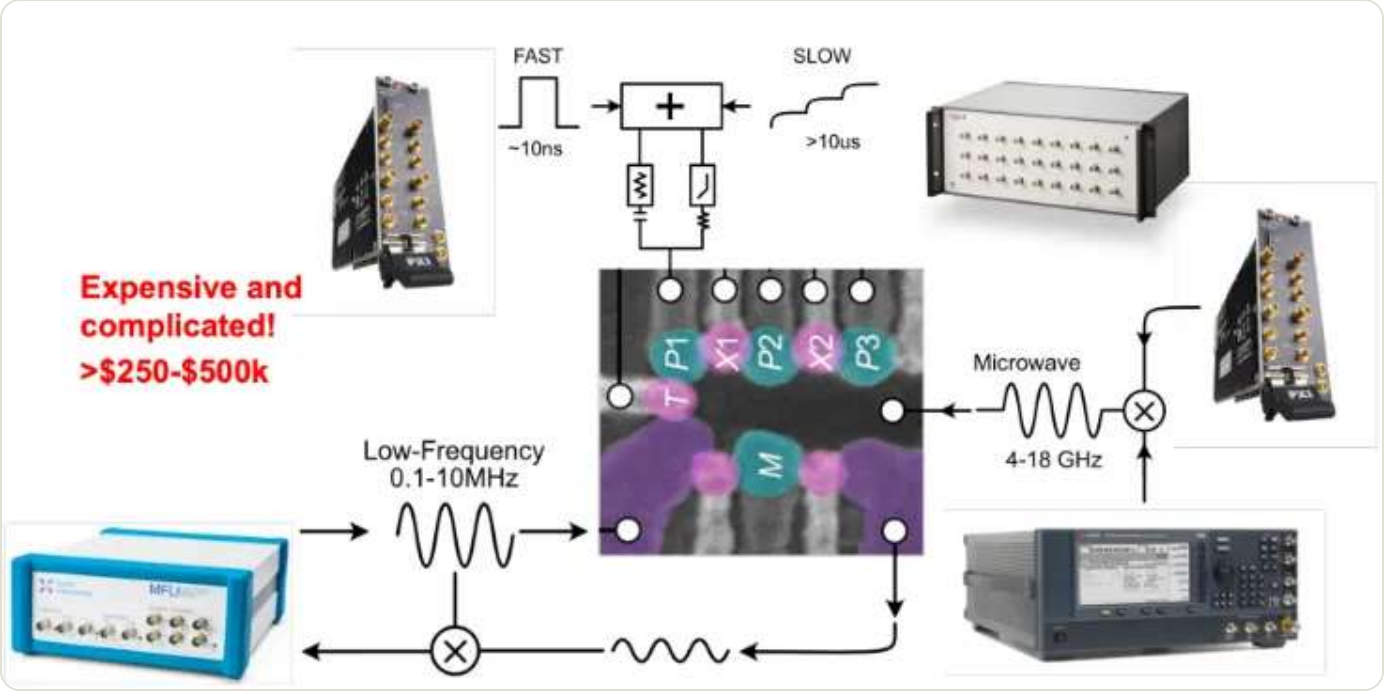
- Designed the team's verification plan, a UVM-style differential flow with tProcV2 as the golden reference, plus legality checks and formal stress tests.
- Built the QICKEmu-compatible Verilator environment and the controller-level control-signal comparison between QICK and RISC-Q.
- Built always-on timing monitors at 7 pipeline checkpoints across 3 clock domains, and characterized 322 on-time pulses with jitter under one generator clock.
- Wrote 35 SymbiYosys formal tasks for RISC-Q's WaveWordBridge. They proved exact release timing and ordering, and exposed three ways pulses could be silently lost or truncated.

The project

QICK is an open-source platform for controlling quantum experiments on Xilinx RFSoC boards. It replaces racks of lab instruments that can cost $250k–$500k with a single board. Its current timed processor, tProcV2, makes firmware development difficult: it uses a proprietary instruction set, so users have to learn QICK's custom processor architecture before they can write a program.

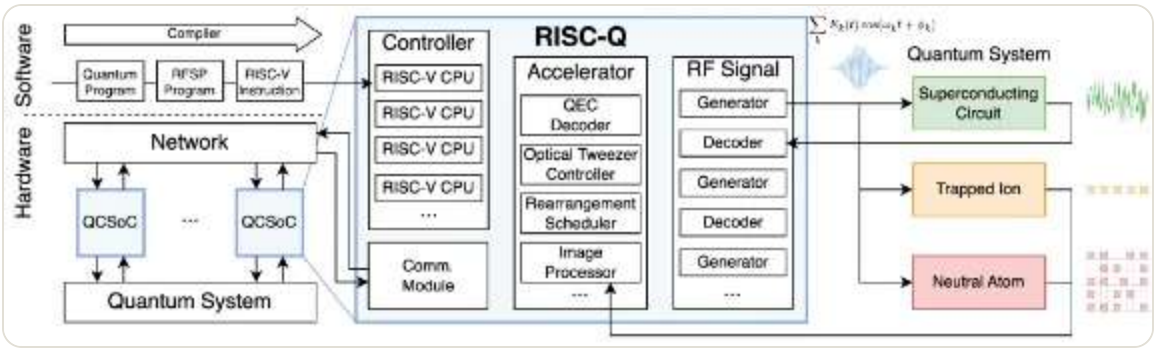


Current QICK stack compared with the RISC-QICK stack, where a RISC-V compiler and RISC-Q controller replace the QICK assembly language and timed processor



A traditional qubit control setup built from separate instruments. Diagram: HRL Laboratories

Our Harvey Mudd clinic team, sponsored by HRL Laboratories, is replacing tProcV2 with a RISC-V-compatible controller generated by the RISC-Q framework. An open, widely supported instruction set lowers the barrier for researchers and engineers who already know RISC-V tools. The new controller has to keep QICK's existing signal generators, readouts and Python API, while supporting compiled RISC-V programs, sub-nanosecond pulse timing, and the loops, branches and arithmetic that quantum control sequences need.



The RISC-Q architecture: RISC-V controller cores, accelerators and RF signal generators. Source: RISC-Q, arXiv:2505.14902

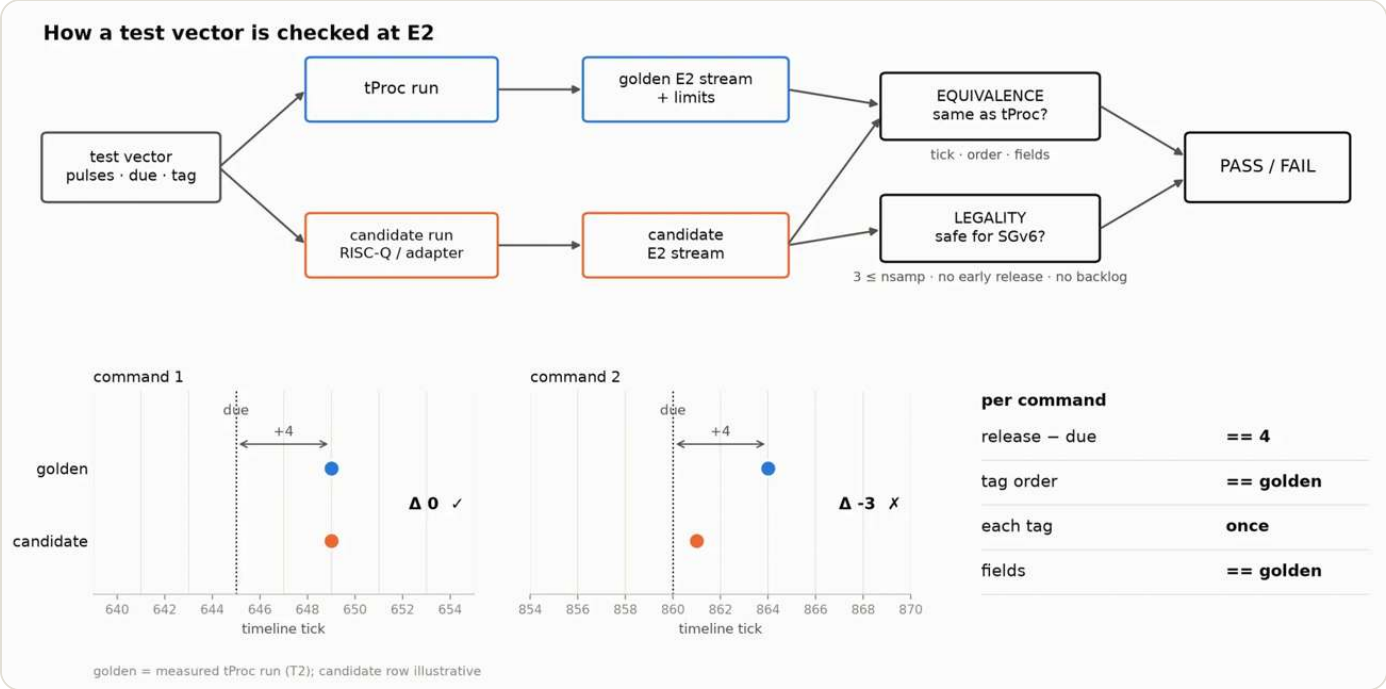
My role: verification

The seven-person team is split into three subgroups: controller and core, software interface, and verification and validation. I'm on verification. Our job is to prove that the RISC-Q controller behaves exactly like tProcV2 wherever the rest of QICK depends on it, and to find out early where it doesn't.

The verification plan

I designed the plan. It borrows UVM's monitor, transaction and scoreboard structure without a full UVM library, and has three layers:

- **Legality:** encode the hardware's assumptions, like valid field ranges and a minimum pulse length, so unsafe commands are caught at the boundary.
- **Equivalence:** differential testing with tProcV2 as the golden reference. The same test vector runs on QICK and on the RISC-Q candidate, and a scoreboard compares *what* each one emits and *when*: release time relative to the due time, tag order, each command exactly once, and every field.
- **Stress and formal:** SymbiYosys on the real RTL explores the corner cases that directed tests miss. For validation, the design is flashed onto a ZCU216 board for capacity, late-command, ordering, skew, restart and long-run tests.

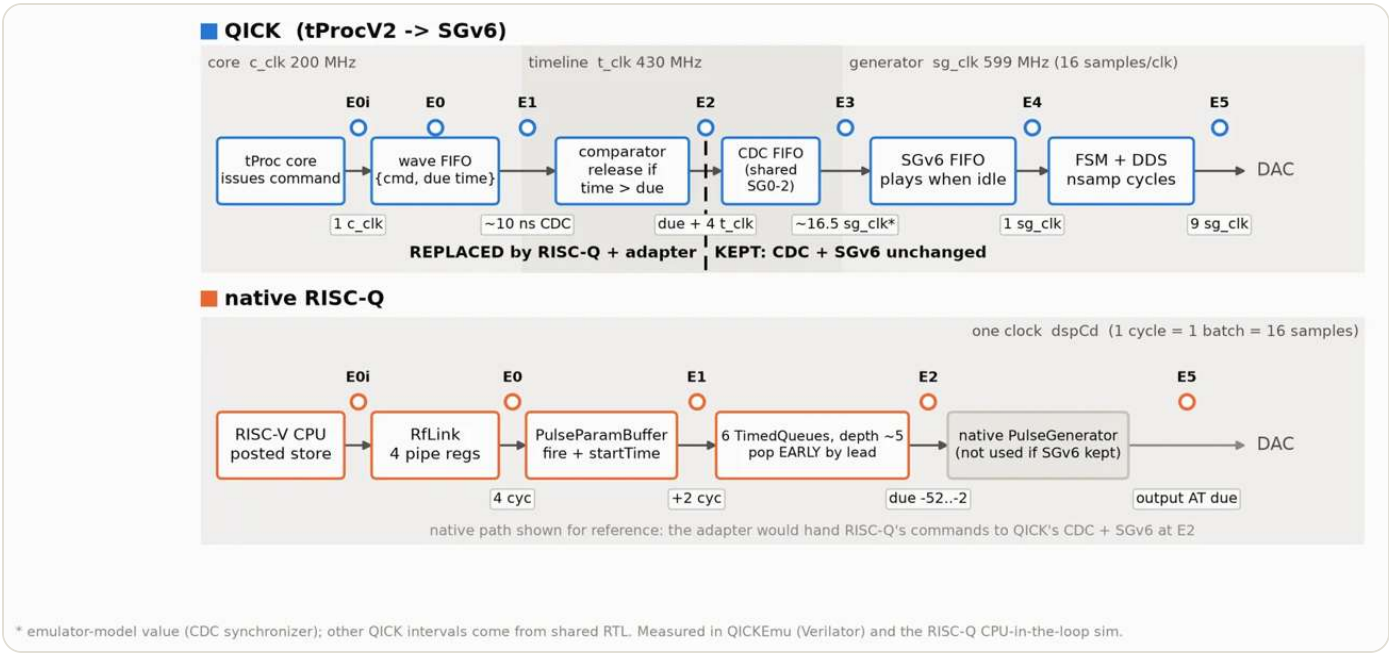


How one test vector is checked: the tProc run produces a golden stream, and the candidate must match it and stay within SGv6's limits

To make that comparison possible, I set up a Verilator simulation environment compatible with QICKEmu, QICK's emulator, and wrote the first controller-level comparison of control-signal outputs between QICK and RISC-Q.

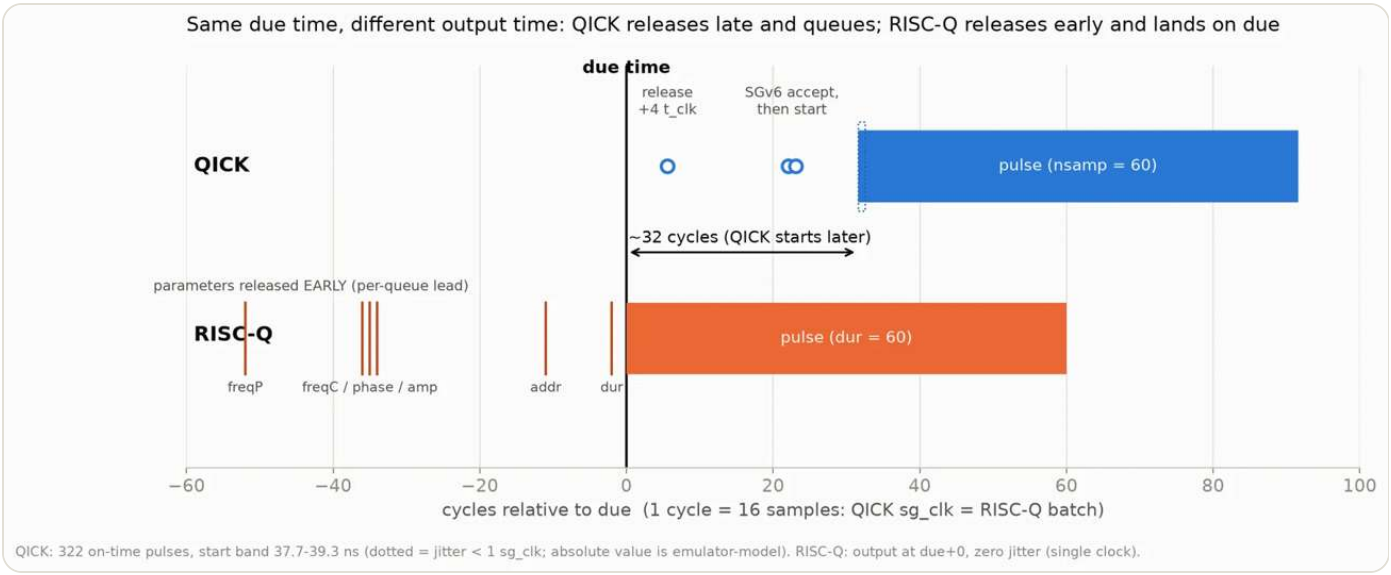
Timing characterization

Timing is the hardest thing to match. A QICK pulse command crosses three clock domains (200, 430 and 599 MHz) and several queues before it reaches the DAC. RISC-Q runs on a single clock and schedules differently. I added always-on monitors to the testbench that log every command at each pipeline event (E0i through E5) in QICK, plus an event logger in RISC-Q's CPU-in-the-loop simulator, so the same events can be compared across both designs.



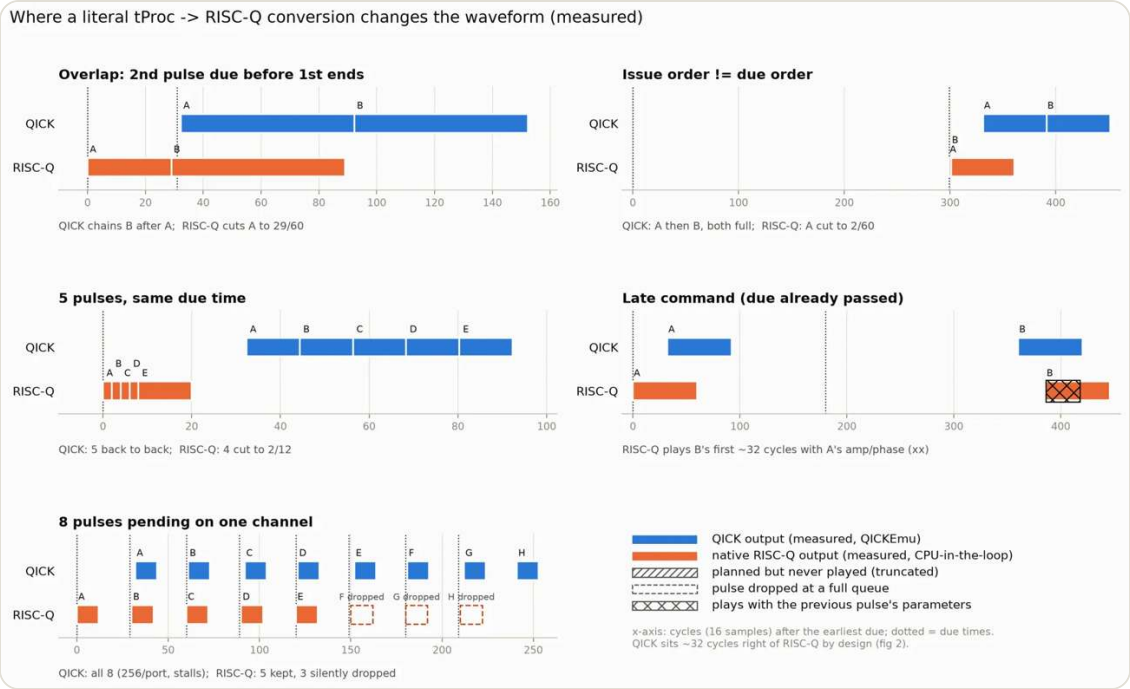
The command pipeline in QICK and in native RISC-Q, with the event checkpoints I instrumented

Across 322 on-time pulses, QICK started each pulse in a narrow band with less than one generator clock of jitter. The measurements also showed a basic difference between the designs. QICK releases a command after its due time and starts the pulse about 32 cycles later. RISC-Q releases pulse parameters early and lands exactly on the due time.



Same due time, different output time: QICK releases late and queues, RISC-Q releases early and lands on due

That difference matters because a literal tProc-to-RISC-Q conversion changes the waveform. Overlapping pulses get cut short, commands issued out of due order get truncated, and with eight pulses pending on one channel RISC-Q keeps five and silently drops three. These cases define what the adapter has to handle.



Measured cases where a literal conversion changes the output waveform

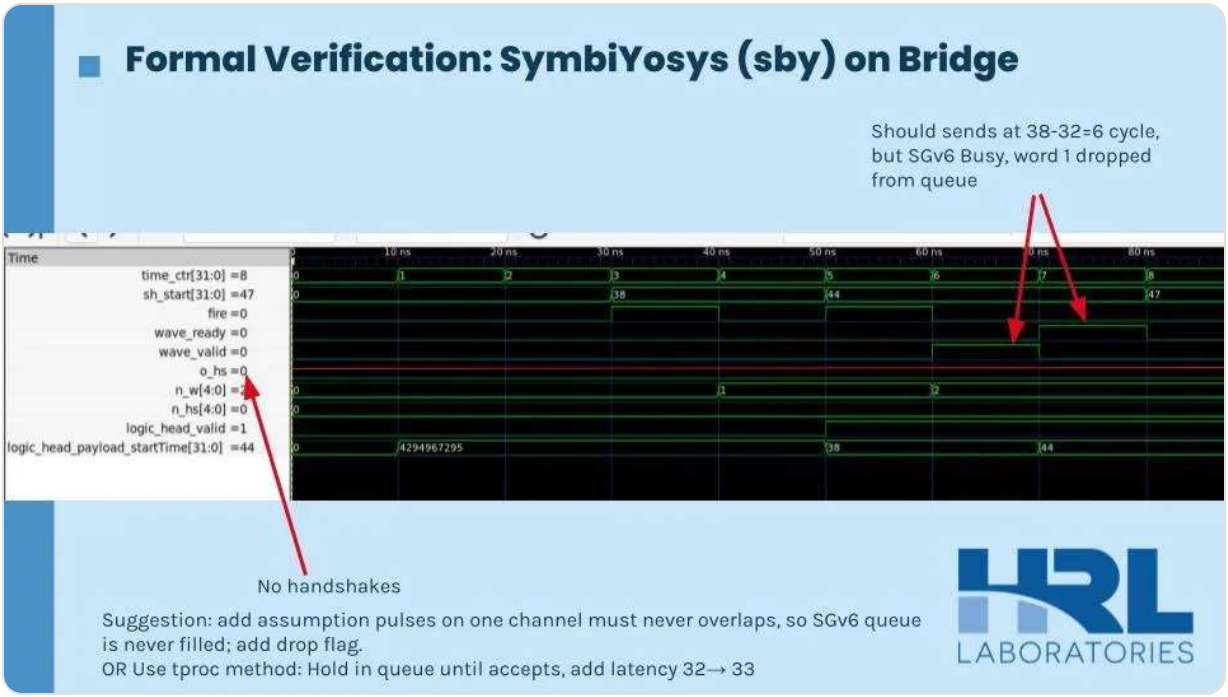
Formal verification of the WaveWordBridge

The WaveWordBridge is the adapter piece that turns RISC-Q's commands into QICK's 168-bit wave words. I wrote 35 SymbiYosys tasks for it, run on the bridge RTL generated with the integration parameters and with QICK's `sg_translator` behind it.

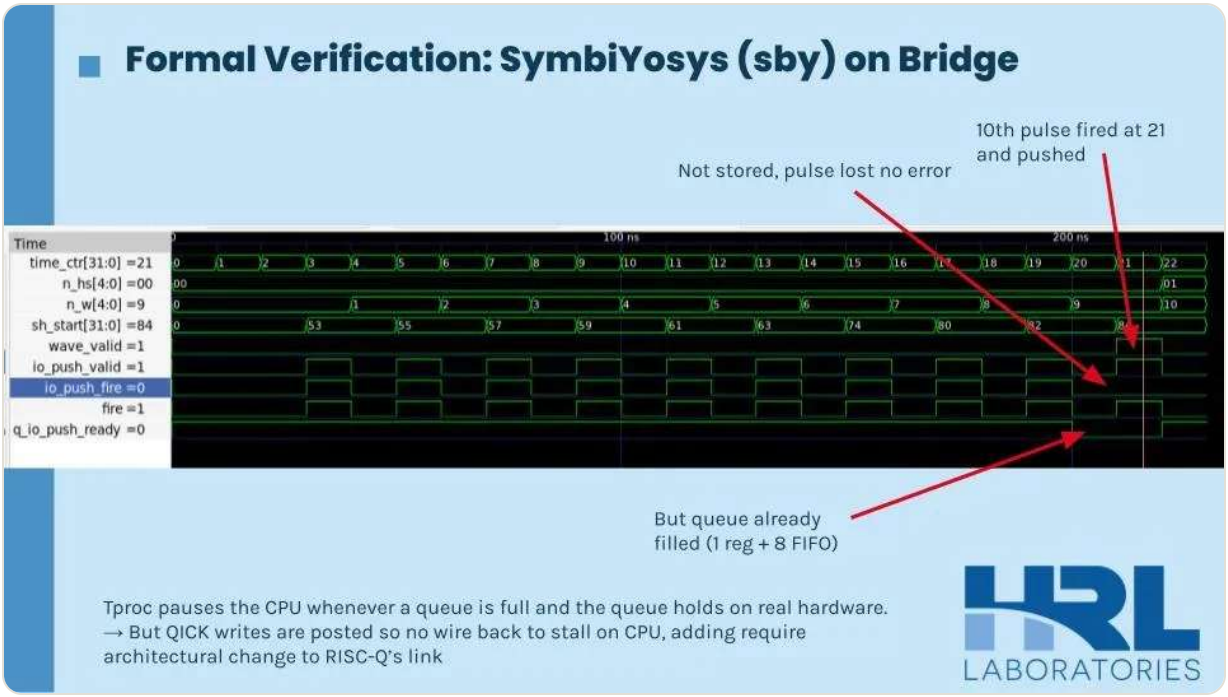
Proven (pass): every word is released exactly at its start time minus 32 cycles and never early. Every word arrives exactly once, in order, with all 168 bits intact. Reset clears pending words, and the signal generator decodes exactly the values the CPU sent.

Measured limits: a pulse must be fired at least 35 cycles ahead (it's one cycle late for each cycle short), and start times must be in order and at least 2 cycles apart.

Found (fail): three ways the bridge can lose or corrupt a pulse with no error:



Counterexample: the signal generator is busy when the bridge releases a word, there is no handshake, and the word is dropped



Counterexample: a 10th pending pulse fires after the queue is full and is lost with no error

- A word is lost when the signal generator isn't ready, because the bridge never checks its ready signal.

- The queue holds 9 pending pulses, and the 10th is silently dropped. tProcV2 avoids this by stalling its CPU when a queue fills, but RISC-Q's writes are posted, so there's no path back to stall the CPU.
- Pulse lengths under 3 and out-of-range fields pass straight through or are silently truncated to 16 bits.

For the dropped words I proposed two fixes to the controller team: either assume pulses on a channel never overlap and add a drop flag, or hold words in the queue until they're accepted, the way tProc does, at the cost of one extra cycle of latency (32 → 33).

Acknowledgements

Thanks to our HRL Laboratories liaisons Abbie Wessels, Paul Jerger and Jessica Liu, our faculty advisor Benson Tsai, and team lead Julia Gong. Diagrams marked HRL Laboratories are from the liaisons' clinic launch slides.

LEADERSHIP

Mudd Amateur Rocketry Club

Avionics Lead

September 2022 – present

Leading avionics for Harvey Mudd's competition rocketry team, which won first place at FAR Unlimited 2026. Now building the team's custom flight computer for its first actively guided rocket.

Embedded C · Sensor fusion · Flight computer · Teaching

MARC on Instagram ↗ · HMC news on the 2025 FAR result ↗

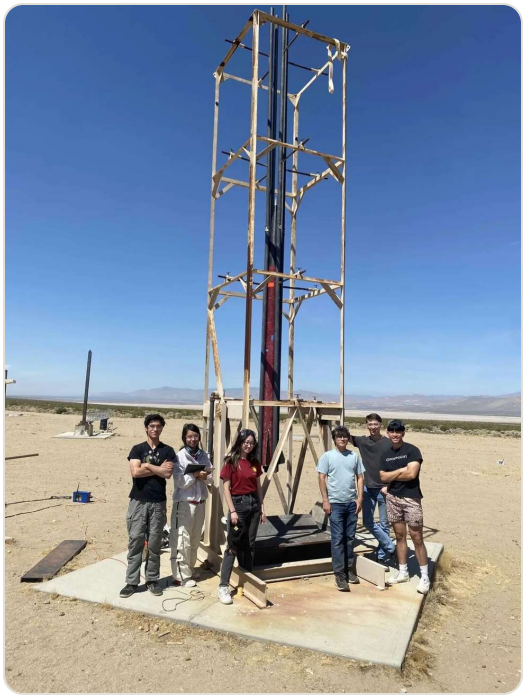


HIGHLIGHTS

- Developing the club's first active-control flight computer driver in embedded C, with redundant sensing for reliable state estimation through high-g saturation during ascent.
- Reduced apogee-detection timing error by about 30% and ran a magnetometer interference study.
- Wrote a Level 1 rocket design guide for beginners and led build sessions for 60+ students.
- Team placed 1st, 3rd and 5th at Friends of Amateur Rocketry Unlimited contests (2026, 2025, 2024).

The team

MARC is Harvey Mudd’s student rocketry club. Its competition team designs, builds and launches a high-power rocket each year at the Friends of Amateur Rocketry (FAR) Unlimited intercollegiate competition in the Mojave Desert. Most members join with no rocketry experience, and the club is built around giving them real engineering tasks across structures, recovery, propulsion and avionics.



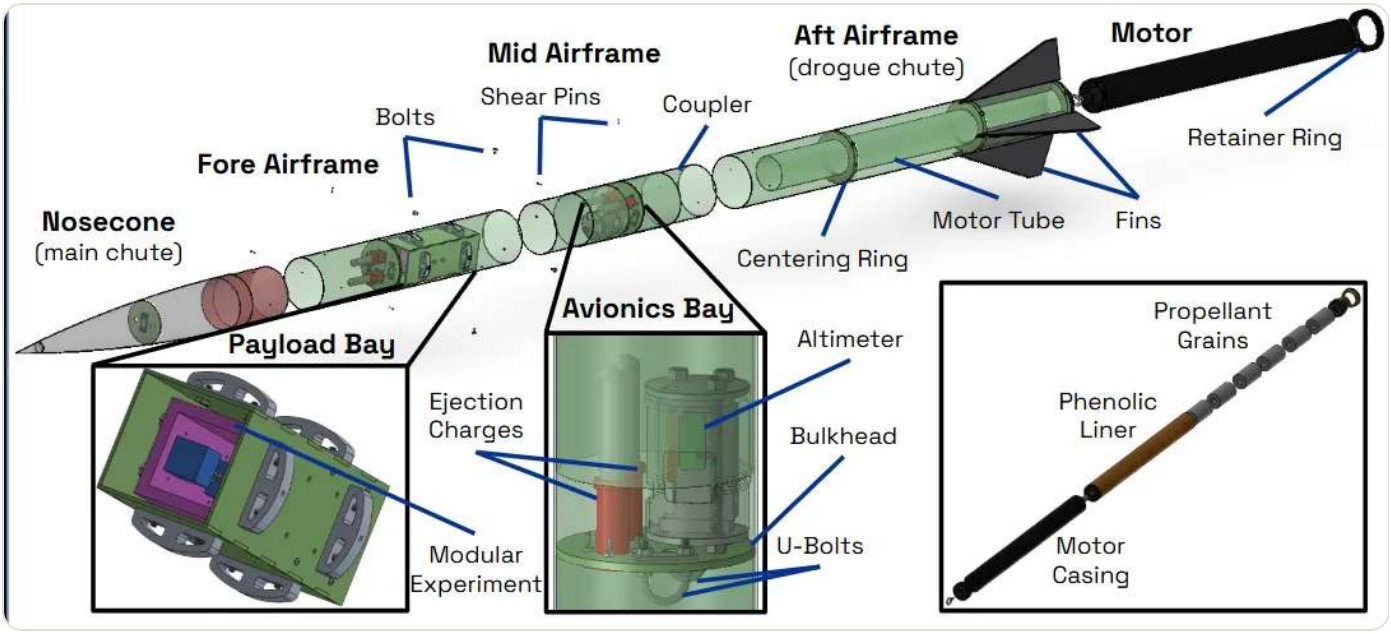
The launch team in front of the tower at FAR

Competition results

The team placed 5th in 2024, 3rd in 2025 with Gladius III, which flew within 0.05% of its simulated apogee, and 1st in 2026 with Apollyon I, the program's first championship.

How the rocket is put together

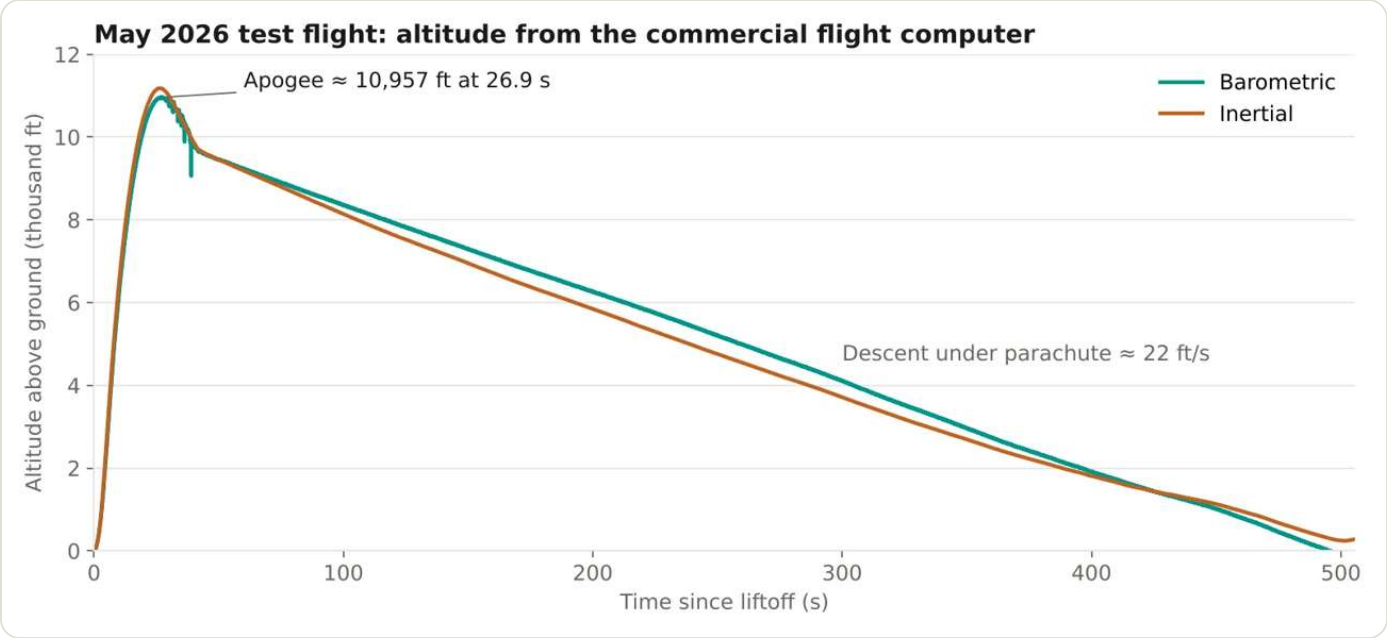
Our competition rockets are dual-deploy. A small drogue parachute comes out at apogee so the rocket falls fast without drifting miles away, and the large main parachute opens closer to the ground for a gentle landing. That drives the layout: the nosecone holds the main chute, the aft airframe holds the drogue, and the avionics bay sits between them so its ejection charges can push each section apart. Shear pins hold the sections together through boost and only break when a charge fires. The payload bay carries a swappable experiment module, and the motor slides into a motor tube held by centering rings and locked in with a retainer ring.



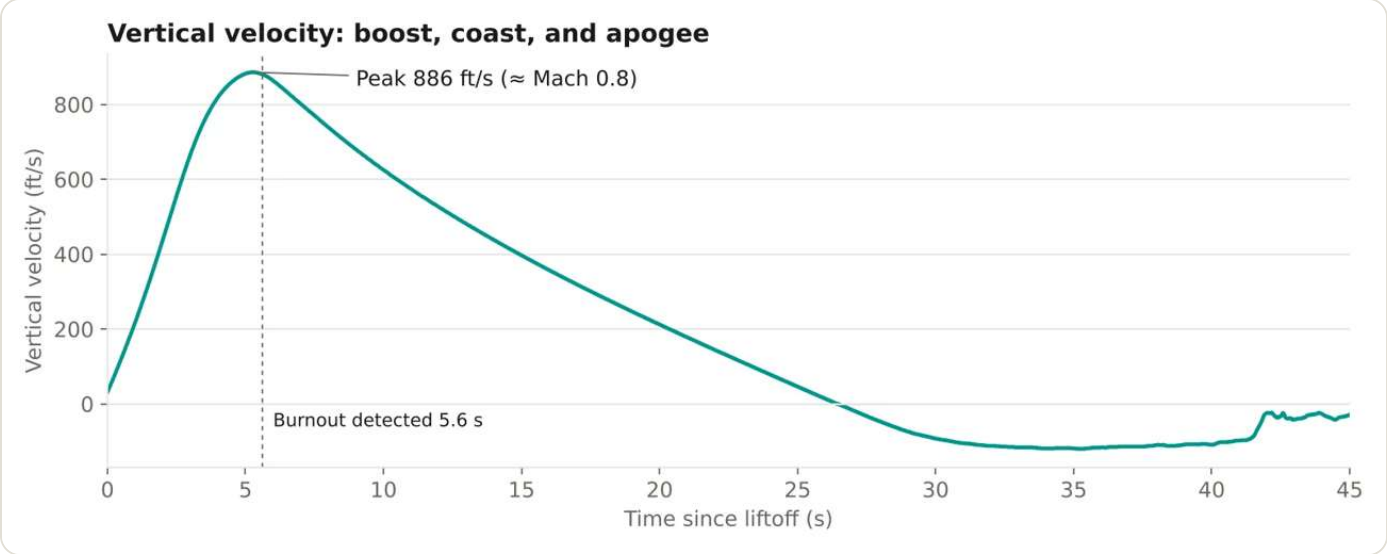
Exploded view of the competition rocket: nosecone with main chute, payload bay, avionics bay with altimeter and ejection charges, aft airframe with drogue chute, fins and motor

What a flight looks like in the data

This is the log from a May 2026 test flight, recorded by the commercial flight computer in the avionics bay. The motor burns for about five and a half seconds and takes the rocket to 886 ft/s, around Mach 0.8. It then coasts for another 21 seconds to an apogee of about 10,960 ft above the ground, and spends the next eight minutes coming down under parachute at roughly 22 ft/s.

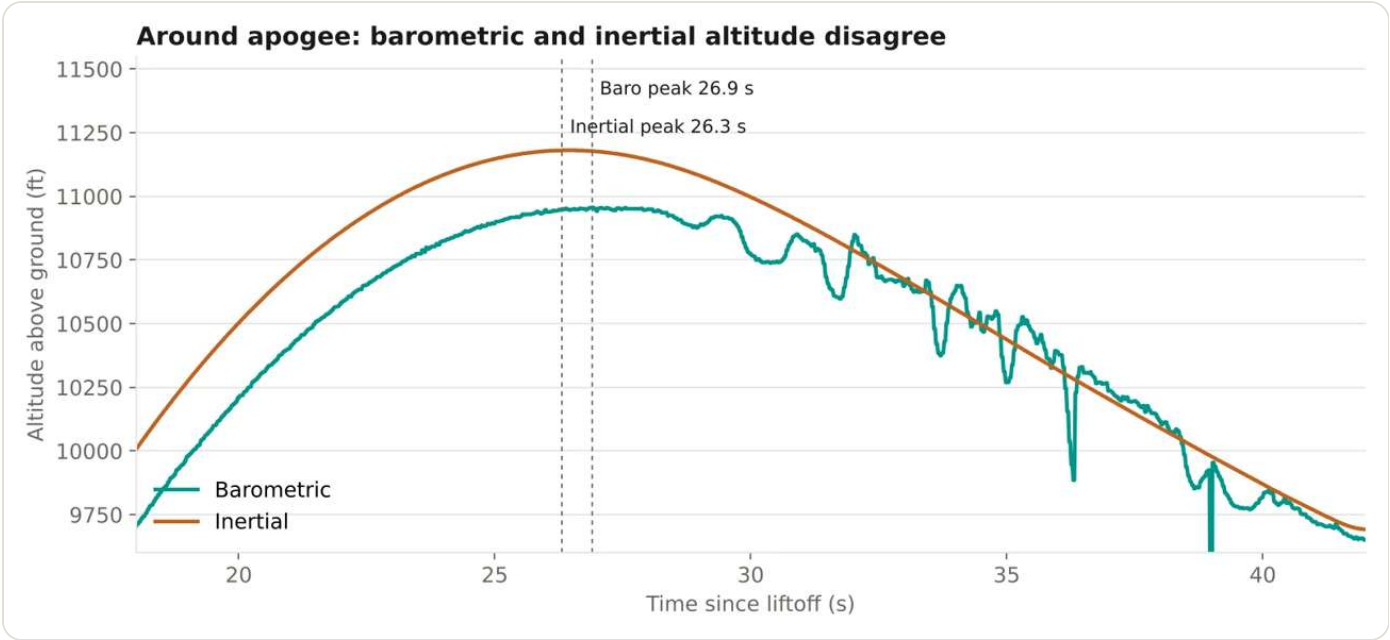


Barometric and inertial altitude over the whole flight, from liftoff to landing about 500 seconds later



Vertical velocity during boost and coast, peaking at 886 ft/s right around burnout

The zoom around apogee shows why avionics is mostly about trusting the right sensor at the right time. The inertial estimate, integrated from the accelerometer, is smooth but peaks about 220 ft higher than the barometer, because small errors add up over 26 seconds of integration. The barometer is closer to the truth on altitude but gets noisy right after apogee, when the rocket turns over and the pressure at its ports jumps around. Deciding that apogee has really happened from two signals that disagree is the problem behind my apogee-detection timing work.



Around apogee the inertial estimate peaks higher and smoother, while the barometric trace gets noisy once the rocket starts to fall

Onboard video



Onboard video: watch it on the project page.

Avionics

Avionics covers everything electronic on the rocket: the flight computers that detect apogee and fire the recovery charges, the sensors, and the arming and safety procedures on launch day. During ascent a rocket can pull enough g’s to saturate a typical

accelerometer, and that's exactly when you need a good estimate of where it is. I combined redundant sensors so the state estimate stays reliable through high-g saturation, which cut our apogee-detection timing error by about 30%, and ran a magnetometer interference study.



Arming the flight computers with the rocket on the launch rail

Now: Atlas, a custom flight computer for active guidance

Until now our rockets have relied on passive stability. This year the team is taking on active guidance, using sensor feedback to move control surfaces in flight. We're comparing three approaches, canards near the nose, air brakes that deploy from the body, and flaps on the rear fins, on smaller Level 2 test rockets before choosing one for the competition vehicle. The team has also written up a two-stage concept, weighing drag separation against charge-separated cold staging and an electronically lit sustainer motor. All of that needs a flight computer we design ourselves.

Atlas is that board, still very much a work in progress. It's built around an STM32H743 running FreeRTOS:

- **Sensing with overlap on purpose:** an ADXL375 high-g accelerometer for the boost phase, an LSM6DSV16B IMU, an MMC5983MA magnetometer, an MS5611 barometer, a

BNO085 orientation sensor, and a u-blox NEO-M9N GNSS receiver whose timing pulse lines up every log.

- **Links:** an RFD900x long-range telemetry radio, Bluetooth through a NINA-B112, USB to a desktop ground-station dashboard, and microSD logging.
- **Outputs:** eight PWM servo channels at 333 Hz for control surfaces and five pyro channels for recovery charges.

Safety is designed in from the start. Every output boots disabled, firing needs a software arm plus a measured armed feed, and each pyro channel fires a fixed 500 ms pulse with a limited number of attempts per power cycle. The first board revision is in bench bring-up now: drivers and host tests are in place, and a fixed bench demo already runs four-servo gravity stabilization. Every subsystem still has to pass physical qualification before it flies.

Teaching

I wrote a Level 1 rocket design guide for beginners and have led rocket-building sessions for more than 60 students. Level 1 is the first high-power certification level, and I built and launched my own L1 rocket before teaching others to.



My Level 1 rocket after the build



The same rocket on its way up

Follow the team on Instagram for launch-day photos and build updates.

RESEARCH

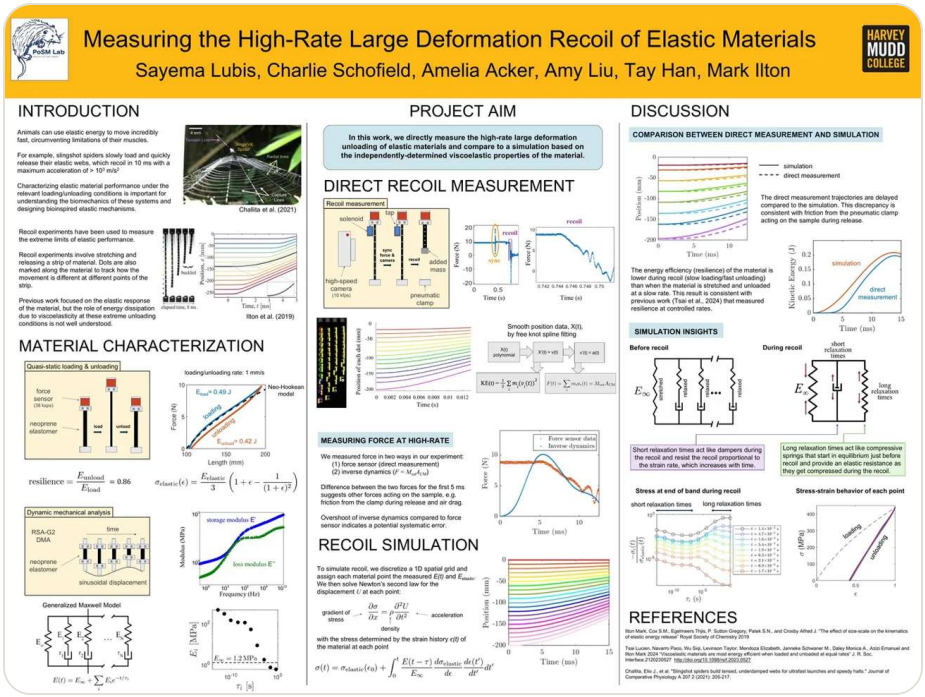
Physics of Soft Matter Lab

Research Assistant

September 2023 – January 2025 · Harvey Mudd College

Studied the physics of latch-mediated spring-actuated systems through experiments and simulation, and automated the lab's recoil-motion analysis.

MATLAB | Computer vision | High-speed video | Data analysis



HIGHLIGHTS

- Automated recoil-motion analysis in MATLAB, tracking elastic-material position with the Computer Vision Toolbox and smoothing the data to characterize energy efficiency.
- Co-authored the poster "Measuring the High-Rate Large Deformation Recoil of Elastic Materials," presented at the SoCal Soft Matter Symposium and the APS Physics Global Summit.

The research

Animals like slingshot spiders store elastic energy slowly and release it almost instantly: their webs recoil in about 10 ms with accelerations above 10^3 m/s². Our lab, led by Prof. Mark Ilton, studies the physics behind these latch-mediated spring-actuated systems. In this project we measured how an elastic strip recoils at high rates and large deformations, and compared it to a simulation built from the material's independently measured viscoelastic properties. You can read more about the lab at posmlab.org.

My contribution

In the recoil experiment, a neoprene strip marked with dots is stretched, held by a pneumatic clamp, and released while a high-speed camera films it at 10,000 frames per second. I automated the analysis in MATLAB. The Computer Vision Toolbox tracks every dot frame by frame, and I smooth the position data with free-knot spline fitting so it can be differentiated cleanly into velocity and acceleration. From those I compute kinetic energy and the force on the strip by inverse dynamics ($F = M \cdot a_{CM}$), which we checked against a force sensor sampling at 38 kHz.

What we found

- The material is less energy-efficient during a fast recoil than when it is loaded and unloaded slowly (0.86 resilience at 1 mm/s), which matches earlier controlled-rate measurements.
- The measured trajectories lag the simulation slightly, which is consistent with friction from the clamp during release.
- In the viscoelastic model, short relaxation times act like dampers during recoil, while long relaxation times act like compressed springs.

Poster

Measuring the High-Rate Large Deformation Recoil of Elastic Materials. S. Lubis, C. Schofield, A. Acker, **A. Liu**, T. Han, M. Ilton. Presented at the SoCal Soft Matter Symposium and the APS Physics Global Summit. Download the poster (PDF)

CLINIC

U.S. Department of Agriculture

Electrical Engineer (Clinic)

August – December 2024 · Harvey Mudd Engineering Clinic, with Auburn University

Member of a Harvey Mudd Clinic team working with the USDA and Auburn University to modernize electropenetograph technology, an instrument entomologists use to study how insects feed on plants.

Engineering Clinic | Electronics | Instrumentation

Project overview on HMC Clinic ↗



HIGHLIGHTS

- Worked on the electronics for the team's instrument as part of a sponsored clinic project.

This was a sponsored Engineering Clinic project, so the technical details stay with the sponsor. Harvey Mudd’s public overview of the project is linked above.

MICROPROCESSOR SYSTEMS · FINAL PROJECT

Interactive 2D Physics Engine

E155 final project · Fall 2024 · with Stephen Xu

A tilt-controlled sand simulator. An FPGA runs the particle physics and drives a 64×64 LED matrix in 12-bit color while an STM32 reads an accelerometer and streams the gravity direction over SPI.

FPGA SystemVerilog STM32 I2C SPI LED matrix

Project site ↗ Source code ↗

What it does

Each lit pixel on a 64×64 LED matrix is a grain of sand. Tilt the box and gravity follows: the grains fall, pile up, collide with each other and with the walls, and settle in whatever direction is now "down."

Demo video: watch it on the project page.

System design

We split the work so each chip does what it is good at.

- **STM32L432KC microcontroller:** reads an MPU6050 accelerometer over 400 kHz I2C (±2 g range, 1 kHz sampling), turns the X and Y acceleration into a tilt angle with `atan2`, maps it to 0–360°, and sends it to the FPGA over a one-way SPI link.
- **FPGA:** acts as a physics accelerator. It treats each pixel as a cellular automaton, updates every particle based on the current gravity direction, and drives the LED matrix.

Display driver

The matrix natively shows only 3-bit color (8 colors). We built a display driver that uses binary-coded modulation (BCM) to reach 12-bit color, which is 4,096 colors, while keeping the refresh rate above 300 Hz with no ghosting. BCM also gives a logarithmic, eye-friendly dimming curve. The driver is a three-state machine: shift a row of pixel data in, latch it, then enable the output.

The hard part: memory

We originally planned to store the simulation in block RAM with double buffering. Timing problems with block RAM forced us to keep the particle state in LUTs instead, which limited how many particles we could simulate. We reworked the design around rotating frame buffers that hold the current and previous frame. It’s a real lesson in checking a resource budget early.

Testing

We checked both serial links with a logic analyzer and verified the physics engine in simulation before putting it on hardware.

Results

We met every spec we set: sub-second response to tilt, 12-bit color, a refresh rate above 300 Hz, movement in eight directions, at least ten particles with particle and wall collisions, and a stable simulation with no drift over time. We hand-soldered the electronics on perfboard and designed a 3D-printed housing.

What I’d do next

Get block RAM working so the engine can handle far more particles, and use more of the IMU’s data so the sand reacts to shaking as well as tilt.

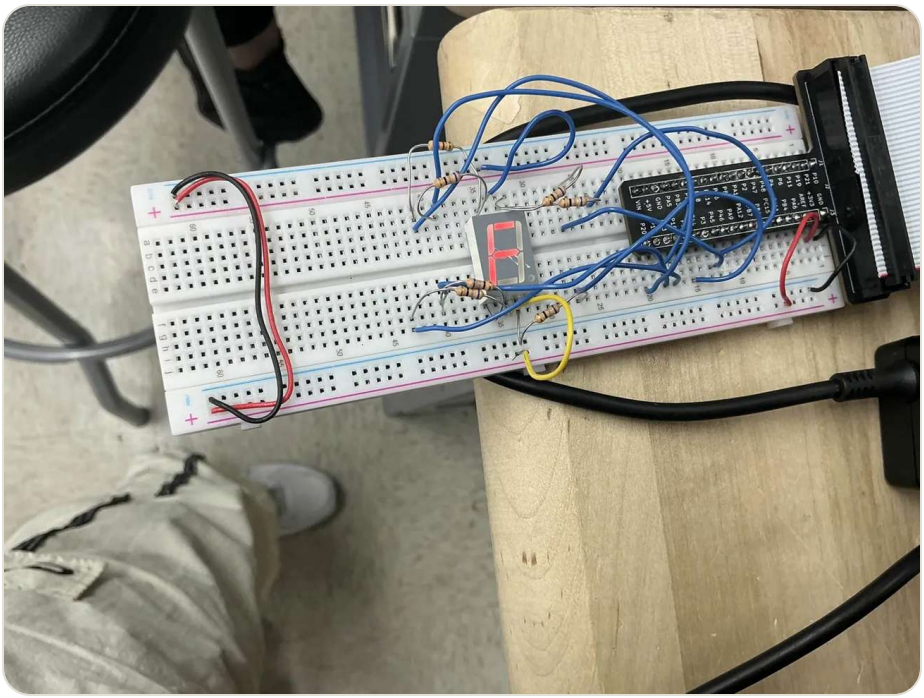
MICROPROCESSOR SYSTEMS · LAB 1

FPGA Bring-up & Seven-Segment Decoder

E155 Lab 1 · Fall 2024

Assembling and testing the FPGA/MCU board, then a combinational SystemVerilog decoder that drives a seven-segment display from four switches, verified on every input.

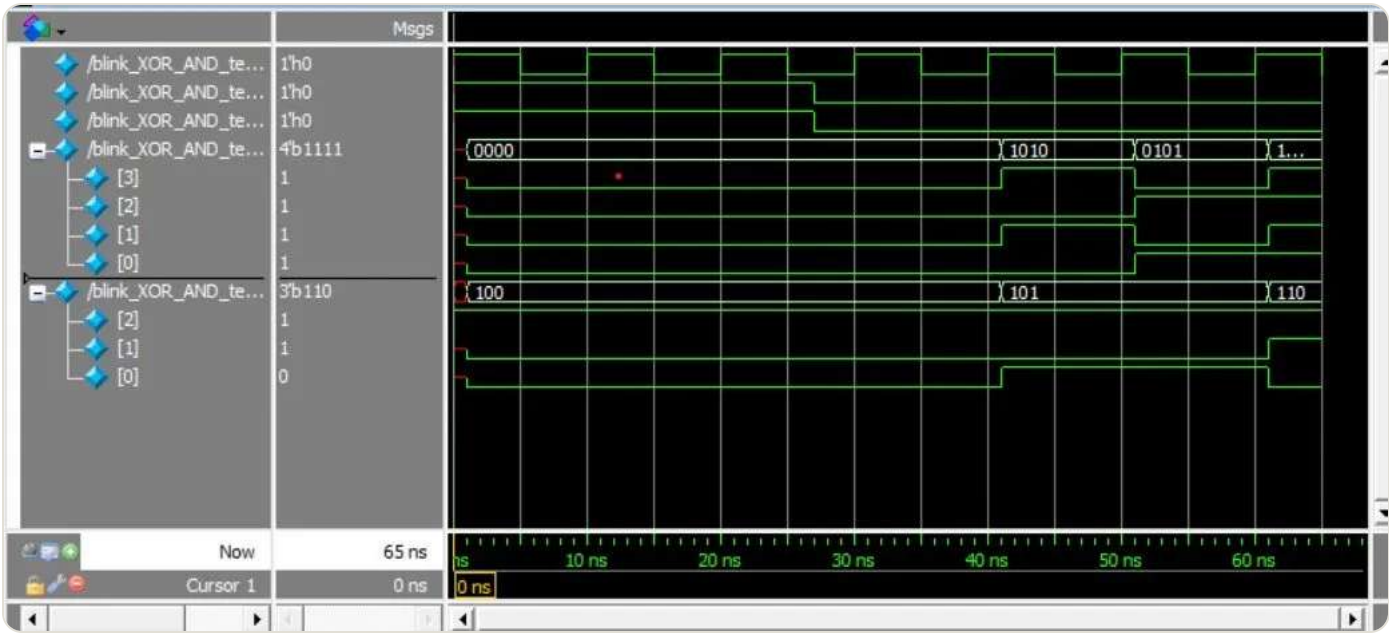
FPGA SystemVerilog SMD soldering QuestaSim



The first lab was about getting comfortable with the hardware: soldering surface-mount parts onto the development board, writing basic combinational SystemVerilog in Lattice Radiant, and simulating it before trusting it on hardware.

Board test

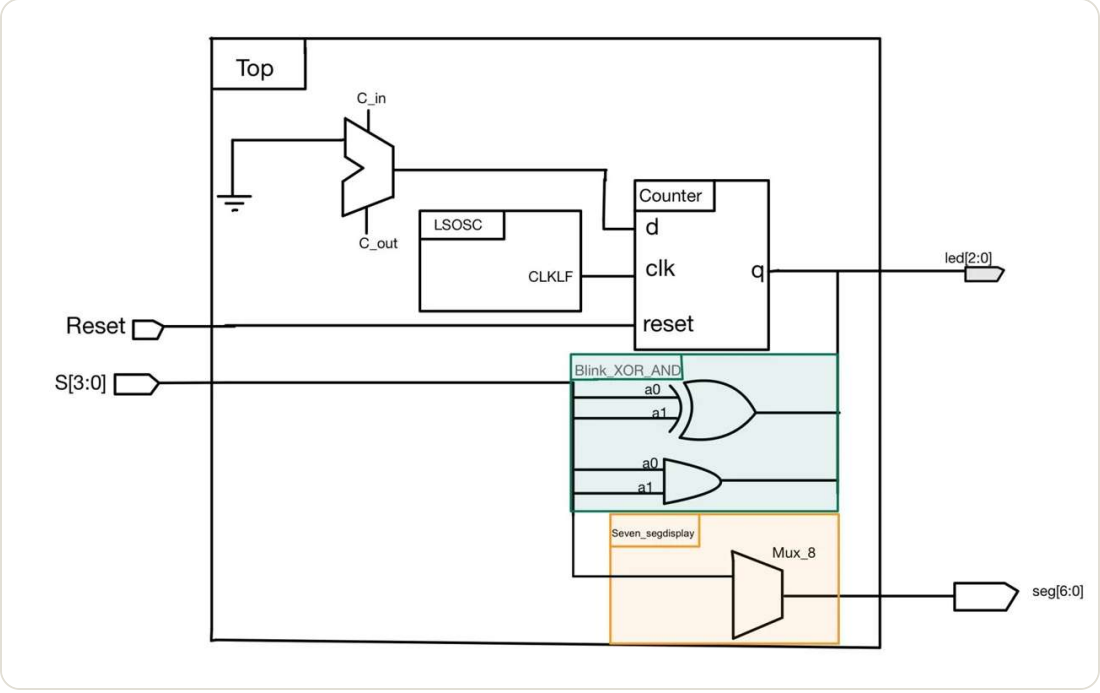
To check the board, I lit two LEDs with the XOR and AND of two switches and blinked a third at 2.4 Hz. The blink comes from the 10 kHz low-speed oscillator and a 12-bit counter: 10 kHz / 2^{12} ≈ 2.44 Hz. A reset makes the simulation start from a known value instead of a random one.



Testbench for the XOR, AND and blink logic

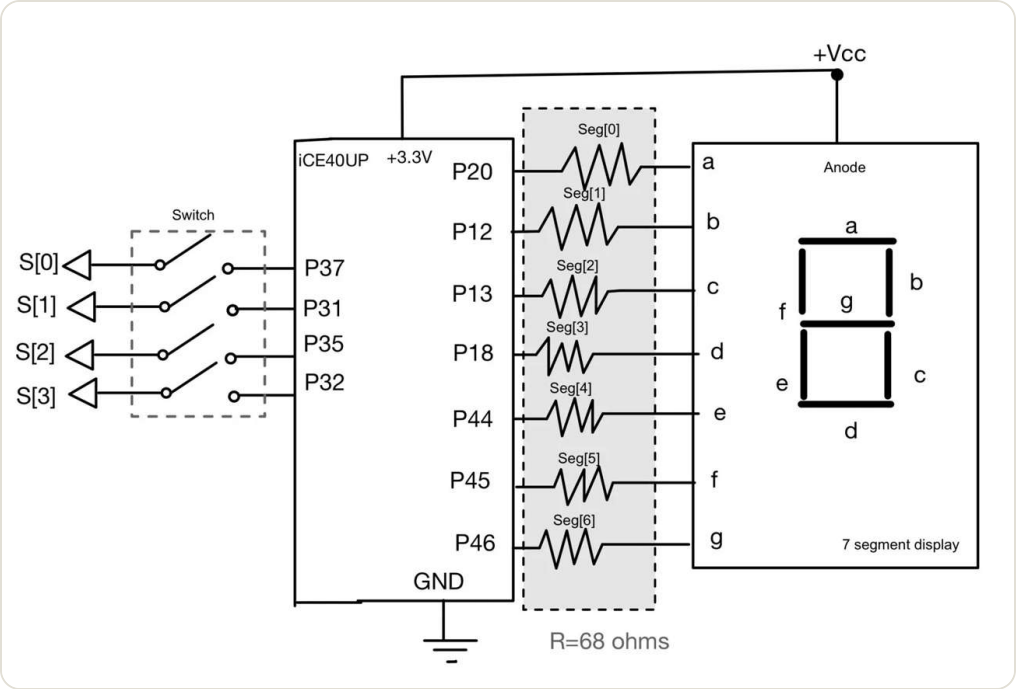
Seven-segment decoder

A combinational case statement maps each of the 16 switch combinations to the segments for hex digits 0–F, with `seg[0]` as segment a, `seg[1]` as segment b, and so on.



Block diagram of the top module

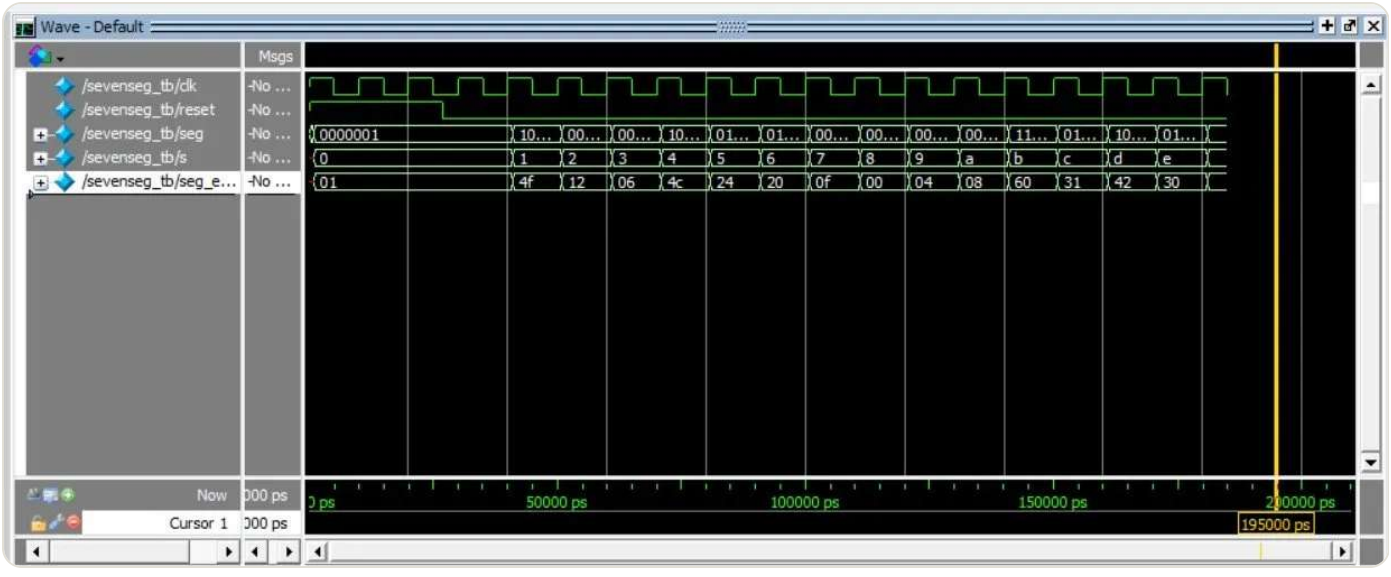
Each LED segment needs 5–20 mA. With a 3.3 V supply and a 2.1 V forward drop, a 68 Ω resistor gives (3.3 − 2.1) V / 68 Ω ≈ 17 mA.



Seven-segment display schematic

Testing

I ran all 16 switch combinations in QuestaSim and then on the breadboard, so no edge case was left untested.



Seven-segment testbench covering all 16 inputs

MICROPROCESSOR SYSTEMS · LAB 2

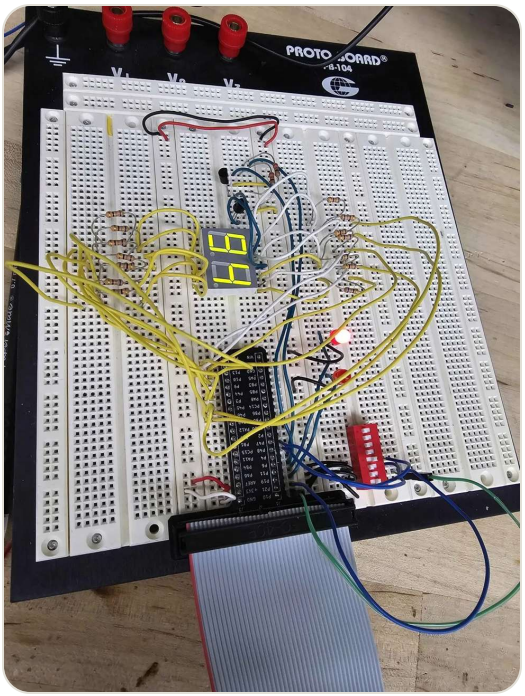
Multiplexed Seven-Segment Display

E155 Lab 2 · Fall 2024

Two seven-segment digits sharing one set of FPGA pins through time multiplexing, with PNP transistor drivers sized from the iCE40 datasheet.

FPGA SystemVerilog Time multiplexing Transistor drivers

Source code ↗

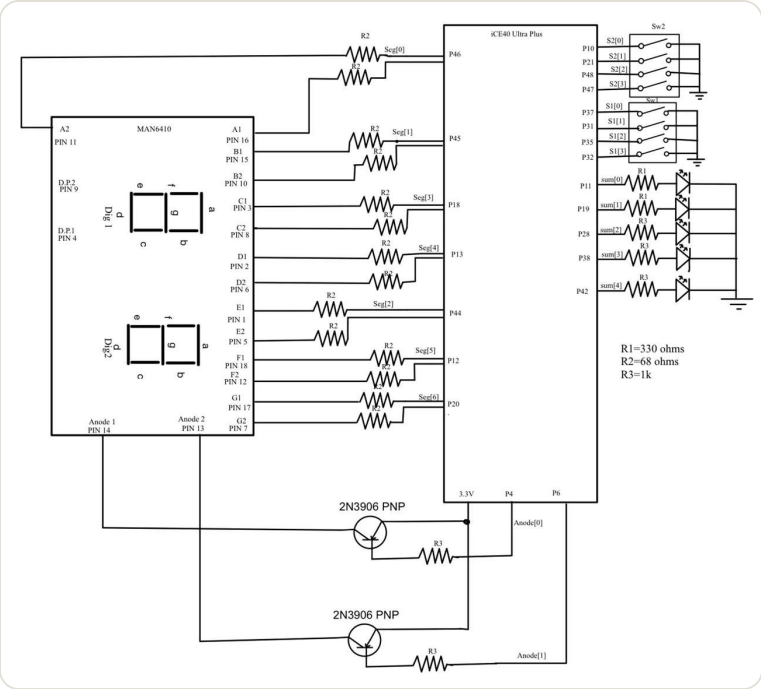


This lab extends Lab 1 to two digits that show two hex inputs and their sum, without using any more I/O pins. The FPGA powers one digit at a time and switches between them faster than the eye can see.

Driving the displays

The iCE40UP datasheet limits each I/O pin to 8 mA, which isn't enough to light a whole digit. I used 2N3906 PNP transistors to switch each display's common anode, with the

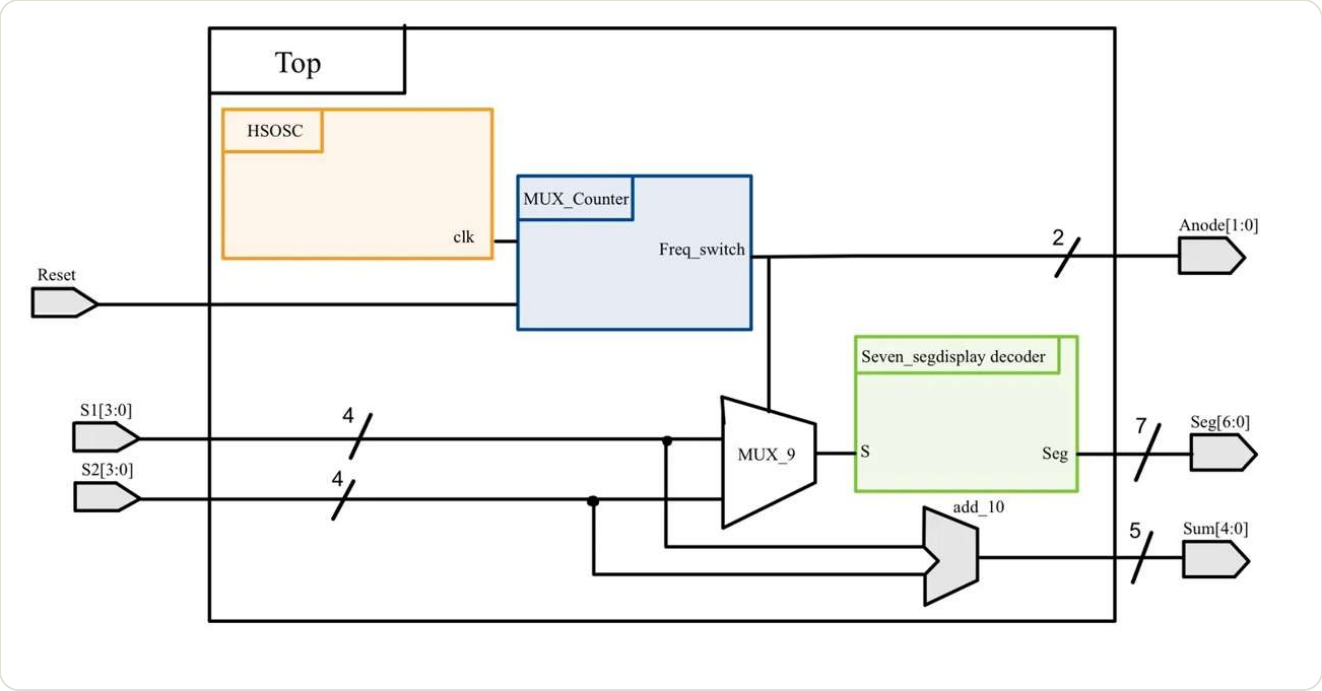
emitter on 3.3 V and the base on a GPIO pin. The minimum base resistor is (3.3 − 0.7) V / 8 mA = 325 Ω, and I used 1 kΩ for margin. The segments keep the 68 Ω resistors from Lab 1.



Schematic with transistor drivers

Logic

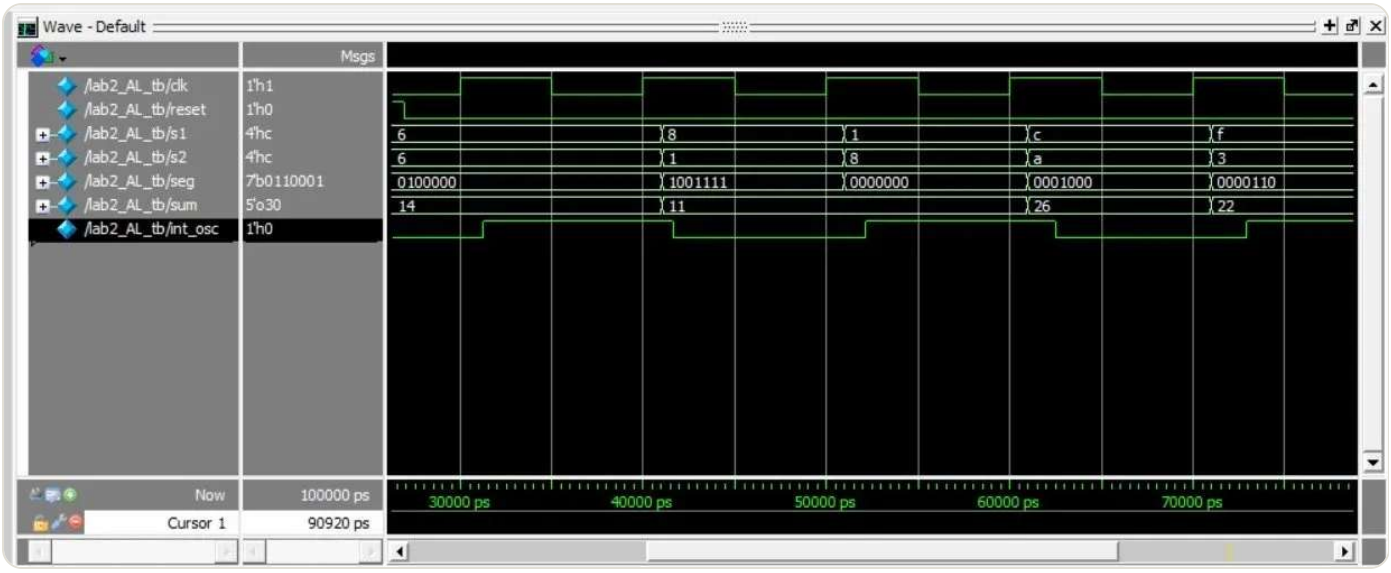
I reused the Lab 1 decoder as its own module. A 12-bit counter on the 24 MHz high-speed oscillator toggles which digit is on at 24 MHz / 2^{12} ≈ 5.9 kHz, and a mux picks which input goes to the decoder.



Module block diagram

Testing

The testbench covers representative cases, including two equal inputs and two different ones. Each vector holds input 1, input 2, the anode select, the segments and the sum.



Testbench results

Looking back

It worked, but the code was messier than it needed to be: a few unused signals and names that didn’t say what they did. Since then I split muxes into their own modules, keep the clock at the top level only, and name signals for their job.

MICROPROCESSOR SYSTEMS · LAB 3

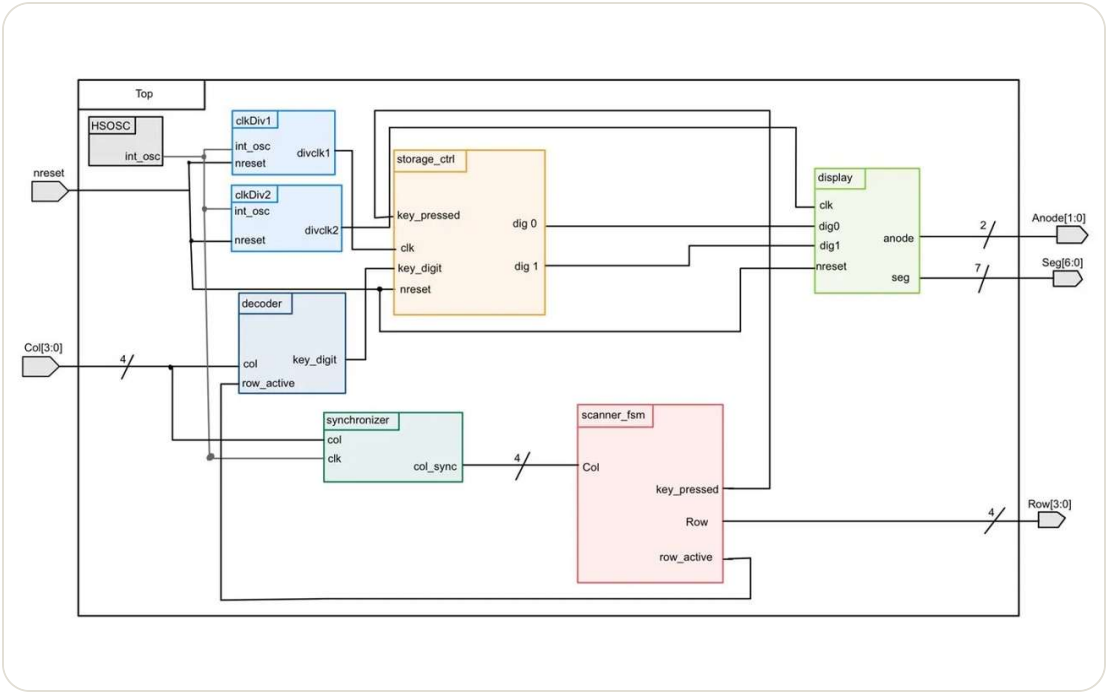
Keypad Scanner

E155 Lab 3 · Fall 2024

FPGA logic that scans a 4×4 matrix keypad, debounces it with a state machine, and shows the last two keys pressed on the multiplexed display from Lab 2.

FPGA SystemVerilog FSM Debouncing Testbenches

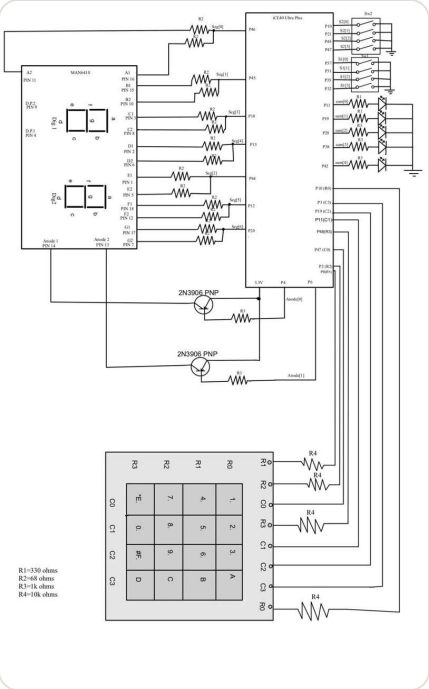
Source code ↗



The keypad had to register each press exactly once: no repeats while a key is held, and no false presses from switch bounce. The last two keys appear on the dual display, newest on the right.

Hardware

The columns sit high on pull-up resistors. The scanner drives one row low at a time (`0111` , `1011` , …), so a pressed key pulls its column low.



Keypad wiring schematic

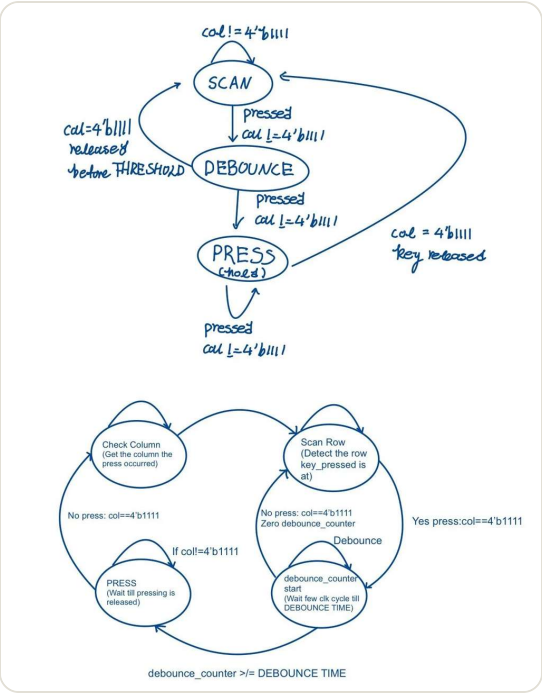
Modules

- **Scanner FSM** watches the columns, decides whether a press happened, and drives the rows.
- **Decoder** turns the active row and column into a hex digit.
- **Storage** keeps the two most recent digits in a pair of registers. Keeping it out of the scanner made both modules simpler to debug.
- **Display** is the multiplexed driver from Lab 2.

Column inputs pass through a two-flop synchronizer before reaching the FSM.

Debouncing

A counter has to reach `DEBOUNCE_TIME` before the FSM moves to the pressed state; if the key is released first, it goes back to scanning. I chose this over an RC filter because the timing can be retuned for a different switch in code, at the cost of a small delay before the digit appears.



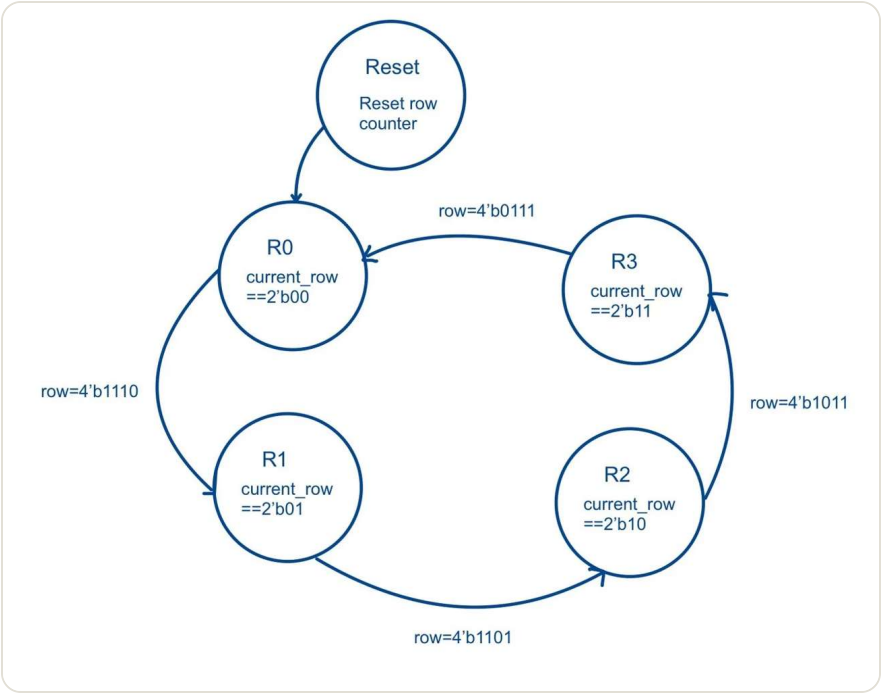
Scanner state machine

Current State	Inputs	Next State	Output
SCAN	col==4'b1111	SCAN	key_pressed = 0
SCAN	col!=4'b1111	DEBOUNCE	key_pressed = 0
DEBOUNCE	debounce_counter<DEBOUNCE TIME, col !=4'b1111	DEBOUNCE	key_pressed = 0, debounce_counter += 1
DEBOUNCE	debounce_counter>/=DEBOUNCE TIME, col !=4'b1111	PRESS	key_pressed = 1, key_stable = 1
DEBOUNCE	debounce_counter<DEBOUNCE TIME, col ==4'b1111	SCAN	key_pressed = 0, key_stable = 0, debounce_counter=0
PRESS	col == 4'b1111	SCAN	key_press = 0, all_released = 1
PRESS	col!=4'b1111	PRESS	key_press = 1, all_released = 0

Scanner state transition table

Registering one press per key

The display updates on the rising edge of `key_pressed`, so holding a key down never adds another digit.



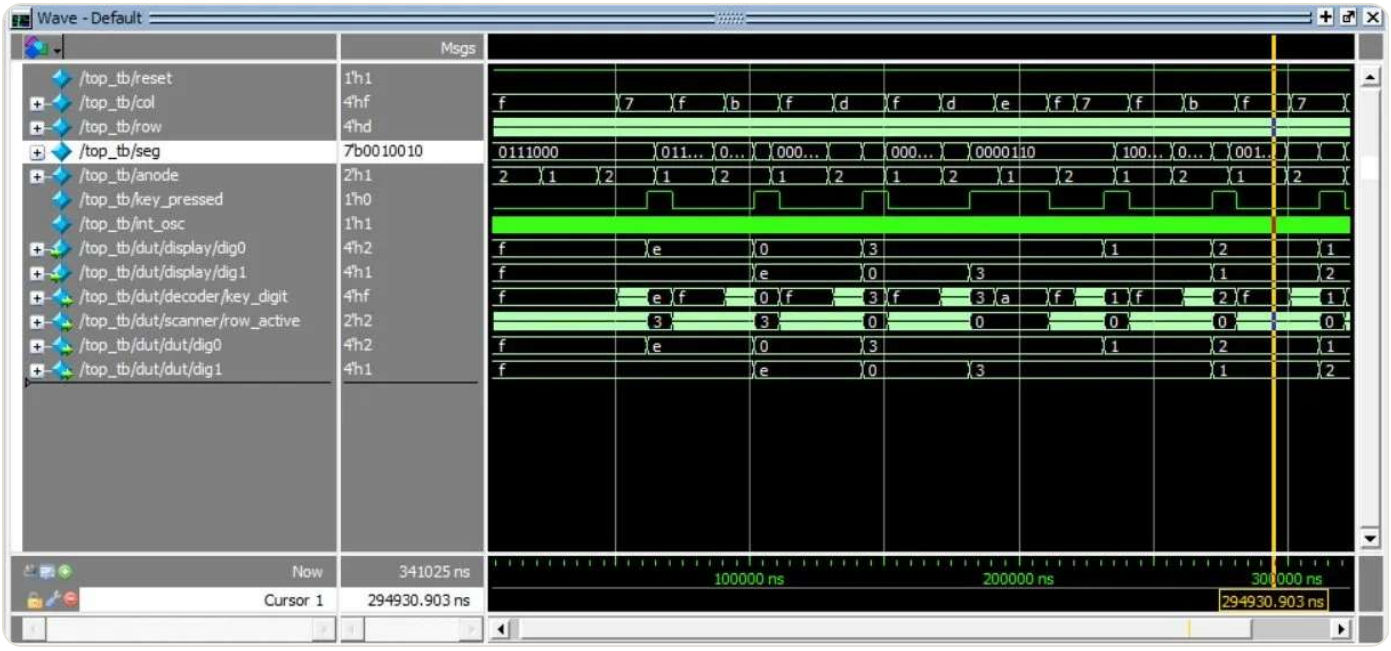
Row scanning states

when...	action	code
key_pressed = 0	wait for input	dig 1 = dig1, dig0 = dig0
key_pressed = 1	store new digits	N/A
x (key_pressed =1 previously)	replace old digit	dig1<=dig0, dig0<=key_digit

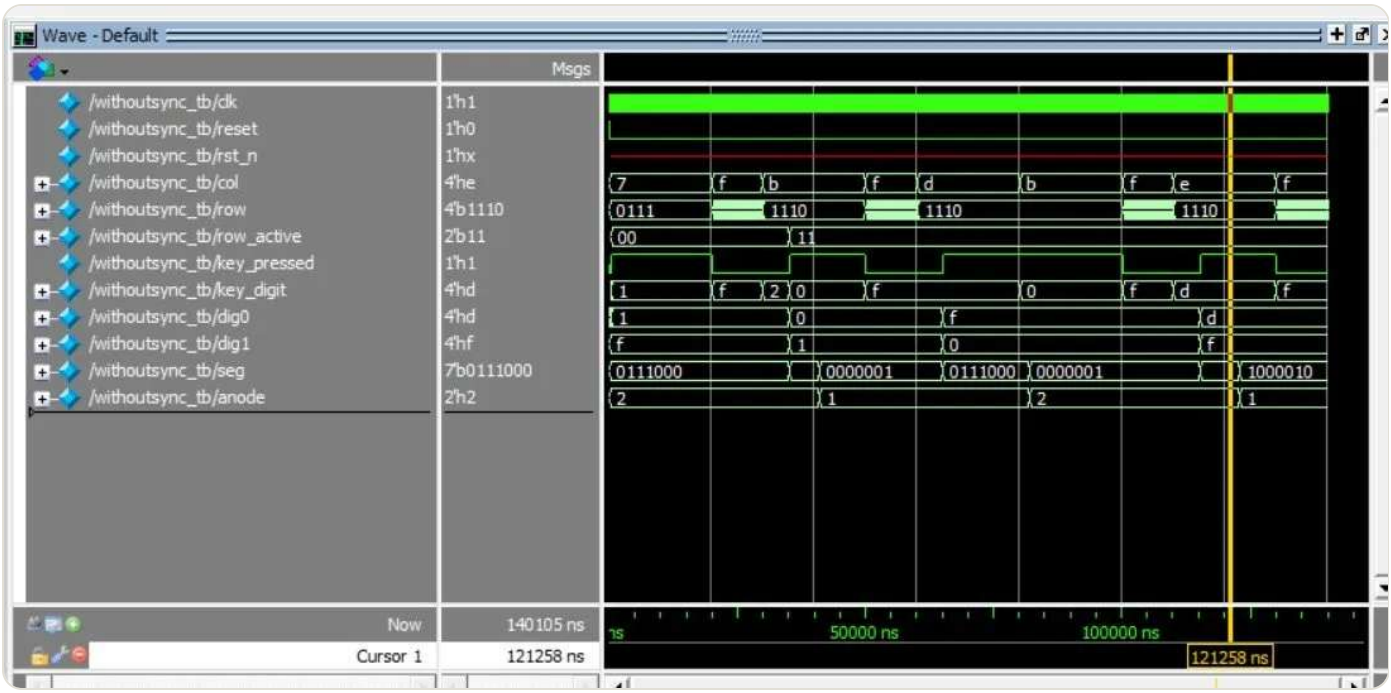
Two-digit storage module

Verification

The top-level testbench checks that a press that is too short is ignored, and that when two keys are pressed close together only the first one registers.



Top-level testbench waveforms



Module tests without the synchronizer, showing the same behavior two cycles earlier

What I'd do differently

Bring an oscilloscope in earlier to confirm each state fires at the right time on real hardware, instead of trusting the testbench alone.

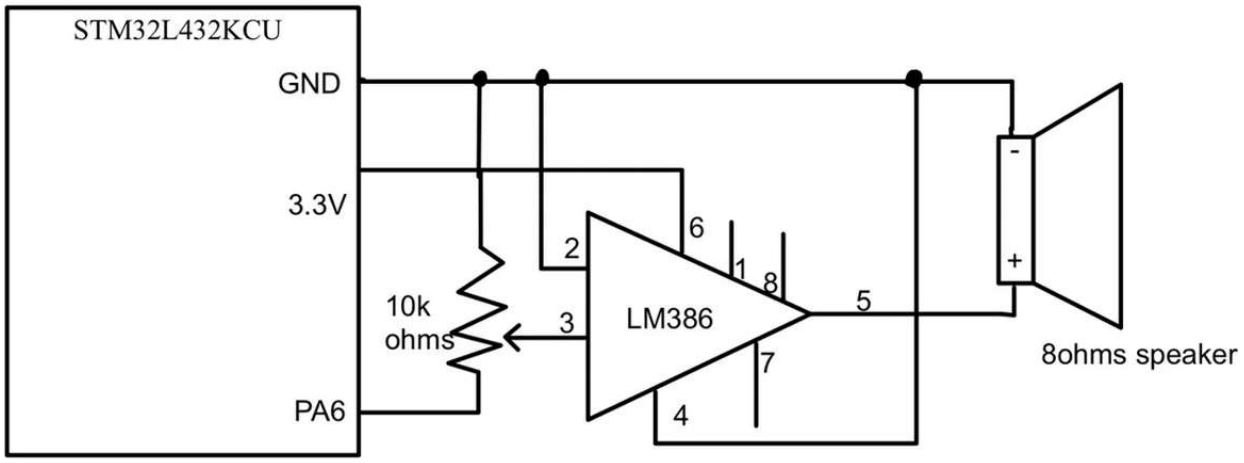
MICROPROCESSOR SYSTEMS · LAB 4

Digital Audio Player

E155 Lab 4 · Fall 2024

Bare-metal STM32 firmware that plays music through a speaker, using two hardware timers for pitch and note length, with frequency error checked across three octaves.

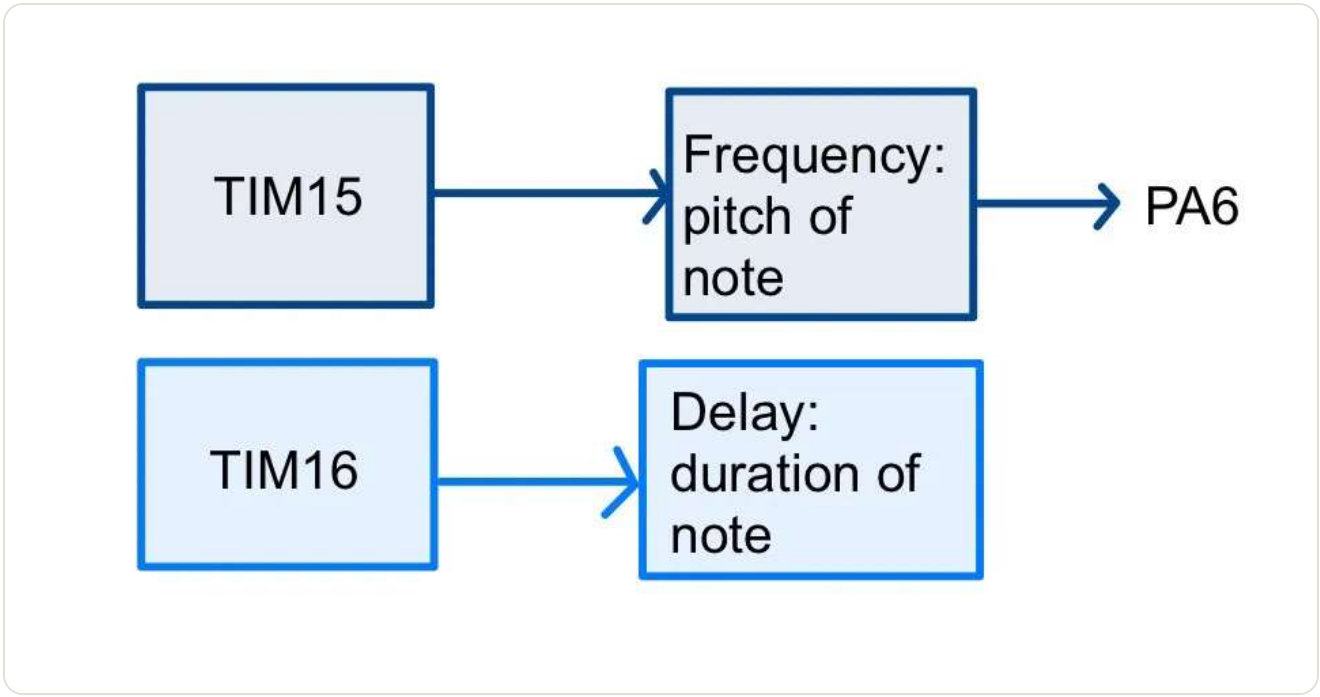
STM32 Timers Embedded C LM386 MATLAB



The STM32L432KC plays a song from a table of note frequencies and durations. One timer generates the tone on a GPIO pin, and a second one times how long each note lasts. An LM386 amplifier (gain of 50) drives an 8 Ω speaker, with a 10 kΩ potentiometer for volume.

Demo video: watch it on the project page.

Timer design



Timer roles and program flow

With an 80 MHz system clock:

	Tone timer (TIM16)	Delay timer (TIM1)
Prescaler	199	7999
Base frequency	400 kHz	10 kHz
Range	6.1 Hz – 200 kHz	0.1 ms – 6.55 s

The auto-reload value for each note is ARR = 80 MHz / ((199 + 1) · f). Rather than checking every note by hand, I wrote a MATLAB script that computes the actual frequency for every pitch from A2 to A5 and every note length in the song.

```
durations = [125, 250, 500];  % in ms

for i = 1:length(durations)
    duration_ms = durations(i);
    ARR_delay = (clock_speed / (prescaler_delay + 1)) * (duration_ms / 1000);
    actual_duration = ARR_delay / (clock_speed / (prescaler_delay + 1)) * 1000;  % Convert back to
    error_percentage = abs((actual_duration - duration_ms) / duration_ms) * 100;

    % Check if error is within 1%
    if error_percentage <= 1
        status = 'Good';
    else
        status = 'Bad';
    end

    fprintf('%-10d %-15d %-15.1f %-15.3f %-10s\n', duration_ms, round(ARR_delay), actual_duration,
end

% Additional validation for note types with different durations
fprintf('\nNote Type Duration Accuracy (in seconds):\n');
note_types = {'Whole', 'Half', 'Quarter', 'Eighth', 'Sixteenth'};
note_durations = [0.5, 0.25, 0.125, 0.0625, 0.03125];  % durations in seconds

for i = 1:length(note_durations)
    duration_s = note_durations(i);
    duration_ms = duration_s * 1000;
    ARR_delay = (clock_speed / (prescaler_delay + 1)) * (duration_ms / 1000);
    actual_duration = ARR_delay / (clock_speed / (prescaler_delay + 1)) * 1000;  % Convert back to
    error_percentage = abs((actual_duration - duration_ms) / duration_ms) * 100;

    % Check if error is within 1%
    if error_percentage <= 1
        status = 'Good';
    else
        status = 'Bad';
    end

    fprintf('%-10s %-15.4f %-15.4f %-15.4f %-10s\n', note_types{i}, duration_s, actual_duration /
end
```

```
Frequency Accuracy Analysis for Notes A3 to A5:
Note       Target Freq (Hz) ARR Value  Actual Freq (Hz) Error (%)       Status
A3         220.00           1818       220.02           0.010           Good
A#3/Bb3    233.10           1716       233.10           0.000           Good
B3         246.90           1620       246.91           0.006           Good
C4         261.60           1529       261.61           0.003           Good
C#4/Db4    277.20           1443       277.20           0.000           Good
D4         293.70           1362       293.69           0.005           Good
D#4/Eb4    311.10           1286       311.04           0.019           Good
E4         329.60           1214       329.49           0.034           Good
F4         349.20           1145       349.34           0.042           Good
F#4/Gb4    370.00           1081       370.03           0.008           Good
G4         392.00           1020       392.16           0.040           Good
G#4/Ab4    415.30           963        415.37           0.017           Good
A4         440.00           909        440.04           0.010           Good
A#4/Bb4    466.20           858        466.20           0.000           Good
B4         493.90           810        493.83           0.015           Good
C5         523.30           764        523.56           0.050           Good
C#5/Db5    554.40           722        554.02           0.069           Good
D5         587.30           681        587.37           0.012           Good
D#5/Eb5    622.20           643        622.08           0.019           Good
E5         659.20           607        658.98           0.034           Good
F5         698.40           573        698.08           0.046           Good
F#5/Gb5    740.90           540        740.74           0.021           Good
G5         784.00           510        784.31           0.040           Good
G#5/Ab5    830.60           482        829.88           0.087           Good
A5         880.00           455        879.12           0.100           Good
Duration Accuracy Analysis:
Duration (ms) ARR Value       Actual Duration (ms) Error (%)       Status
125           1250            125.0                0.000           Good
250           2500            250.0                0.000           Good
500           5000            500.0                0.000           Good
Note Type Duration Accuracy (in seconds):
Whole      0.5000          0.5000          0.0000          Good
Half       0.2500          0.2500          0.0000          Good
Quarter    0.1250          0.1250          0.0000          Good
Eighth     0.0625          0.0625          0.0000          Good
Sixteenth  0.0312          0.0312          0.0000          Good
```

Frequency and duration error for every note used

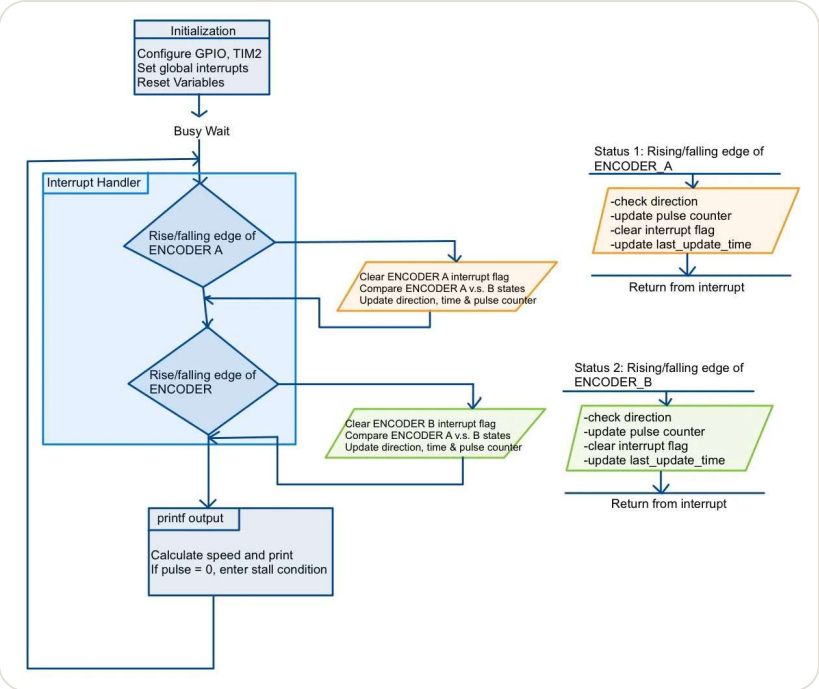
MICROPROCESSOR SYSTEMS · LAB 5

Motor Speed with Interrupts

E155 Lab 5 · Fall 2024

Interrupt-driven quadrature decoding on an STM32 that measures motor speed and direction to within 0.05%, compared directly against a polling implementation.

STM32 Interrupts Quadrature encoder Embedded C

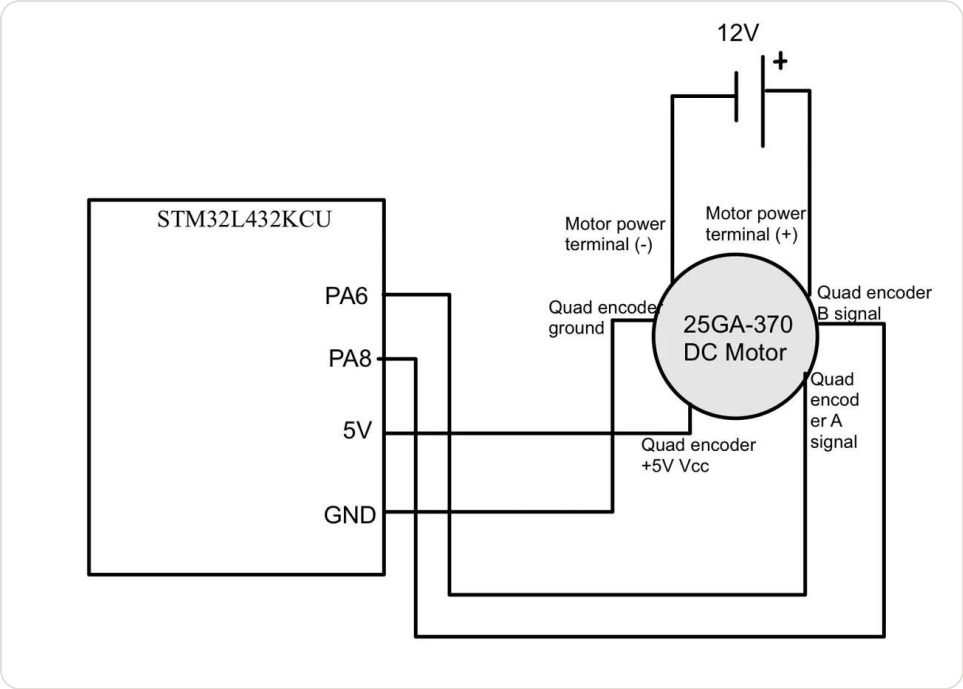


A DC motor's quadrature encoder outputs two square waves, A and B, that are 90° out of phase. The firmware counts edges to find speed and compares A and B to find direction, then prints revolutions per second at least once a second.

Design

- The encoder runs at 5 V, so I put its outputs on the STM32's 5 V-tolerant pins.
- Both channels fire external interrupts on rising and falling edges, which gives 4× resolution: 360 pulses per revolution × 4 = 1,440 counts per revolution. Encoder interrupts have priority over other tasks.

- TIM2 timestamps the measurement interval, and the main loop converts counts to rev/s. Positive values mean clockwise rotation.



Encoder wiring schematic

Interrupts vs. polling

At 12 V the motor should spin at about 10 rev/s, and I checked readings against that reference in both directions. I also built a polling version to compare:

	Interrupts	Polling
Measured speed at 12 V	9.918 rev/s	15.9 rev/s
Error	< 0.05%	~50%
Response time	< 1 µs	~1 ms
CPU load	Low	High

Polling misses edges at speed, which is why its reading is so far off. Interrupts take more care to get right, but they’re the only approach that holds up here.

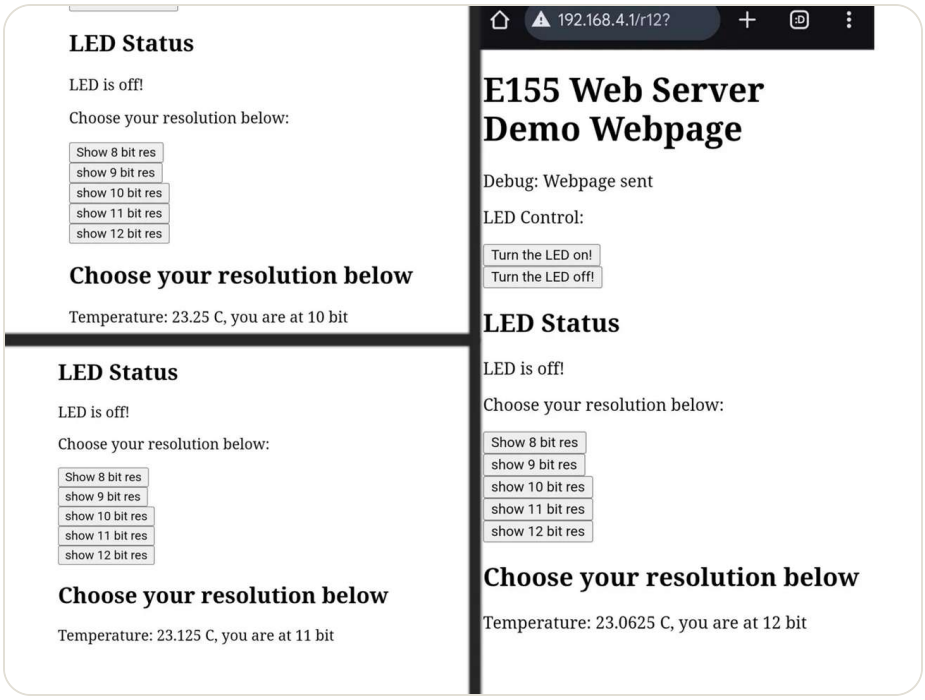
MICROPROCESSOR SYSTEMS · LAB 6

IoT Temperature Sensor & Web Server

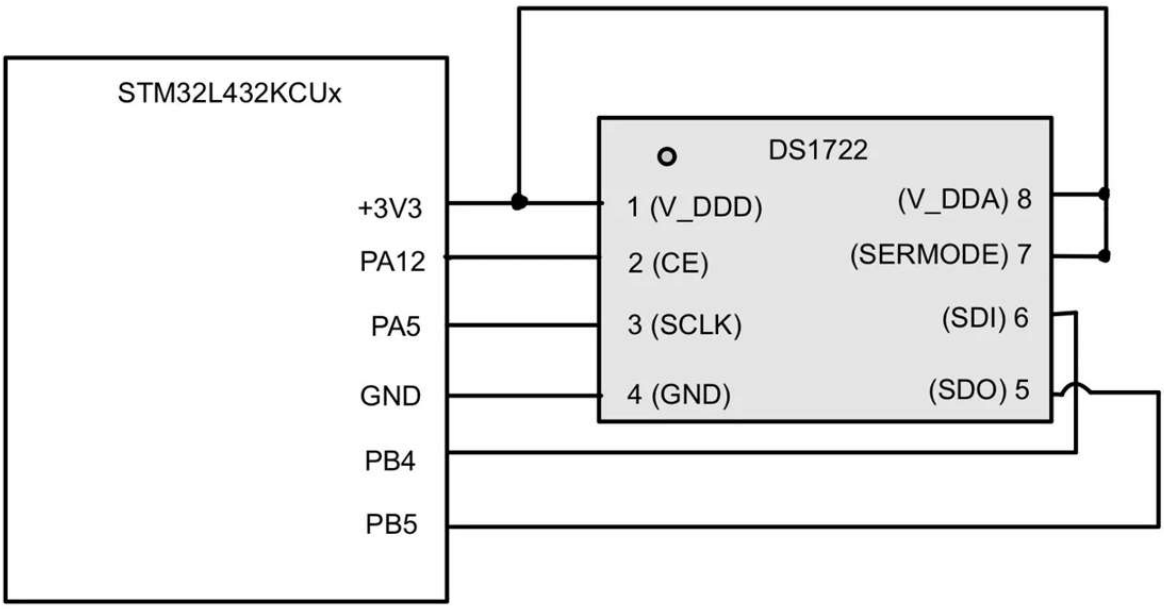
E155 Lab 6 · Fall 2024

An STM32 reads a DS1722 temperature sensor through a register-level SPI driver I wrote, and serves live readings, adjustable resolution and LED control on a web page through an ESP8266.

STM32 SPI UART ESP8266 Embedded C Logic analyzer



The goal was an internet-connected temperature sensor. An STM32L432KC microcontroller talks to a DS1722 digital temperature sensor over SPI and to an ESP8266 Wi-Fi module over UART. Anyone on the network can open a web page to see the current temperature, change the sensor's resolution from 8 to 12 bits, and switch an LED on the board on or off.



Wiring between the STM32 and the DS1722 sensor

Firmware design

I used CMSIS for hardware abstraction and wrote the SPI driver myself at the register level. The code is split into three layers:

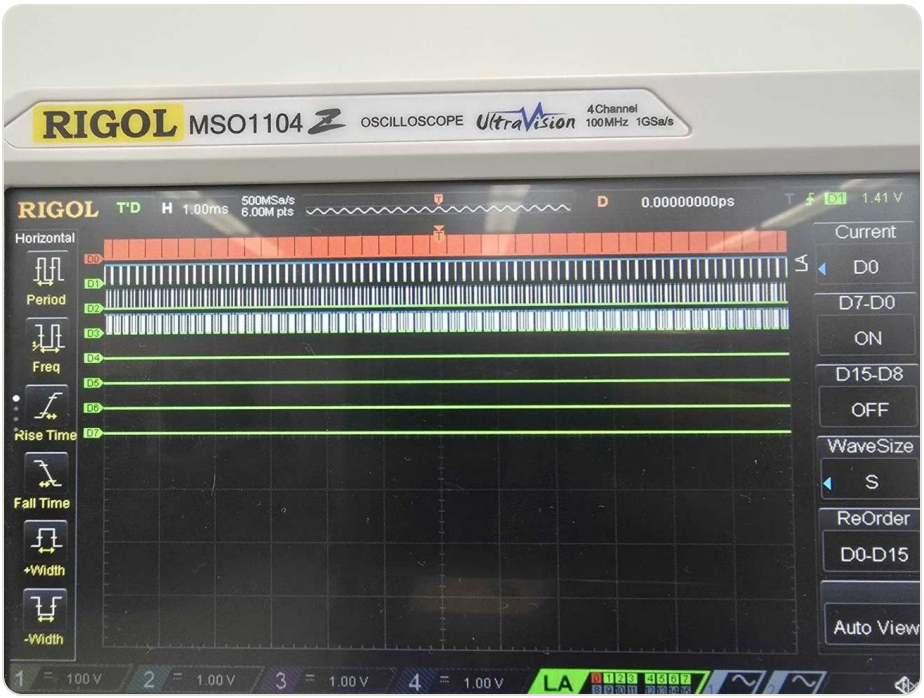
- **SPI and USART drivers.** SPI is configurable for baud rate, clock polarity and clock phase so it can work with other peripherals, and runs full-duplex with 8-bit frames. The transfer routine waits on the TXE flag before sending and the RXNE flag before reading, and accesses `SPI1→DR` through a `volatile` pointer so the compiler can't reorder or drop register accesses.
- **DS1722 driver.** It reads the two temperature bytes, combines them into a 16-bit signed fixed-point value (`hi << 8 | lo`), and divides by 256 to get degrees Celsius.
- **Web server.** It parses HTTP GET requests from the ESP8266 to set the LED and resolution, and builds the response page.

Testing

I checked the sensor's range by warming it with a finger and cooling it with canned air to get readings below zero. Then I confirmed the SPI transactions on a logic analyzer: clock, chip select, CIPO and COPI from top to bottom.



A single temperature read on the logic analyzer



SPI traffic during a page refresh

Lessons

Most of my debugging time went into SPI configuration, and into a broken breadboard wire that caused intermittent connections. That second problem is why I now check physical

connections with a meter before assuming a bug is in the code.

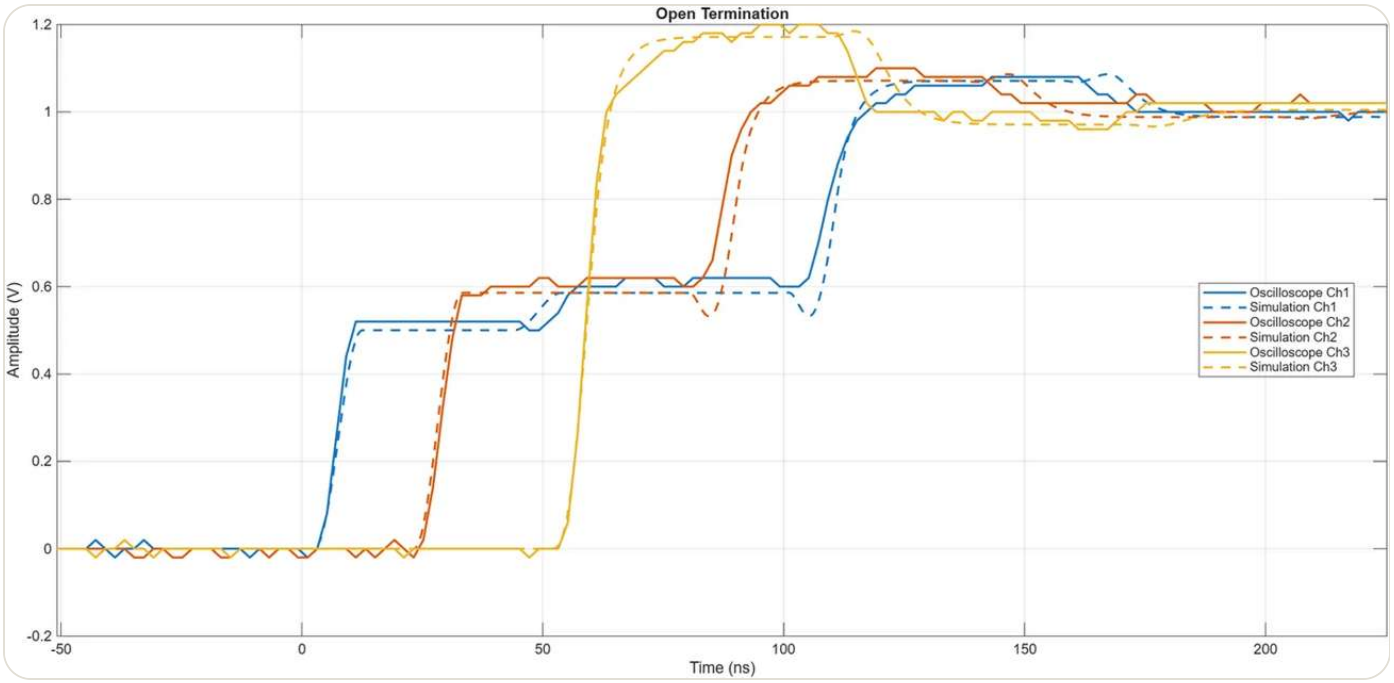
RF CIRCUIT DESIGN · LAB 2

Reflections on Transmission Lines

E157 Lab 2 · Fall 2026

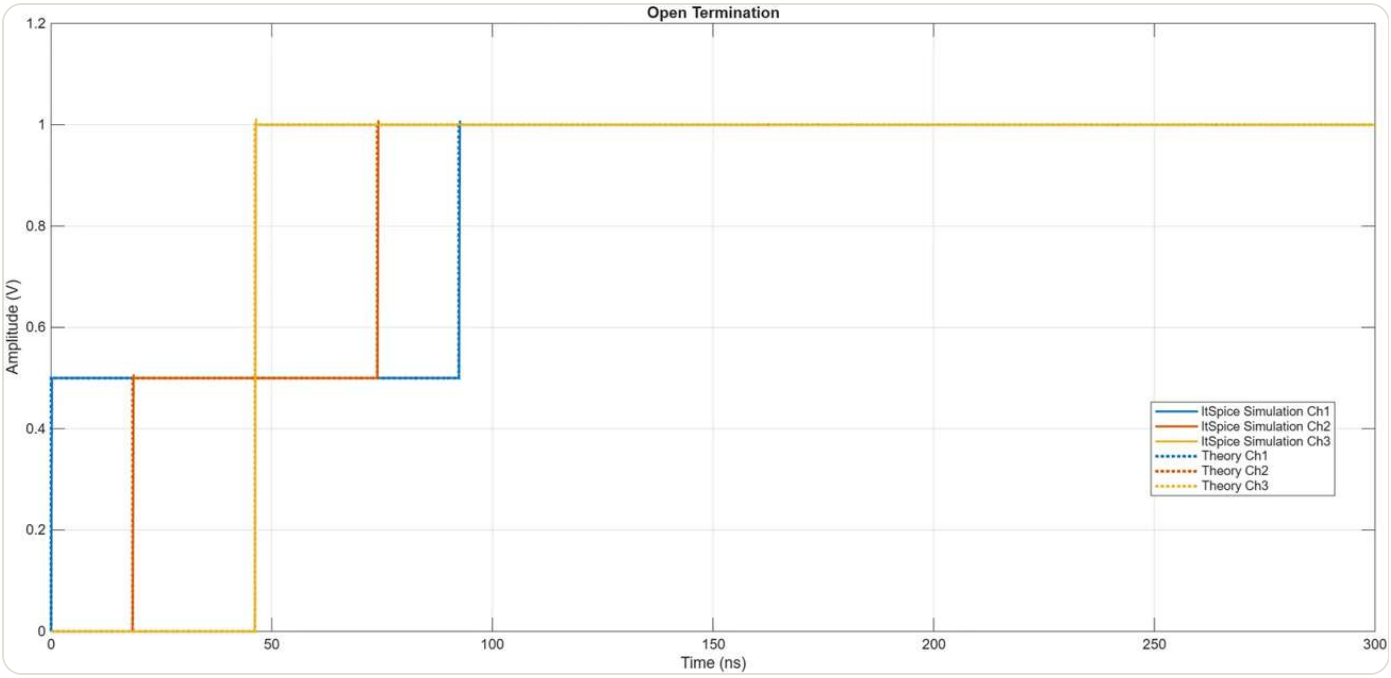
Predicting step responses along a chain of BNC cables in Python, then measuring them on an oscilloscope and fitting an LTspice model with extracted parasitics until simulation and measurement agree.

Transmission lines | Oscilloscope | LTspice | Python | Parasitic extraction



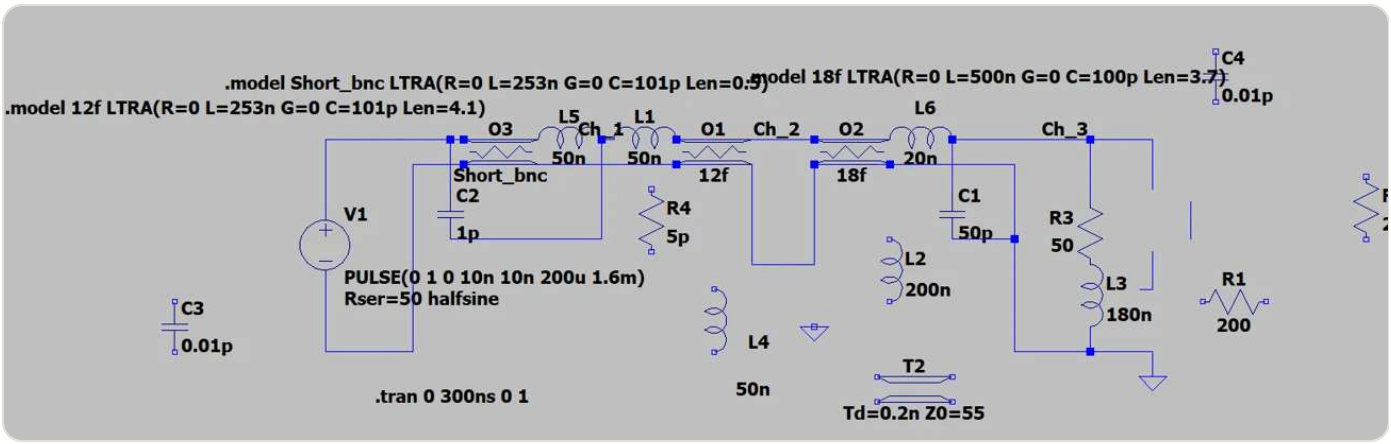
Goal: predict the step response at three points along a chain of 50 Ω BNC cables for open, short, matched, 22 Ω and 200 Ω terminations, then build each case and match it in simulation.

I wrote a Python model that tracks each incident and reflected wave, using $\Gamma = (Z_L - Z_0) / (Z_L + Z_0)$ and a delay of 1.54 ns per foot for 0.66c cable. For a shunt resistor partway down the line with a mismatched load, the wave bounces between the two discontinuities, so the model loops until the voltage settles at the DC voltage-divider value.

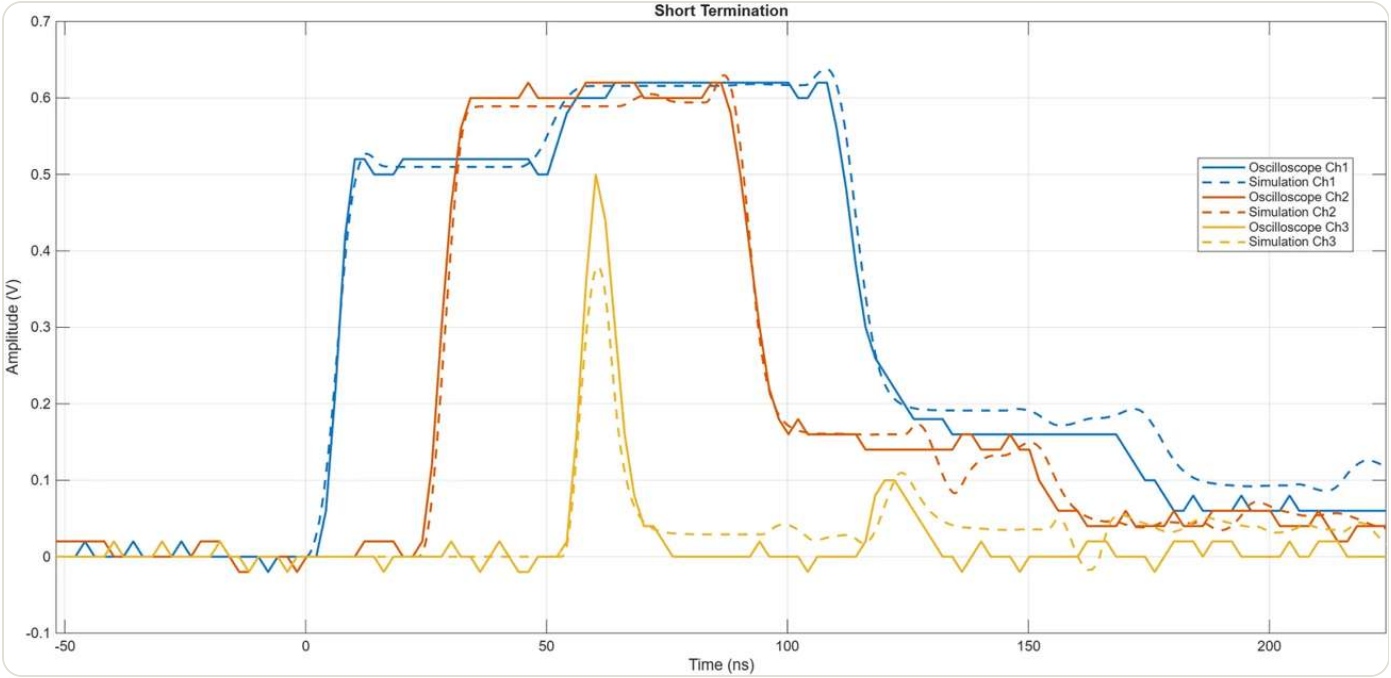


Python theory vs. ideal LTspice for an open termination, at all three channels

The real measurements had slower edges, overshoot and some loss. To explain them I added parasitics to the LTspice model one at a time: series inductance at the connectors, shunt capacitance at the probes, and short sections of line for the BNC tees. With those in place the simulation follows the oscilloscope closely.



LTspice model with the parasitic elements I extracted

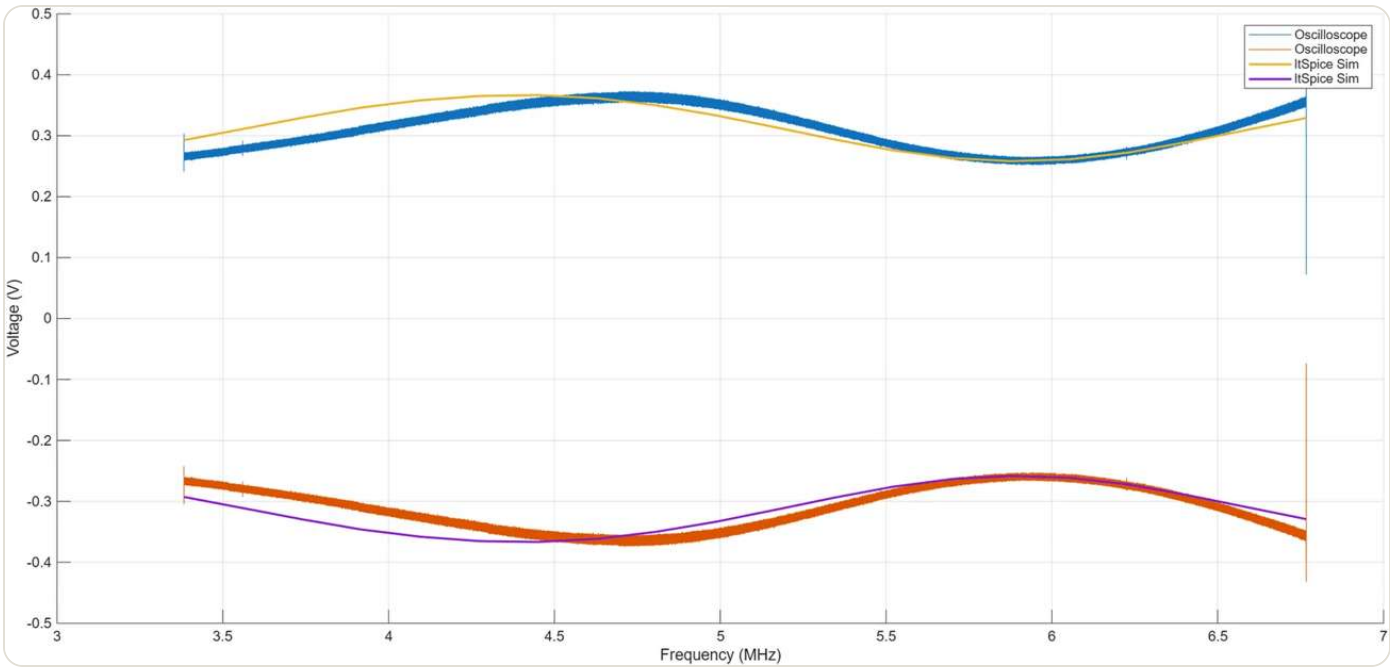


Short termination: measured vs. simulated

I also used the measurements to work backwards. With a 200 Ω termination, the reflected step implied an effective load of about 128 Ω at the edge, which pointed to parasitic capacitance at the resistor and connector.

Standing waves

For a 22 Ω load I predicted a VSWR of 2.27 and swept a sine wave from 3.4 to 6.8 MHz, the range that spans one maximum and one minimum at the probe point. Fitting a lossy line model to the measured sweep lined the simulation up with the oscilloscope.



Standing wave amplitude vs. frequency: oscilloscope against lossy LTspice model

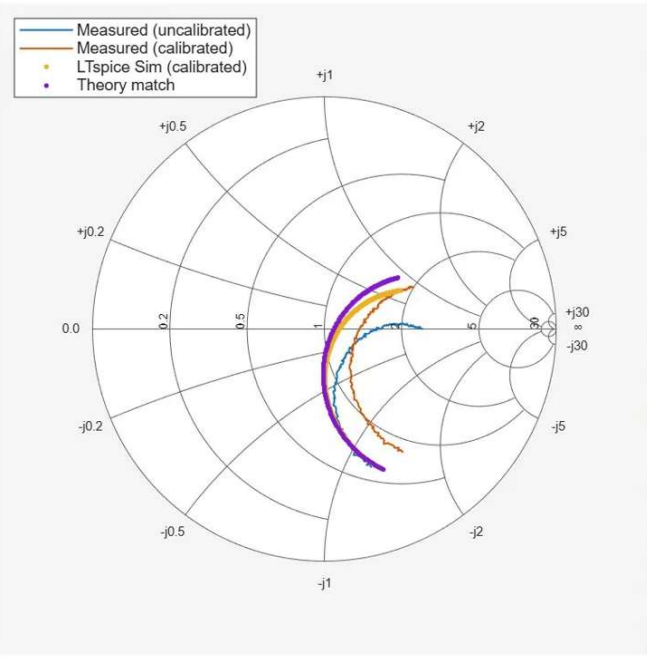
RF CIRCUIT DESIGN · LAB 3

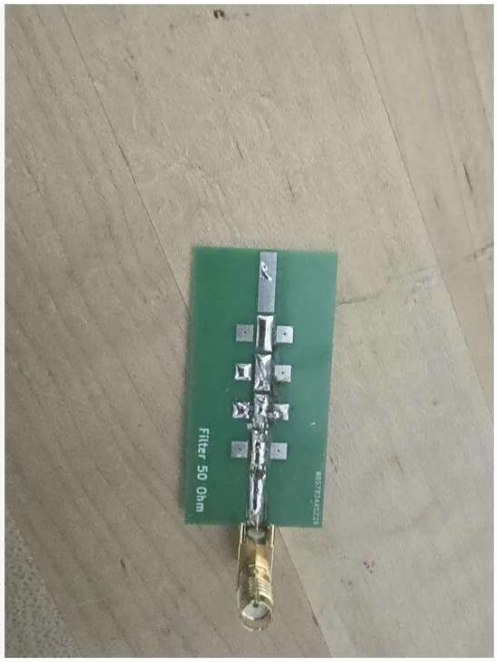
VNA Calibration & L-Match

E157 Lab 3 · Fall 2026

Calibrating a vector network analyzer, then designing and hand-soldering an L-match from 270 Ω to 50 Ω at 100 MHz, checking the Smith chart after every part. Measured Q of 2.77 against 2.34 in theory.

VNA · Smith chart · Impedance matching · LTspice · MATLAB · SMD soldering

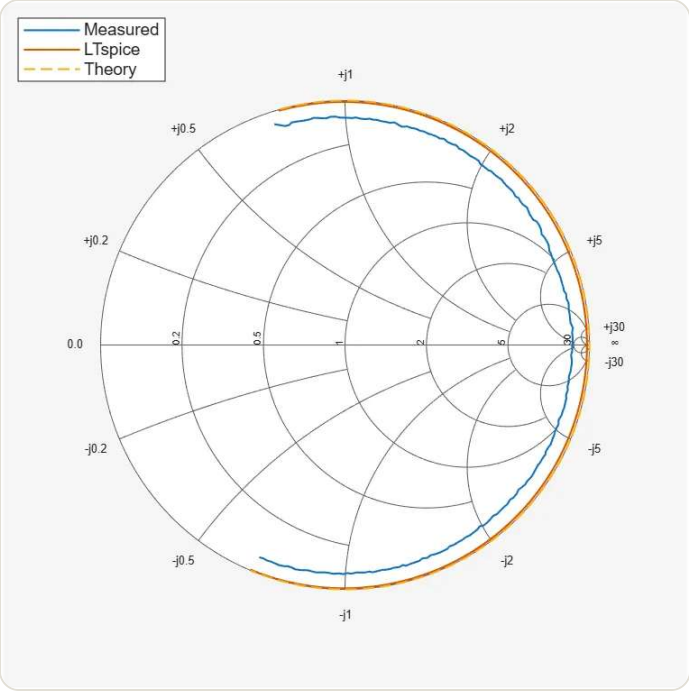




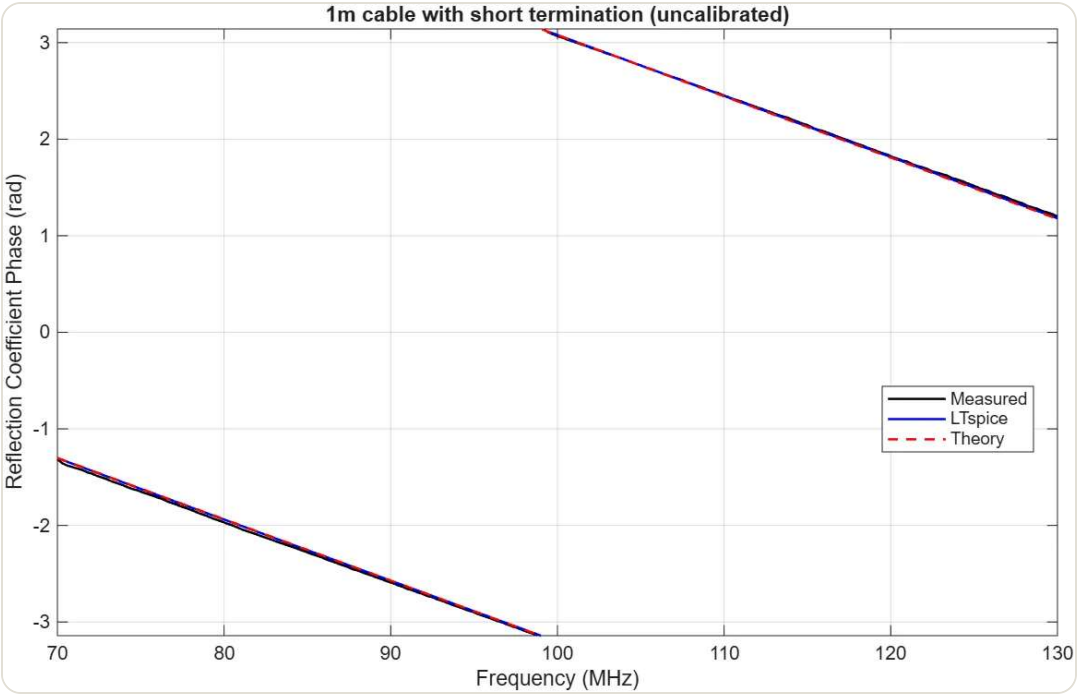
Goal: match a 270 Ω load to a 50 Ω line at 100 MHz with an L-network, built component by component on a surface-mount test board.

Calibrating the VNA

Before calibration the VNA measures everything between its internal port and the load, including a meter of coax. That cable adds phase that grows with frequency, so a plain short circuit sweeps around the edge of the Smith chart instead of sitting at one point. Cable loss also pulls the magnitude below 1, and more so at higher frequency because of skin effect and dielectric loss. My measurement, LTspice simulation and hand analysis all agreed once I used the lab cable’s velocity factor of 0.66.

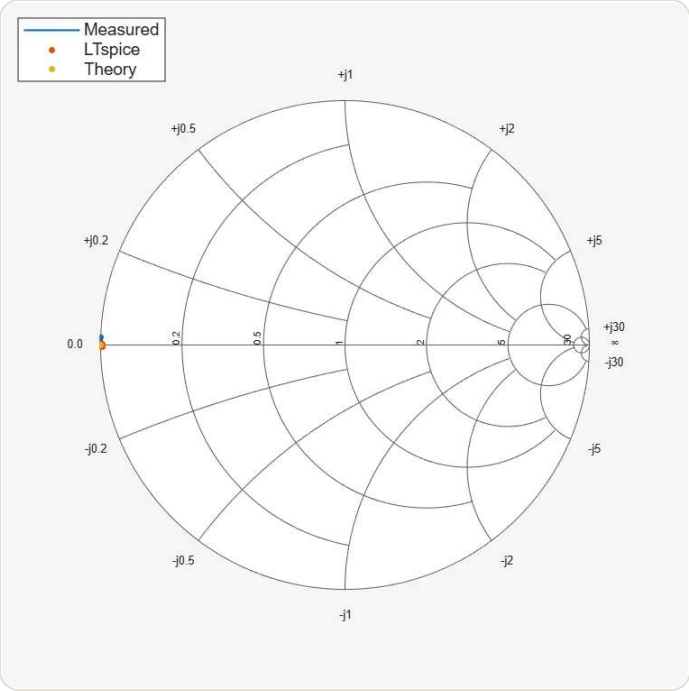


Uncalibrated: a 1 m cable with a short circuit, measured against LTspice and theory



Uncalibrated reflection phase wrapping across 70–130 MHz

An open/short/load calibration solves for the error terms at every frequency point. After it, the short collapses to $\Gamma = -1$, the open to +1 and the 50 Ω load to the center. Changing the span to 90–110 MHz without recalibrating smeared the points out again slightly, a good reminder that a calibration only holds for the frequencies it was taken at.

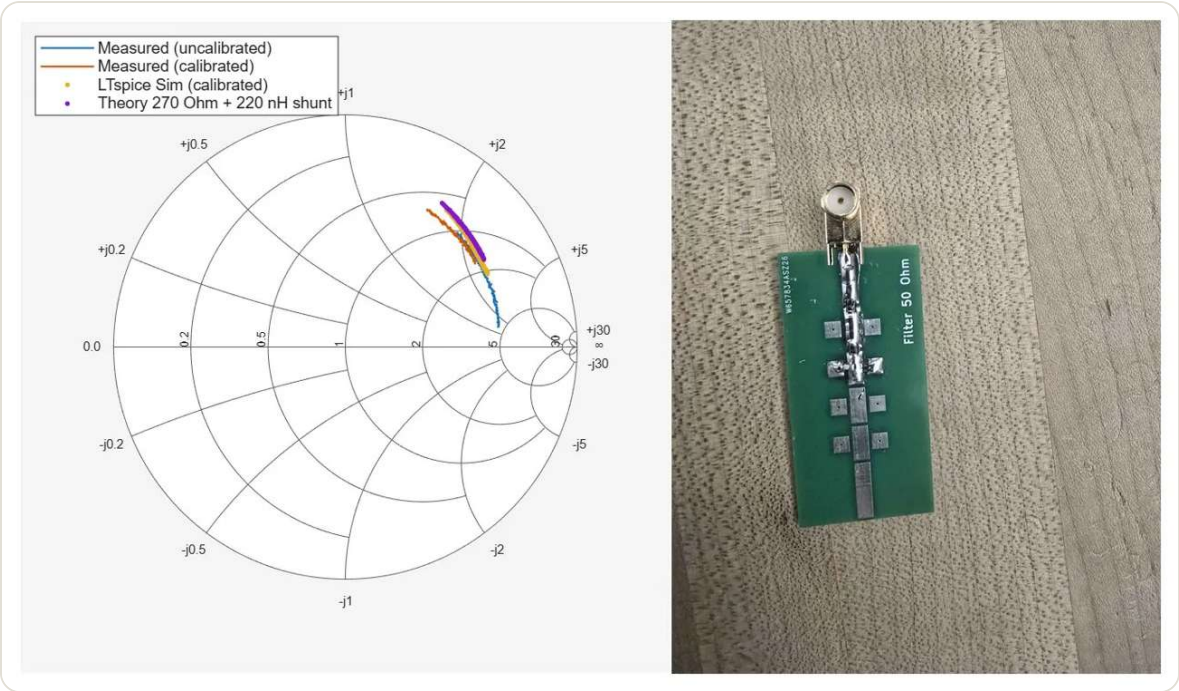


Calibrated: the same short now sits at a single point, Γ = −1

Building the match one part at a time

My 270 Ω load was a 330 Ω and a 1.5 kΩ resistor in parallel. Its measured reflection at 100 MHz pointed to about 257 Ω instead, so I redesigned around the measured value rather than the nominal one.

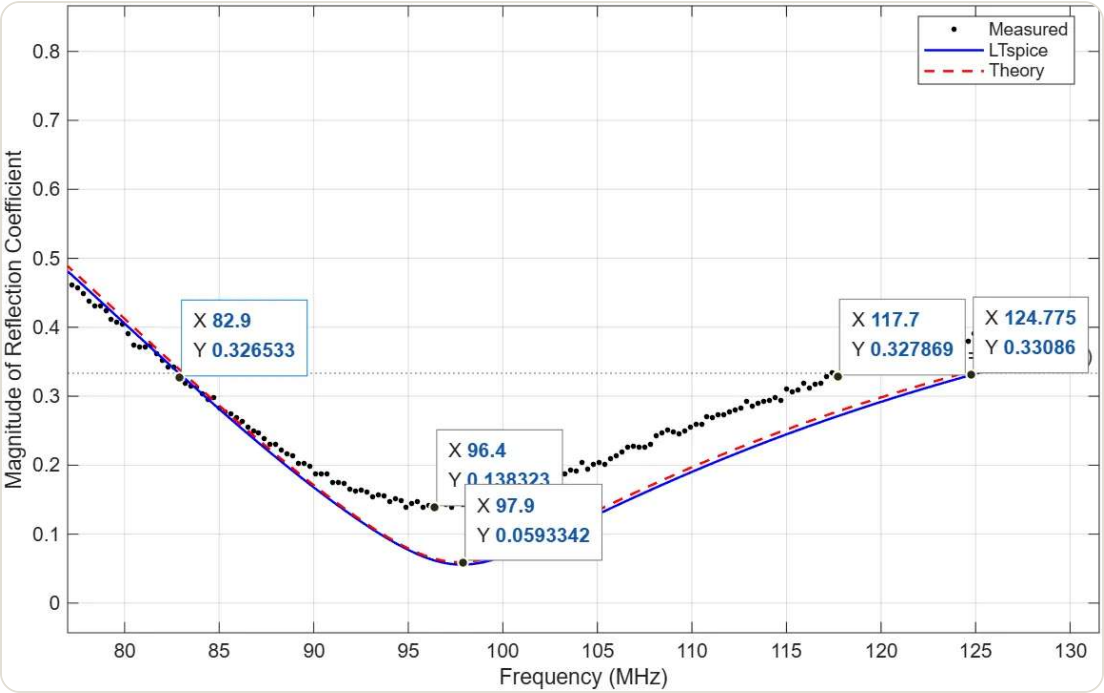
I soldered the shunt inductor first and measured again. The closest part to my calculated 204 nH was 220 nH, which put the load at 57.7 + j107 Ω. From that measured point I recalculated the series capacitor, $C = 1 / (2\pi \cdot 107\ \Omega \cdot 100\ \text{MHz}) \approx 14.9\ \text{pF}$, and used 15 pF. Checking the Smith chart after every part is how you get a good match from hand-assembled boards.



After the first part: the 220 nH shunt inductor moves the load up the chart

Results

The best match landed at 96.4 MHz. Using a $|\Gamma| = 1/3$ bandwidth from 82.9 to 117.7 MHz, the measured Q was **2.77**, against **2.34** from my model. I traced the 15% difference to part tolerances, the inductance of the 0 Ω jumpers, and some unmodeled parasitic capacitance: the low band edge matched exactly, but the high edge came in early, which is what extra capacitance would do.



Measured |Γ| vs. frequency with the markers used to compute Q

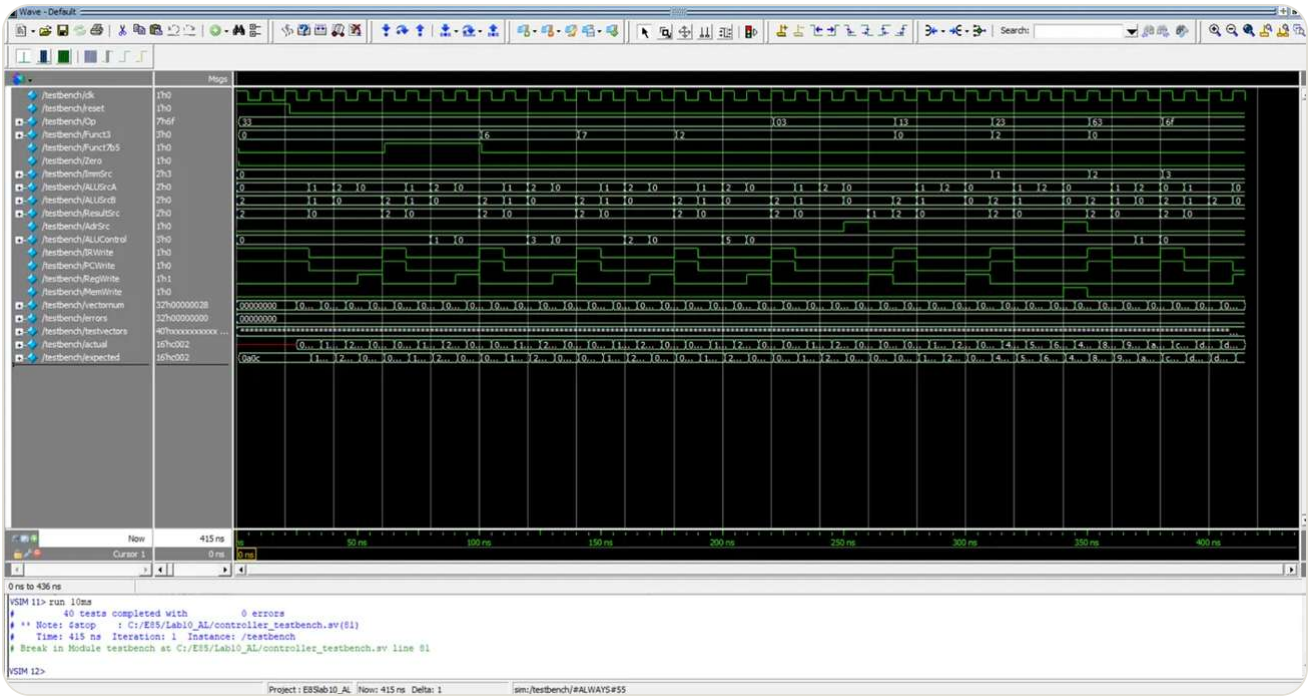
MULTICYCLE RISC-V PROCESSOR · LAB 10

Multicycle Controller

E85 Lab 10

The control unit for a multicycle RISC-V processor in hierarchical SystemVerilog, a main state machine plus ALU and instruction decoders, passing all 40 test vectors.

SystemVerilog RISC-V FSM QuestaSim



The controller reads the opcode and function fields of each instruction and drives every select and enable line in the datapath. It supports `lw`, `sw`, R-type arithmetic (`add`, `sub`, `and`, `or`, `slt`), I-type arithmetic, `beq` and `jal`.

Structure

I followed the hierarchy of the reference design: a top-level controller wires together three submodules. Don’t-care outputs are set to 0, so every signal has a known value during testing.

```
module controller(input logic clk,
  input logic reset,
  input logic [6:0] op,
  input logic [2:0] funct3,
  input logic funct7b5,
  input logic zero,
  output logic [1:0] immsrc,
  output logic [1:0] alusrca, alusrcb,
  output logic [1:0] resultsrc,
  output logic adrsrc,
  output logic [2:0] alucontrol,
  output logic irwrite, pcwrite,
  output logic regwrite, memwrite);

  logic [1:0] ALUOp;
  logic Branch;
  MainFSM Main(
  .clk(clk),.reset(reset),.zero(zero),.op(op),.irwrite(irwrite),.adrsrc(adrsrc),
  .alusrcb(alusrcb),.alusrca(alusrca),.resultsrc(resultsrc),.ALUOp(ALUOp),.Branch(Branch),
  .memwrite(memwrite),.pcwrite(pcwrite),.regwrite(regwrite)
   );
   ALUDecoder ALU(
    .funct3(funct3),.funct7b5(funct7b5),.ALUOp(ALUOp),.alucontrol(alucontrol)
   );
   InstrDecoder Instr(
     .op(op),.immsrc(immsrc)
   );

endmodule
```

Top-level controller module

Main state machine

The main FSM steps through fetch and decode, then branches to the right execute states for each instruction type: memory address, memory read and write-back for loads, memory write for stores, ALU write-back for arithmetic, and so on. Each state sets the multiplexer selects and write enables for that step.



```
module MainFSM(
  input logic clk, reset,
  input logic [6:0] op,
  input logic zero,
  output logic irwrite, adrsrc,
  output logic [1:0] alusrca, alusrcb,
  output logic [1:0] resultsrc, ALUOp,
  output logic Branch, memwrite,
  output logic pcwrite, regwrite
  );

typedef enum logic [3:0]{
Fetch, Decode, MemAdr, MemRead, MemWB,MemWrite,
ExecuteR, ALUWB, ExecuteI, BEQ, JAL} statetype;
statetype state, nextstate;
always_ff @(posedge clk, posedge reset)
if (reset) state <= Fetch;
else state <= nextstate;

//Fig 2
always_comb begin
  case (state)
  Fetch: nextstate = Decode;
  Decode: begin
  if (op==7'b0000011||op==7'b0100011) begin
  nextstate=MemAdr;
  end
  else if (op==7'b0110011) begin
  nextstate=ExecuteR;
  end
  else if (op==7'b0010011) begin
  nextstate=ExecuteI;
  end
  else if (op==7'b1100011) begin
  nextstate=BEQ;
  end
  else if (op==7'b1101111) begin
  nextstate=JAL;
  end
  else begin nextstate=Fetch;
  end
end
MemAdr: begin
      if (op==7'b0000011)
      nextstate=MemRead;
      else
      nextstate=MemWrite;
      end
      MemRead: nextstate=MemWB;
      MemWB: nextstate=Fetch;
      MemWrite: nextstate=Fetch;
      ExecuteR: nextstate=ALUWB;
      ALUWB: nextstate=Fetch;
      ExecuteI: nextstate=ALUWB;
      JAL: nextstate=ALUWB;
      default:nextstate=Fetch;
      endcase
end

always_comb begin
irwrite = 0; adrsrc = 0;alusrca = 2'b00; alusrcb =2'b00;
ALUOp = 2'b00; resultsrc = 2'b00; irwrite =0;
regwrite=0; Branch=0; memwrite=0;pcwrite=0;
```

Main FSM state register and next-state logic

Decoders

The ALU decoder turns `ALUOp`, `funct3` and `funct7` bit 5 into the 3-bit ALU control, telling `add` from `sub` and picking `slt`, `or` and `and`. The instruction decoder picks the immediate format for each opcode.

```
module ALUDecoder (
  input logic [2:0] funct3,
  input logic funct7b5,
  input logic [1:0] ALUOp,
  output logic [2:0] alucontrol
);

always_comb begin
   case (ALUOp)
   2'b00: begin
   alucontrol=3'b000;//add
   end
   2'b01: begin
   alucontrol=3'b001;//sub
   end
   2'b10: begin
      if (funct3==3'b000) begin
      if (funct7b5==1)
         alucontrol=3'b001;//sub
         else
         alucontrol=3'b000; //add
         end
   else if (funct3==3'b010&& funct7b5==0) begin
   alucontrol=3'b101; //slt
   end
   else if (funct3 == 3'b110&& funct7b5==0) begin
   alucontrol=3'b011;//or
   end
   else if (funct3==3'b111 && funct7b5==0) begin
   alucontrol=3'b010; //and
   end
   else begin
   alucontrol=3'b000; //add
   end
 end
default: alucontrol=3'b000;
endcase
end
endmodule
```

ALU decoder

```
module InstrDecoder(
input logic [6:0] op,
output logic[1:0] immsrc);
always_comb begin
    immsrc=2'b00;
    if (op==7'b0110011) begin
    immsrc=2'b00;//R-type
    end
    else if (op==7'b0010011) begin
    immsrc=2'b00;//I-type
    end
    else if (op==7'b0000011) begin
    immsrc=2'b00;//lw
    end
    else if (op==7'b0100011) begin
    immsrc=2'b01;//sw
    end
    else if (op==7'b1100011) begin
    immsrc=2'b10;//beq
    end
    else if (op==7'b1101111) begin
    immsrc=2'b11;//jal
    end
    else begin
    immsrc=2'b00;
    end
  end
endmodule
```

Instruction decoder for the immediate format

Testing

I ran the controller in Questa against the provided testbench and vectors. All 40 tests passed with 0 errors, so the controller was ready to drop into the full processor in Lab 11.

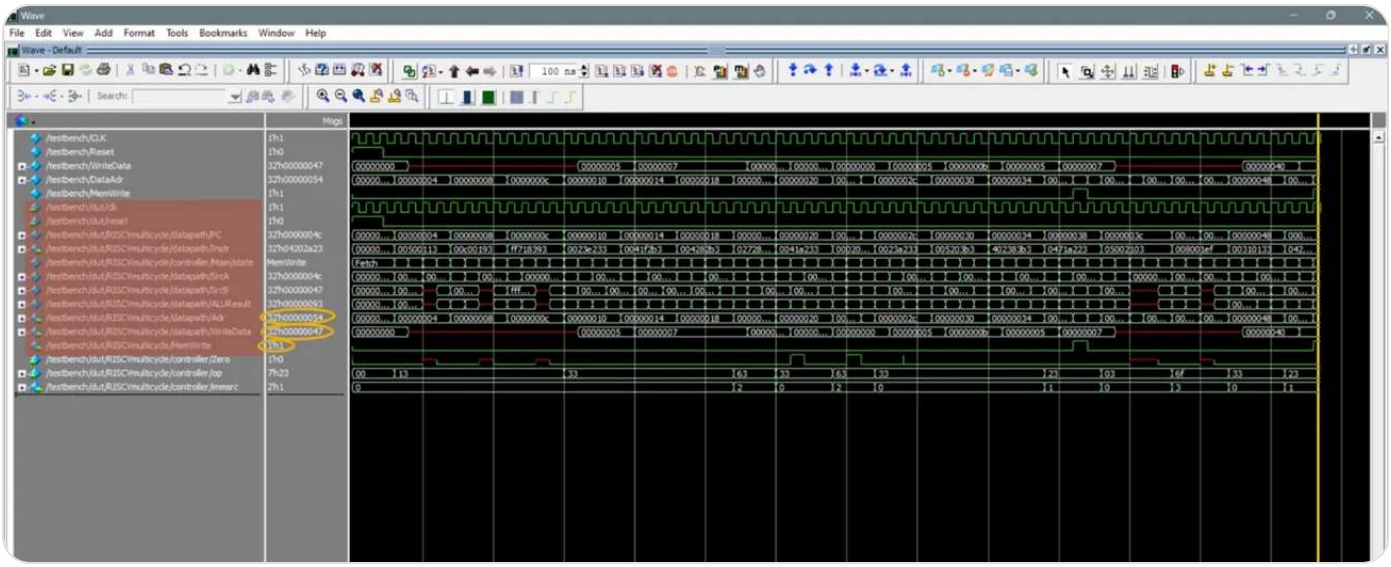
MULTICYCLE RISC-V PROCESSOR · LAB 11

Datapath & Full Processor

E85 Lab 11

The datapath for the multicycle RISC-V processor, connected to the Lab 10 controller and a shared instruction/data memory, then verified by tracing a test program cycle by cycle.

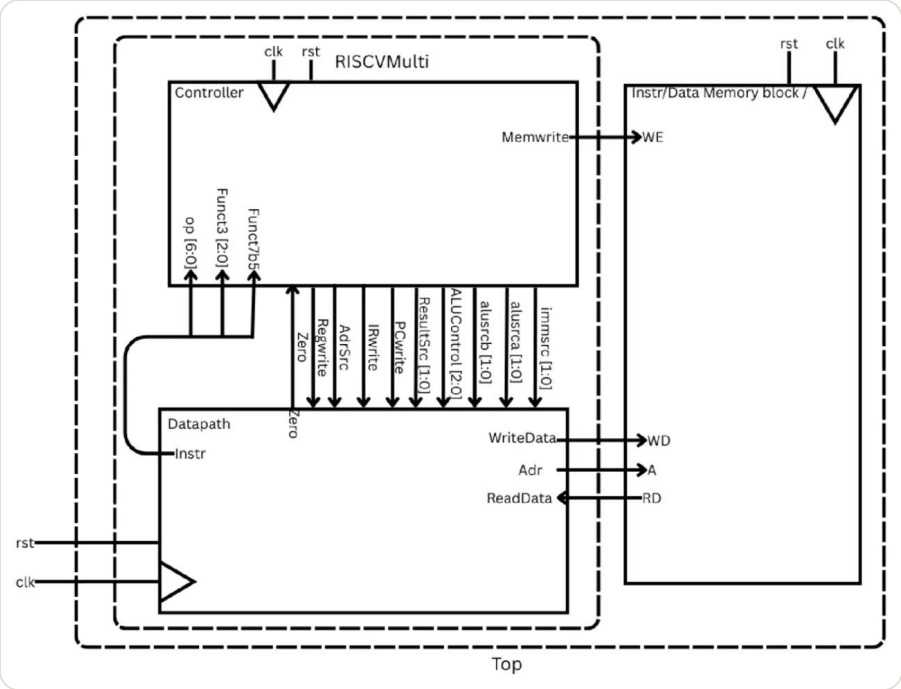
SystemVerilog RISC-V Datapath Debugging QuestaSim



With the controller done, this lab adds the datapath and wraps both in a top-level module with one memory shared by instructions and data, the way a multicycle design expects.

System design

The controller takes `op`, `funct3`, `funct7` bit 5 and `Zero` from the datapath and sends back every select and enable. The datapath holds the program counter, instruction and data registers, the register file, the ALU and the multiplexers between them.



Top-level block diagram of the controller, datapath and memory

```
module RISCVmulticycle(input logic clk, reset,
    input logic [31:0] ReadData,
    output logic [31:0] A,WriteData,
    output logic MemWrite);

 logic RegWrite, IRWrite, PCWrite, AdrSrc, Zero;
  logic [1:0] ALUSrcA,ALUSrcB,ResultSrc,ImmSrc;
  logic [2:0] ALUControl;
  logic [31:0] Instr;

controller controller(.clk(clk),.reset(reset),.op(Instr[6:0]),.funct3(Instr[14:12]),.funct7b5(Instr[30]),.Zero(Zero),
.irwrite(IRWrite),.adrsrc(AdrSrc),.alusrca(ALUSrcA),.alusrcb(ALUSrcB),.resultsrc(ResultSrc),.memwrite(MemWrite), .pcwrite(PCWrite),
.regwrite(RegWrite),.alucontrol(ALUControl),.immsrc(ImmSrc));

datapath datapath(.clk(clk),.reset(reset),.ResultSrc(ResultSrc),.ALUControl(ALUControl),.ALUSrcA(ALUSrcA),.ALUSrcB(ALUSrcB),
.ImmSrc(ImmSrc),.RegWrite(RegWrite),.PCWrite(PCWrite),.AdrSrc(AdrSrc),.IRWrite(IRWrite),.WriteData(WriteData),
.ReadData(ReadData),.Adr(A),.Zero(Zero),.Instr(Instr));
endmodule
```

The processor module connecting the controller and datapath

```
module datapath(input logic clk, reset,
    input logic [2:0] ALUControl,
    input logic [1:0] ALUSrcA,ALUSrcB,ResultSrc,ImmSrc,
    input logic RegWrite,PCWrite,AdrSrc,IRWrite,
    input logic [31:0] ReadData,//From Instr/Data Memory
    output logic [31:0] WriteData,
    output logic [31:0] Adr,
    output logic [31:0] Instr,
    output logic Zero
);

logic [31:0] PCNext,ImmExt,SrcA,SrcB,Result,RD1,RD2,ALUOut,OldPC,Data,A,PC,ALUResult;
flopenr #(32) pcflop(.clk(clk), .reset(reset), .en(PCWrite), .d(PCNext), .q(PC));
assign PCNext = Result;
mux2 #(32) FstMux(.d0(PC), .d1(Result), .s(AdrSrc), .y(Adr));//Adr=DataAdr=>dmem

flopenr #(32) uplatchflop(.clk(clk),.reset(reset),.en(IRWrite),.d(PC), .q(OldPC));
flopenr #(32) downlatch(.clk(clk), .reset(reset), .en(IRWrite), .d(ReadData), .q(Instr));
flopr #(32) datareg(.clk(clk), .reset(reset), .d(ReadData), .q(Data));
RegFile RegFile(.clk(clk),.WE3(RegWrite),.A1(Instr[19:15]),.A2(Instr[24:20]),.A3(Instr[11:7]),.WD3(Result),.RD1(RD1),.RD2(RD2));
Extend Ext(.Instr(Instr[31:7]), .ImmSrc(ImmSrc), .ImmExt(ImmExt));

flopr #(32) RD1Flop(.clk(clk),.reset(reset),.d(RD1), .q(A));
flopr #(32) RD2Flop(.clk(clk),.reset(reset), .d(RD2), .q(WriteData));
mux3 #(32) AMux(.d0(PC), .d1(OldPC), .d2(A),.s(ALUSrcA),.y(SrcA));
mux3 #(32) BMux(.d0(WriteData),.d1(ImmExt), .d2(32'd4), .s(ALUSrcB),.y(SrcB));

alu alu(.a(SrcA),.b(SrcB),.alucontrol(ALUControl),.result(ALUResult),.Zero(Zero));
flopr #(32) LastOutFlop(.clk(clk),.reset(reset),.d(ALUResult),.q(ALUOut));
mux3 #(32) LastOutMux(.d0(ALUOut),.d1(Data),.d2(ALUResult),.s(ResultSrc),.y(Result));

endmodule
```

Datapath: registers, multiplexers, register file and ALU

Because the memory is shared, I moved the `$readmemh` that loads the program from the instruction memory into the combined memory module.

Writing down what should happen

Before simulating, I traced the test program by hand: for every cycle, the PC, the instruction, the state the controller should be in, and the result it should produce. Having that table made it quick to spot the first cycle where the waveform disagreed with my expectations, which is where every bug hunt starts.

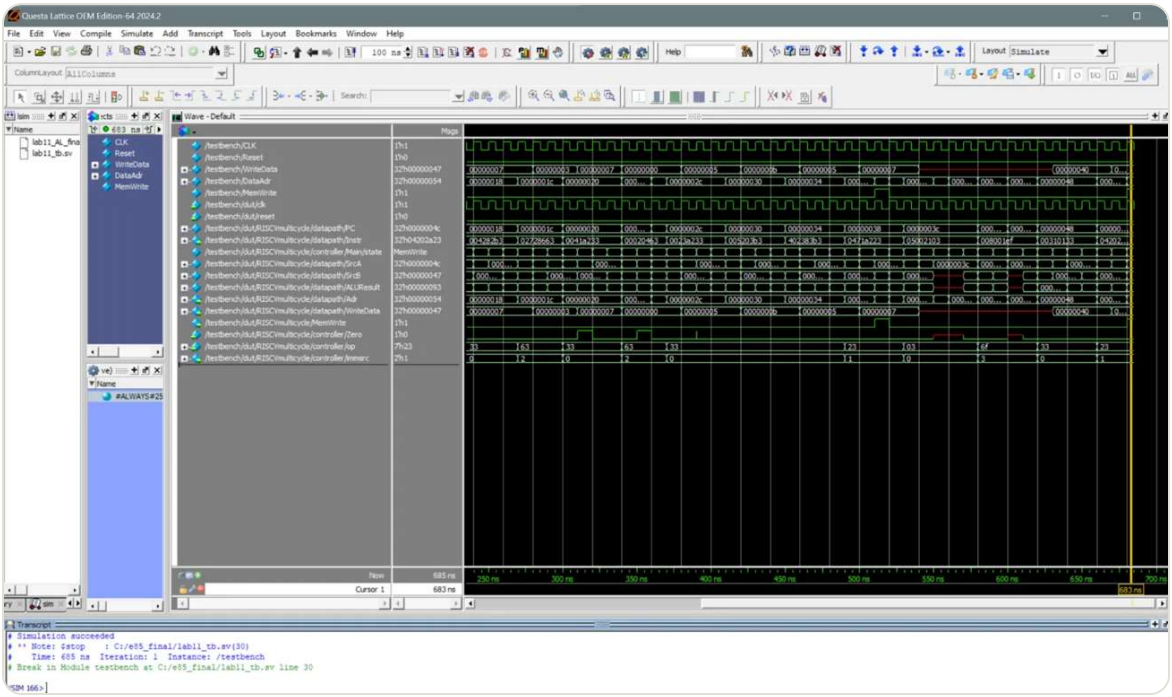
Table 1: Expected Operation (after two cycles of reset)

Step	PC	Instr	State	Result	Result Notes
3	00	00500113	S0: Fetch	4	PC+4
4	04	""	S1: Decode	X	OldPC+Immediate
5	04	""	S8: ExecuteI	X	ALUResult = x0 (0) + 5 = 5
6	04	""	S7: ALUWB	5	Result = ALUOUT
7	04	""	S0: Fetch	8	PC+4
8	08	00c00193	S1: Decode	X	OldPC+Immediate
9	08	" "	S8:ExecuteI	X	ALUResult = x0 + 12 = 12
10	08	" "	S7:ALUWB	12(c)	write 12 to x3
11	08	" "	S0:Fetch	12(c)	PC + 4 = 12
12	0C	ff7183	S1: Decode	X	oldPC + Imm
13	0C	" "	S8:ExecuteI	X	x3 + (-9) = 3
14	0C	" "	S7:ALUWB	3	result = 3
15	0C	" "	S0: fetch	16	PC+4 = 12+4 = 16
16	10	0023e2 33	S1: Decode	X	3 or 5 = 7
17	10	" "	S6: Execute R	X	
18	10	" "	S7: ALUWB	7	
19	10	" "	S0: fetch	20	
20	14	0041f2 b3	Decode	X	

Expected operation for the first instructions after reset, completed by hand

Results

The full simulation runs the program to the end. The final check is the store: when `MemWrite` goes high, the processor writes `0x47` (71) to address `0x54`, exactly what the program expects.



Full simulation in Questa, ending with "Simulation succeeded"

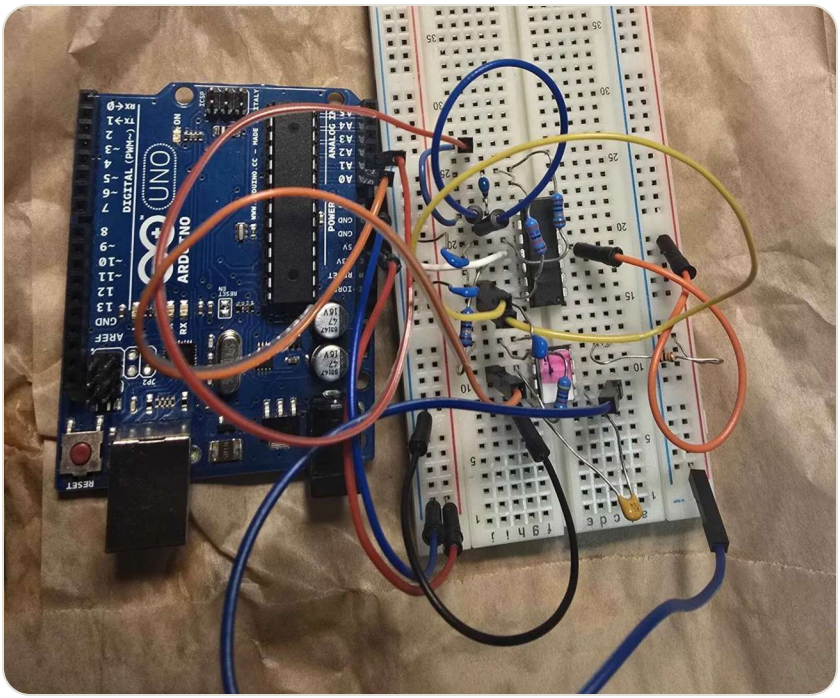
CIRCUIT DESIGN · E101 PROJECT

Analog & Digital Filters

E101 final project · Fall 2024 · with Leran Hu and Betsy Wu

Turning a 10 Hz pulse train into a clean 10 Hz sine wave with a 4th-order Sallen-Key Butterworth filter on a breadboard followed by a modified comb filter running on an Arduino, checked harmonic by harmonic against the predicted response.

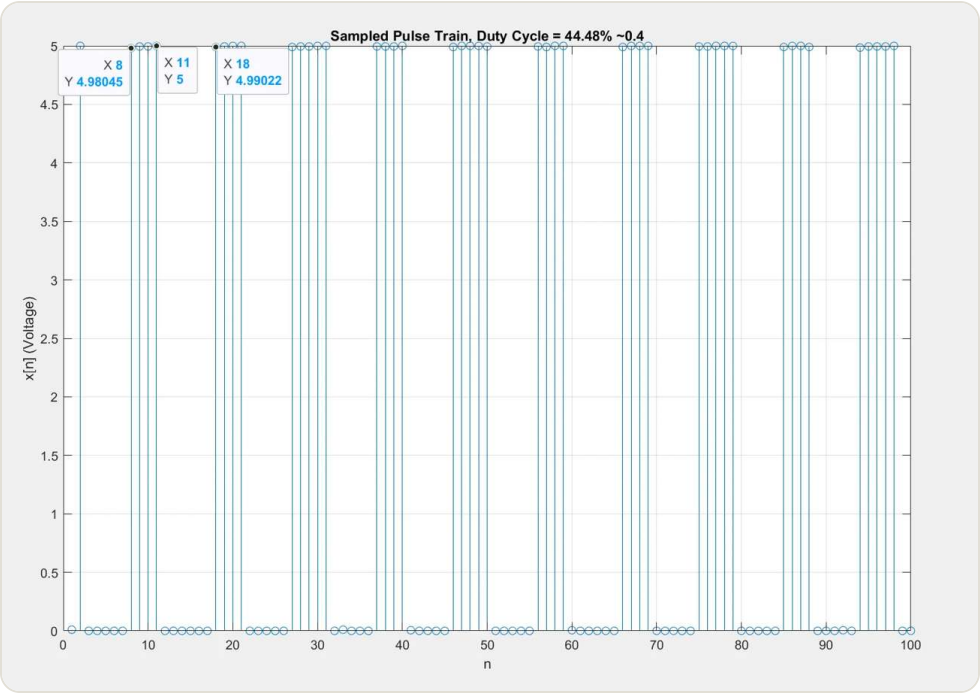
Analog filters | Sallen-Key | Digital filters | Fourier analysis | MATLAB | Arduino



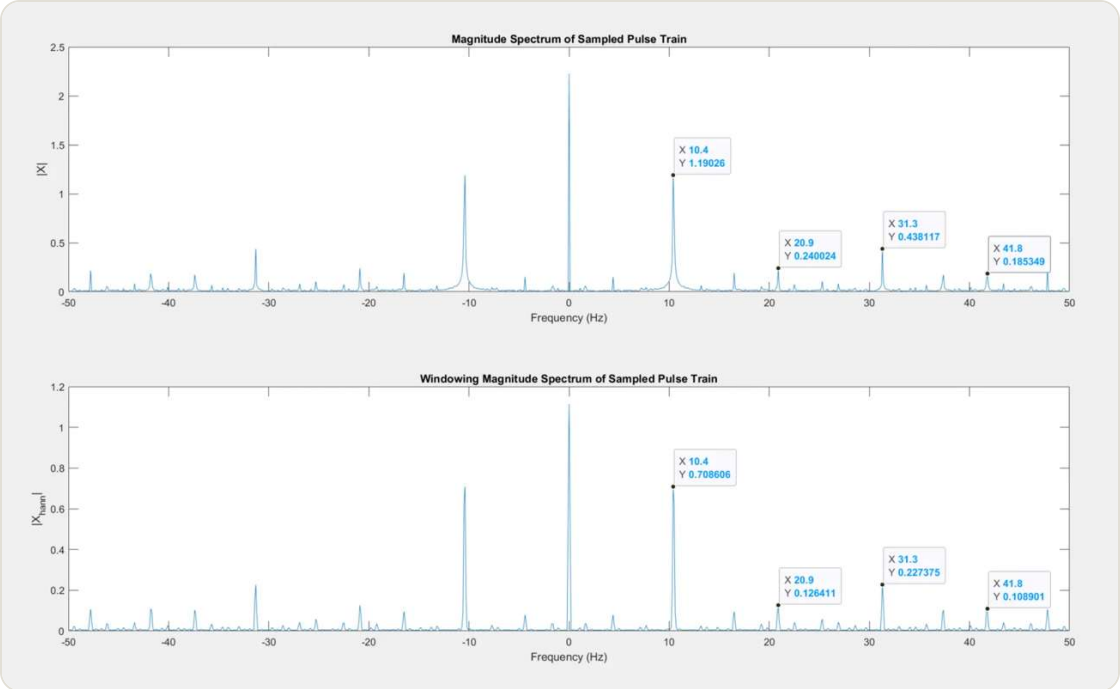
Goal: start from a 10 Hz square-ish pulse train from an oscillator chip and end with a 10 Hz sinusoid, using one analog stage and one digital stage, and show with Fourier analysis that each stage does what its math says.

Characterizing the input

We sampled the oscillator at 100 Hz on an Arduino. From the mean and peak of the samples, its duty cycle was 44.48%. A 1000-point FFT with a Hann window to limit spectral leakage gave the first four harmonics, and they lined up with the analytical Fourier series coefficients for that duty cycle.



The first 100 samples of the pulse train

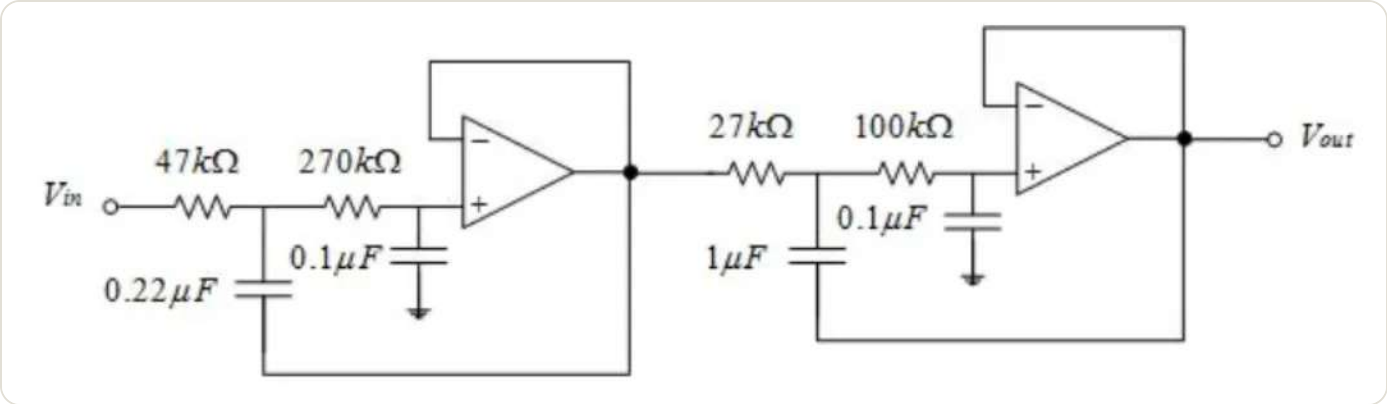


FFT of the pulse train, without (top) and with (bottom) the Hann window

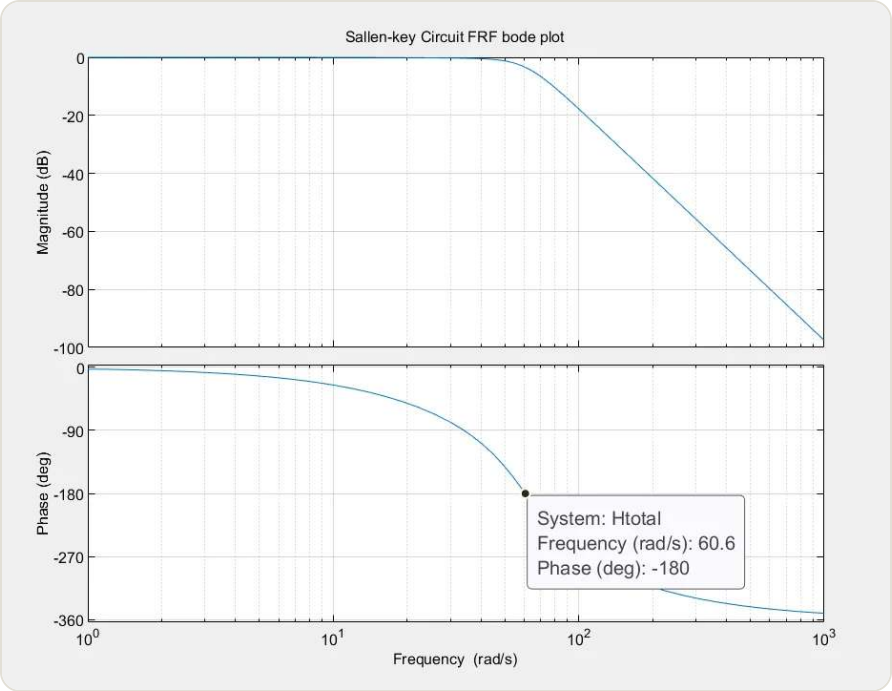
Stage 1: analog Butterworth filter

Two cascaded Sallen-Key stages make a 4th-order Butterworth low-pass with poles evenly spaced on the left half-plane. We picked standard resistor values (47 kΩ, 270 kΩ, 27 kΩ

and 100 kΩ) that put the cutoff at 10 Hz, and the Bode plot of the real component values gave a −3 dB point of 9.93 Hz.

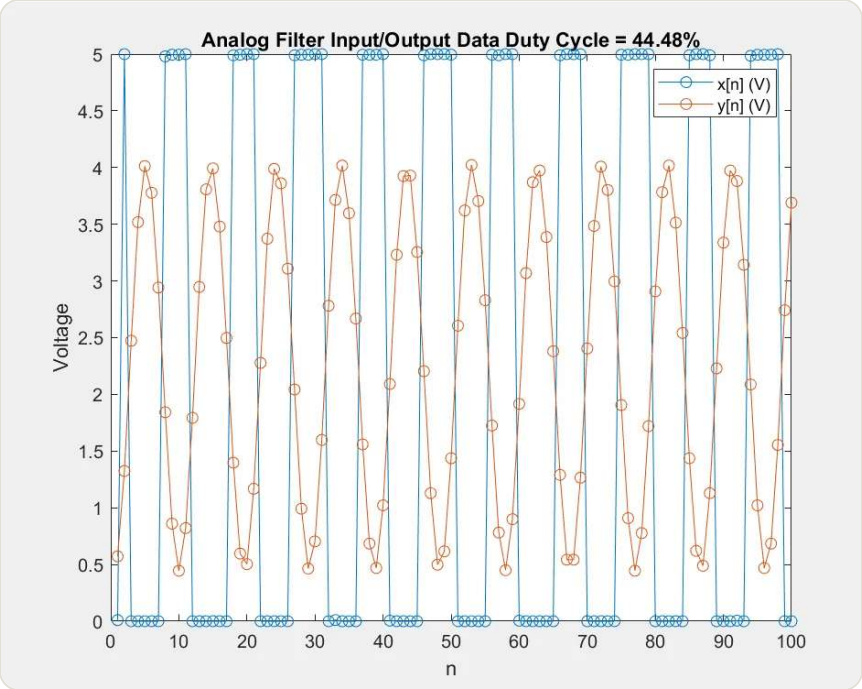


Two-stage Sallen-Key filter with the component values we used

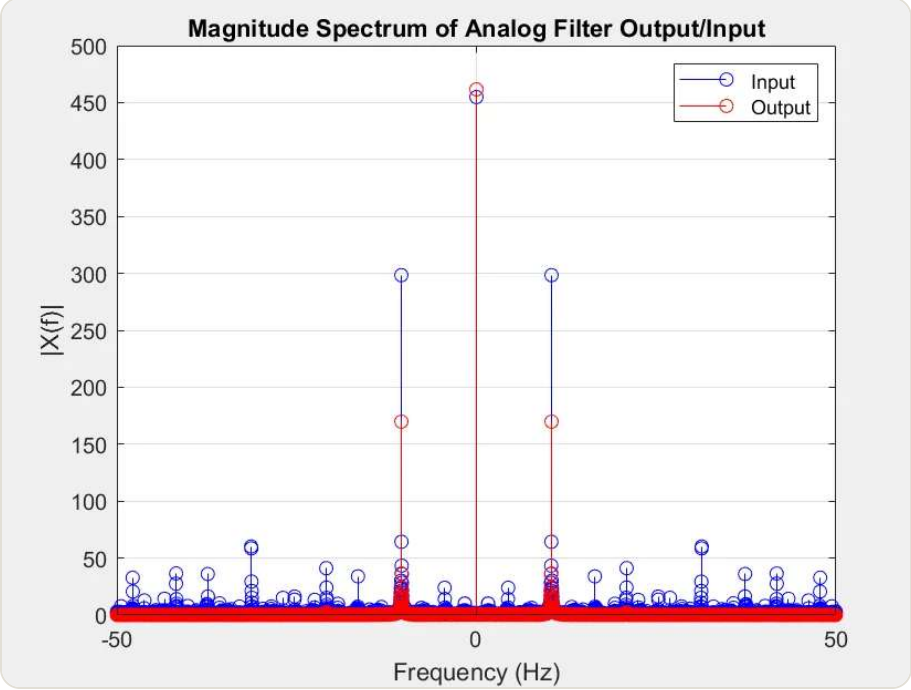


Bode plot of the filter, with the cutoff marked at 60.6 rad/s

The filter rounds the pulse train into a sine wave with a small phase shift. Measured harmonic by harmonic, it kept the 10 Hz fundamental and cut the 20, 30 and 40 Hz harmonics by more than 15×, in line with the predicted transfer function.



Analog filter input (blue) and output (orange)



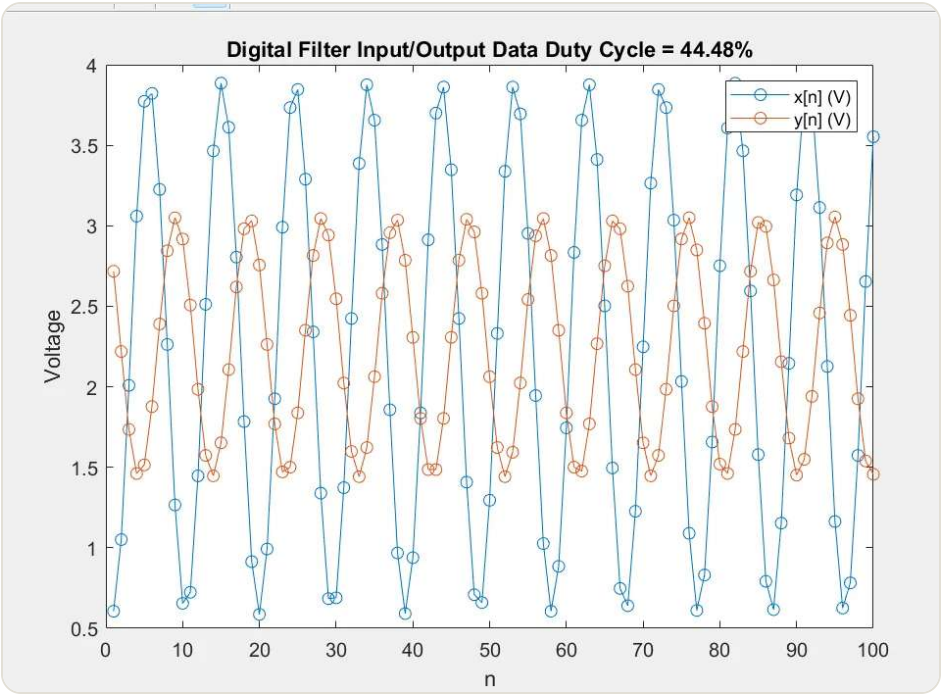
Fourier coefficients of the analog filter's input and output

Stage 2: modified comb filter on the Arduino

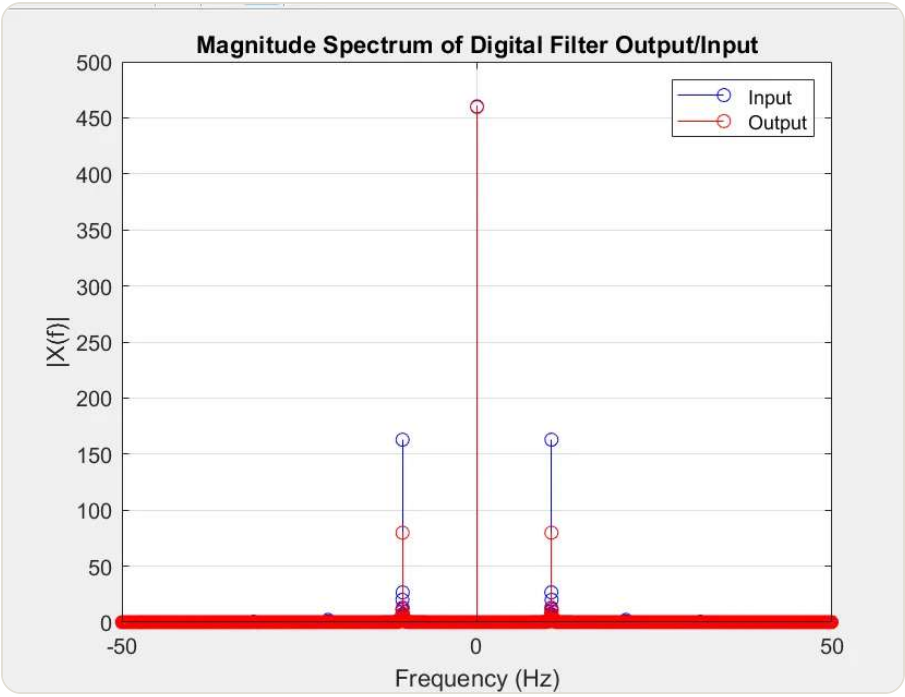
A comb filter puts zeros evenly around the unit circle. Removing the zeros at z = 1 and z = $e^{\pm j\pi/5}$ opens passbands at DC and at 10 Hz, and scaling the gain to 1 at DC gives the final

filter. The Arduino runs its difference equation in real time at each sample.

After the comb filter, only the 10 Hz coefficient is left. The 2nd and 3rd harmonics drop to about 1% of their input level, and the 4th, already tiny after the analog stage, to about 11%.



Digital filter input (blue) and output (orange)



Fourier coefficients of the digital filter's input and output

Sources of error

Small 20 and 30 Hz residuals got through the analog stage, which is expected for a 4th-order filter. In the digital stage, quantization and finite precision on the Arduino caused small amplitude and phase differences, and component tolerances and electrical noise accounted for the rest.

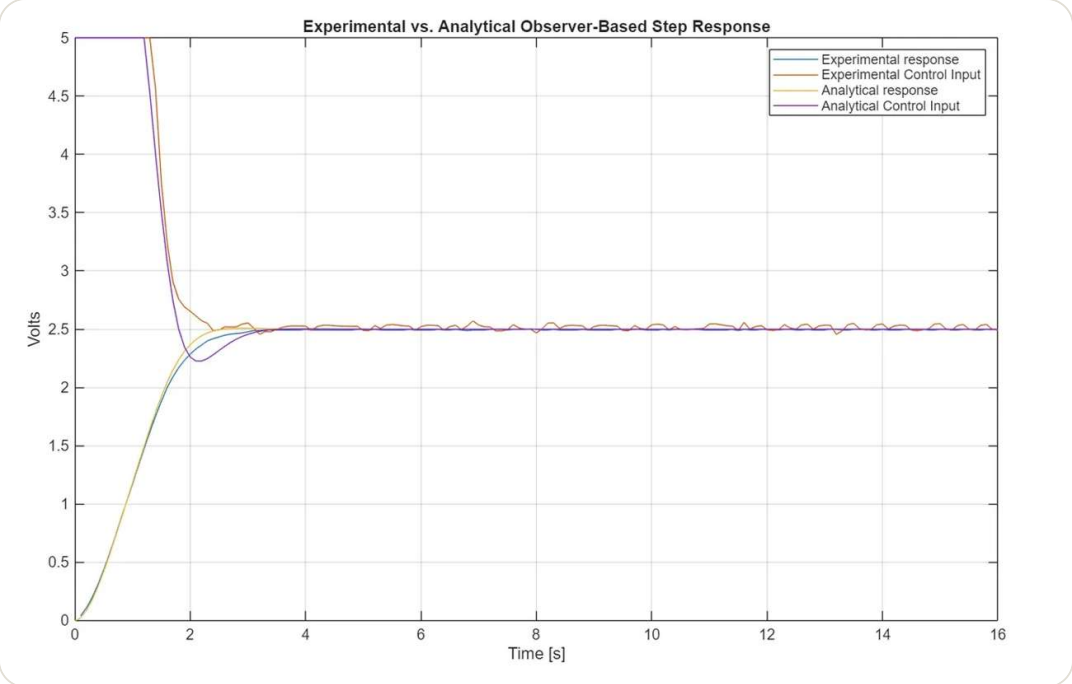
SIGNAL PROCESSING & CONTROL · PROJECT

Digital Controller for an RC Circuit

E102 final project · Spring 2025 · with Sweny

Two digital controllers for a second-order RC op-amp circuit, a tuned PI loop and an observer-based LQR state feedback, designed in MATLAB and Simulink and run on an Arduino at 10 Hz. The observer design settled in about 3 s with under 0.4% overshoot.

Digital control · LQR · State observer · MATLAB · Simulink · Arduino · C++

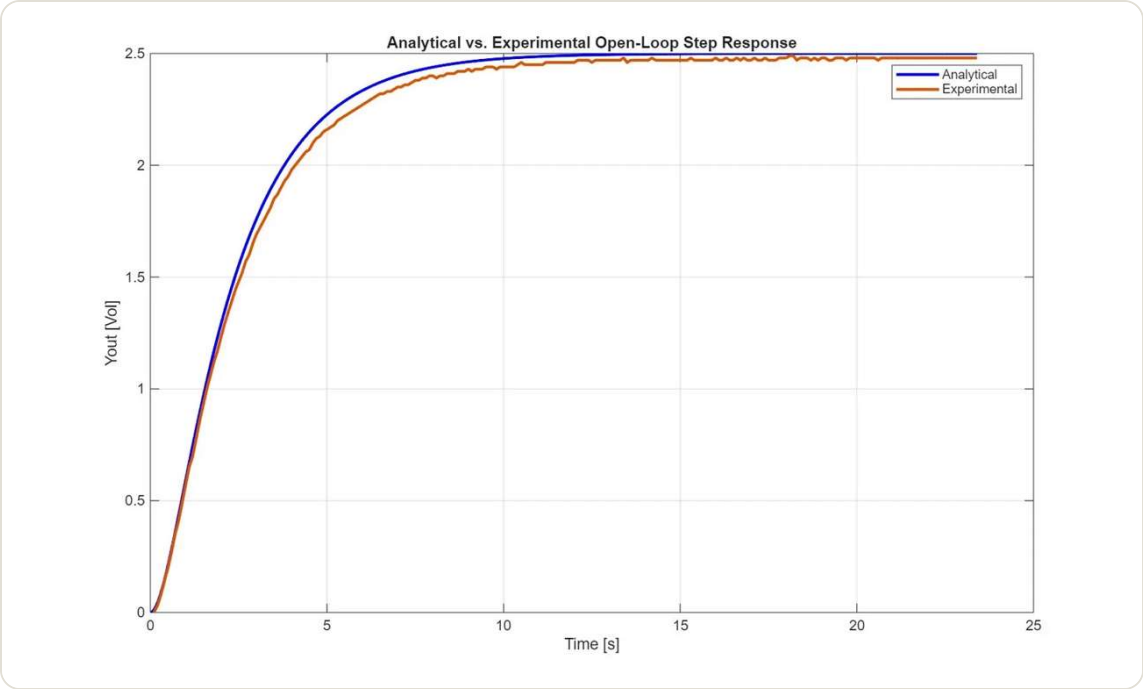


Goal: drive an overdamped second-order RC circuit to a 2.5 V step with less than 1% overshoot, zero steady-state error, a 1% settling time under 4 s and as little control effort as possible, sampling at 10 Hz with the input limited to 0–5 V.

Modeling the plant

The circuit is two RC stages (R = 50 kΩ, C = 10 µF) around an LMC6484 op amp. We derived its transfer function, H(s) = 1 / ((0.5s + 1)(2s + 1)), the state-space model, and the analytical step response. Then we built the circuit and drove it from an Arduino Uno,

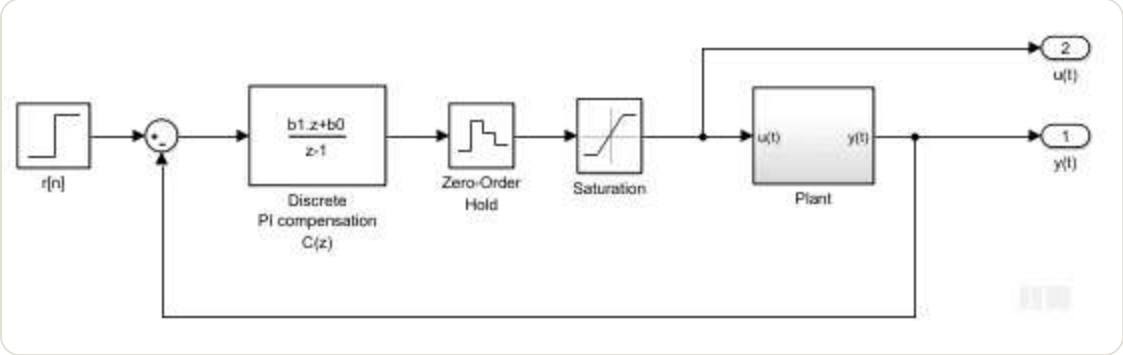
reading the output on the ADC and setting the input with PWM. The measured open-loop response lined up with the analytical one.



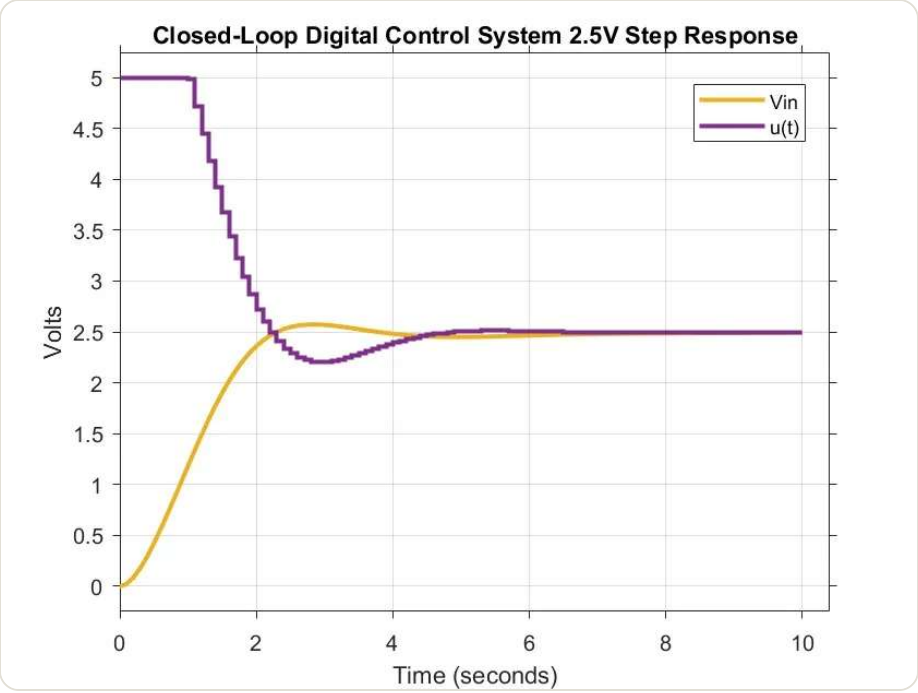
Open-loop step response: analytical vs. measured

Controller 1: digital PI

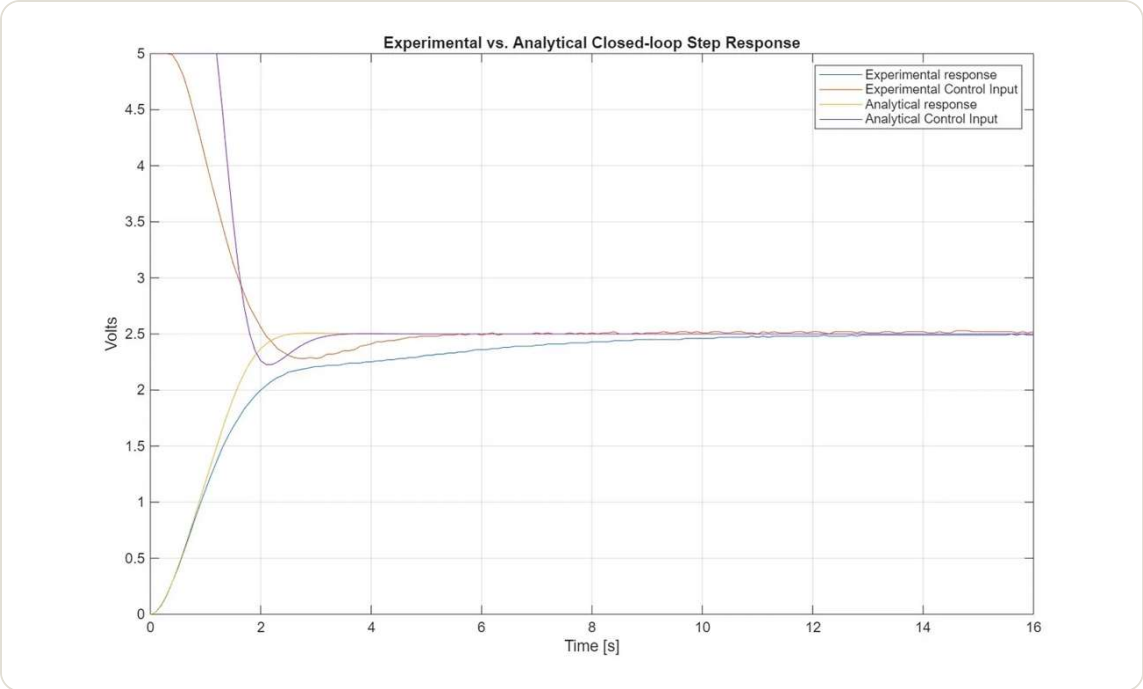
We designed a PI compensator for a 70° phase margin at a 1 rad/s crossover by solving the phase and magnitude conditions in MATLAB, which gave Kp = 2.35 and Ki = 0.86. Discretized with backward differencing, it becomes a two-term difference equation the Arduino runs each sample: $u[n] = u[n-1] + b_0 e[n] + b_1 e[n-1]$.



Simulink model of the digital PI loop with zero-order hold and 0–5 V saturation



Simulated closed-loop response and control input



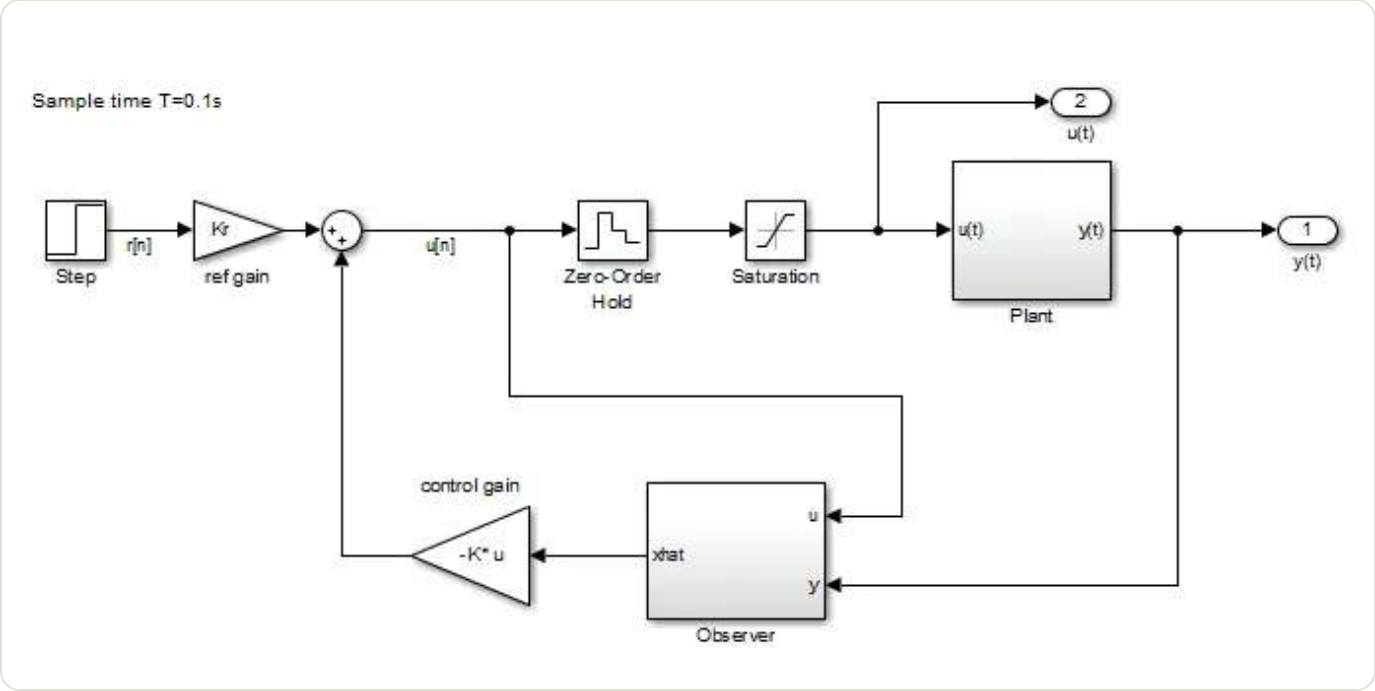
Measured vs. simulated PI response

We swept phase margin and crossover frequency on the real circuit. Some trends matched theory: raising the crossover frequency shortened settling, and a 90° phase margin at 1 rad/s drives Ki to zero, which explained the steady-state error we saw there. Overshoot stayed at zero for nearly every setting, but the best settling time we measured was about

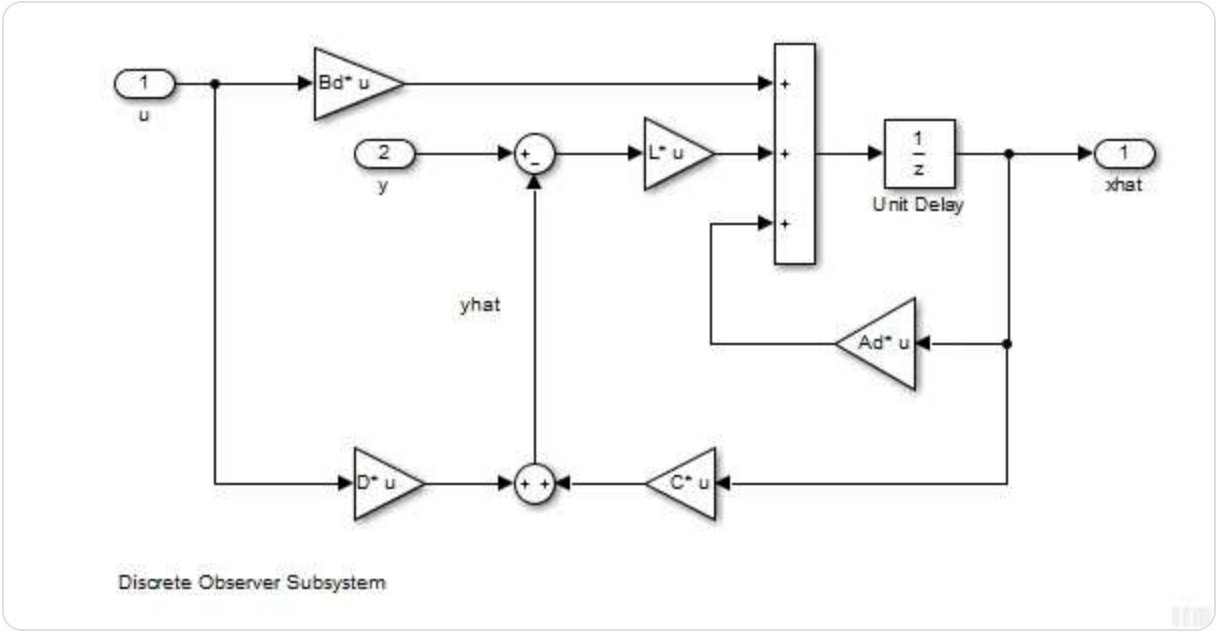
5.9 s (52°, 6 rad/s). The output reached 2.3–2.4 V quickly and then crept the rest of the way to 2.5 V.

Controller 2: observer-based state feedback

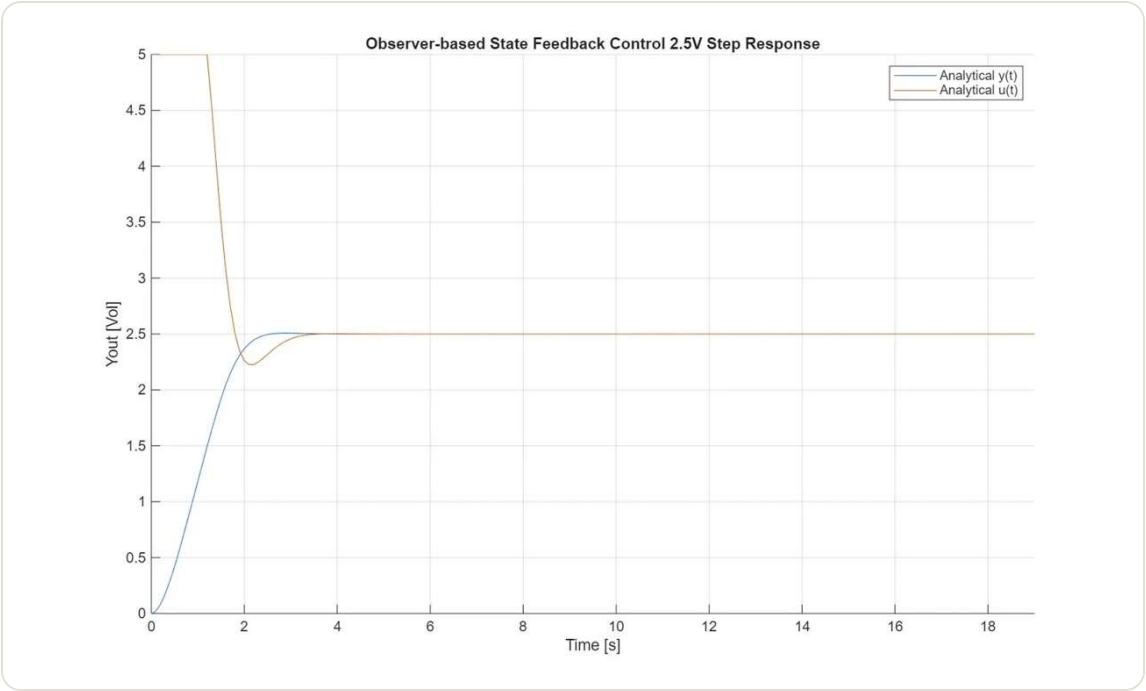
We discretized the plant with a zero-order hold at T = 0.1 s and designed full state feedback with discrete LQR (`dlqr`), weighting output error over the second state with Q = diag(100, 1) and R = 1. A reference gain Kr gives zero steady-state error, and since only the output is measured, an observer estimates both states, with its poles placed at twice the speed of the closed-loop poles.



Simulink model of the observer-based controller



Discrete observer subsystem



Simulated response and control input

On the circuit this controller met every spec. My MATLAB script sweeps Q and recomputes K and Kr so we could compare trials quickly:

Q_{11}	Q_{22}	1% settling	Overshoot	Steady-state error
100	1	3.07 s	0.36%	0.003 V
100	10	3.98 s	0.2%	0.005 V
200	1	2.82 s	0.36%	0.005 V
50	10	5.18 s	0%	0.006 V

Raising Q_{11} pushes the poles left and shortens settling at the cost of a little more control effort. Raising Q_{22} trades speed for less actuation and less peaking.

Why simulation and hardware differ

The circuit was consistently slower and more damped than the model. The main reasons:

- **Part values.** We used a 47 kΩ resistor in place of 50 kΩ, on top of 5% tolerances, plus pF-level parasitic capacitance.
- **Measurement noise.** The Uno's ADC read about ±0.02 V of noise at steady state, which the controller multiplies when it computes the next input.
- **Idealizations.** The model assumes an ideal op amp and a smooth input voltage, while the Uno actually outputs 980 Hz PWM.

Integral action in the PI loop, and a little retuning on the hardware, made the controllers robust to those differences.